Dokumentasi Resmi & Sintaks Java

Bahasa: Java | Total Chunk/Dokumen: 229 | Disiapkan untuk RAG System

[java_001] Java - Installation & Setup - macOS

macOS PKG installer packages: Overview

Sumber: ht

= macOS PKG installer packages
files, which can be run with an interactive user interface or run
silently from the Terminal command line.

[java_002] Java - Installation & Setup - macOS

macOS PKG installer packages: GUI installation

Sumber: ht

Instructions for running an interactive installation using the macOS PKG
installer:
. Download the `.pkg` file.
. Navigate to the folder that contains the file and open it to launch
the installation program or drag the icon to your Application folder.
. The Introduction screen indicates the target location for the
installation, which you can change later in the install process. Click
Continue.
. Read the license, click Continue and accept the license if you are
happy with the terms.
. Change the target location for the installation. Click Install to
complete the installation.

[java_003] Java - Installation & Setup - macOS

macOS PKG installer packages: Command-line installation

Sumber: ht

A silent installation allows you to install the macOS package without
user interaction, which can be useful for widescale deployment. You must
have administrator privileges. Follow these steps
. Enter the Administrator password
. Temurin installs to
`/Library/Java/JavaVirtualMachines/temurin-<version>.<jdk|jre>/`

Contoh Kode / Perintah CLI:

```
installer -pkg <path_to_pkg>/<pkg_name>.pkg -target /
```

[java_004] Java - Installation & Setup - Windows

Windows MSI installer packages: Overview

Sumber: ht

// Some portions generated by Co-Pilot
= Windows MSI installer packages
.msi files, which can be run with an interactive user interface or run
silently from the command line. The installer is designed for use on a
per-machine basis, not per-user basis, so you can have only one
installation of the MSI on a machine for all users.

Note: Windows installer packages are supported only on Windows x64 systems.

[java_005] Java - Installation & Setup - Windows

Windows MSI installer packages: GUI installation

Sumber: ht

Instructions for running an interactive installation using the Windows MSI installer
the following features, which you can deselect, if necessary
. When you have chosen the features that you want to install, click Next.
. Click Install to begin the installation.
. When the installation is finished, click Finish to close the program.

[java_006] Java - Installation & Setup - Windows

Windows MSI installer packages: Command-line installation

Sumber: ht

A silent installation allows you to install the Windows package with pre-selected features without user interaction, which can be useful for wide scale deployment. Follow these steps
[width="100%",cols="50%,50%",options="header",]
|===
|Feature |Description
|`FeatureMain` |Core Temurin installation (DEFAULT)
|`FeatureEnvironment` |Update the PATH environment variable (DEFAULT)
|`FeatureJarFileRunWith` |Associate .jar files with Java applications (DEFAULT)
|`FeatureJavaHome` |Update the JAVA_HOME environment variable
|`FeatureOracleJavaSoft` |Updates registry keys HKLM
|===
+
Note
[cols=",",options="header",]
|===
|Parameter |Features
|`INSTALLLEVEL=1` |FeatureMain,FeatureEnvironment,FeatureJarFileRunWith
|===
. Run the command on the target workstation. The following example silently installs Temurin, updates the PATH, associates .jar files with Java applications and defines JAVA_HOME
+
The following example silently installs all the features for `INSTALLLEVEL=1`
+
+
For a list of supported codes, see the https://github.com/adoptium/installer/blob/master/wix/Lang/LanguageList.config[Language list].

Contoh Kode / Perintah CLI:

```
msiexec  /i  <package>.msi  ADDLOCAL=FeatureMain,FeatureEnvironment,FeatureJarFileRunWith,FeatureJavaHome
INSTALLDIR="c:\Program Files\Temurin\" /quiet
```

```
# ----------------------------------------

msiexec /i <package>.msi INSTALLLEVEL=1 /quiet

# ----------------------------------------

msiexec /i <package>.msi INSTALLLEVEL=1 TRANSFORMS=:1031
```

[java_007] Java - Installation & Setup - Windows

Windows MSI installer packages: Reinstalling or upgrading

Sumber: ht

To reinstall Temurin in silent mode with default features, run the
following command
https
* `m`
* `u`
existing shortcuts or icons
Note: `REINSTALL=ALL` automatically sets `REINSTALLMODE=omus`

Contoh Kode / Perintah CLI:

```
msiexec /i <package>.msi REINSTALL=ALL /quiet

# ----------------------------------------

msiexec /i <package>.msi REINSTALL=ALL REINSTALLMODE=amus /quiet
```

[java_008] Java - Installation & Setup - Windows

Windows MSI installer packages: Upgrade limitation:

Sumber: ht

Upgrading `.msi` files works only for the first 3 digits of the build
number due to an
https
* Upgrading 8.0.2.1 to 8.0.2.2 does not work. Uninstall the previous
.msi and install the new one.
* Upgrading 8.0.2.1 to 8.1.2.1 works.
* Upgrading 8.0.2.1 to 11.0.2.1 does not work. Temurin does not provide
upgrades for major versions. Either keep both installations or uninstall
the older one.

[java_009] Java - Installation & Setup - Windows

Windows MSI installer packages: Reference reading

Sumber: ht

* https://www.advancedinstaller.com/user-guide/msiexec.html[Msiexec.exe
Command Line]

[java_010] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: Overview

Sumber: ht

= Linux (RPM/DEB/APK) installer packages
Eclipse Temurin RPM, DEB and APK packages are now available for installation on
your favourite Linux distribution.

[java_011] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: Eclipse Temurin Package Names

Sumber: ht

The following name schema is being used:
[subs="attributes"]
....
temurin-<version>-jdk
e.g temurin-{latest-lts}-jdk or temurin-8-jdk
....

[java_012] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: Deb installation on Debian or Ubuntu

Sumber: ht

. Ensure the necessary packages are present
+
. Configure the Eclipse Adoptium apt repository. To check the full list of versions supported take a look at the list in the tree at https
+
. Install the Temurin version you require:
+

Contoh Kode / Perintah CLI:

```
sudo apt install -y wget apt-transport-https gpg

# -----------------------------------------

wget -qO - https://packages.adoptium.net/artifactory/api/gpg/key/public | gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/adoptium.gpg > /dev/null

# -----------------------------------------

echo "deb https://packages.adoptium.net/artifactory/deb $(awk -F= '/^VERSION_CODENAME/{print$2}' /etc/os-release) main" | sudo tee /etc/apt/sources.list.d/adoptium.list

# -----------------------------------------

sudo apt update # update if you haven't already
sudo apt install temurin-{latest-lts}-jdk
```

[java_013] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: CentOS/RHEL/Fedora Instructions

Sumber: ht

. Add the RPM repo to `/etc/yum.repos.d/adoptium.repo` making sure to change the distribution name if you are not using CentOS/RHEL/Fedora. To check the full list of versions supported take a look at the list in the tree at https
Alternatively, if you are using `yum`:
+

Contoh Kode / Perintah CLI:

```
# Uncomment and change the distribution name if you are not using CentOS/RHEL/Fedora
# DISTRIBUTION_NAME=centos

sudo cat <<EOF > /etc/yum.repos.d/adoptium.repo
[Adoptium]
name=Adoptium
baseurl=https://packages.adoptium.net/artifactory/rpm/${DISTRIBUTION_NAME:-$(.    /etc/os-release;    echo
$ID)}/\$releasever/\$basearch
enabled=1
gpgcheck=1
gpgkey=https://packages.adoptium.net/artifactory/api/gpg/key/public
EOF

# ----------------------------------------

sudo dnf install temurin-{latest-lts}-jdk

# ----------------------------------------

sudo yum install temurin-{latest-lts}-jdk
```

[java_014] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: Installing the JRE

Sumber: ht

If you only require the Java Runtime Environment (for running applications,
not compiling them), install the JRE package

Contoh Kode / Perintah CLI:

```
sudo dnf install temurin-<version>-jre

# ----------------------------------------

sudo dnf install temurin-21-jre
```

[java_015] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: Installing the Headless JRE

Sumber: ht

For server environments where no graphical components are required, a
headless JRE package is also available. This excludes GUI-related libraries
such as AWT and reduces the overall installation footprint.
Example
- Minimal server installations
- CI/CD build agents
- Headless production workloads

Contoh Kode / Perintah CLI:

```
sudo dnf install temurin-<version>-jre-headless
```

```
# ----------------------------------------

sudo dnf install temurin-21-jre-headless
```

[java_016] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: Verifying Installation

Sumber: ht

You can also confirm installed packages with:

Contoh Kode / Perintah CLI:

```
java -version

# ----------------------------------------

rpm -qa | grep temurin
```

[java_017] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: openSUSE/SLES Instructions

Sumber: ht

. Import the RPM repo as follows. RPM's are also available for SLES 12 and 15. To check the full list of versions supported take a look at
https

Contoh Kode / Perintah CLI:

```
sudo zypper ar -f https://packages.adoptium.net/artifactory/rpm/opensuse/$(. /etc/os-release; echo $VERSION_ID)/$(uname -m) adoptium

# ----------------------------------------

sudo zypper install temurin-{latest-lts}-jdk
```

[java_018] Java - Installation & Setup - Linux

Linux (RPM/DEB/APK) installer packages: Alpine Linux Instructions

Sumber: ht

. Download the Eclipse Adoptium RSA key
+
. Install the Temurin version you require

Contoh Kode / Perintah CLI:

```
sudo wget -O /etc/apk/keys/adoptium.rsa.pub https://packages.adoptium.net/artifactory/api/security/keypair/public/repositories/apk

# ----------------------------------------

echo 'https://packages.adoptium.net/artifactory/apk/alpine/main' | sudo tee -a /etc/apk/repositories

# ----------------------------------------

sudo apk add temurin-{latest-lts}-jdk
```

[java_019] Java - Getting Started & Tooling

Getting Started with Java: Elements of a Java Application

Sumber: ht

I understand that you are eager to type some code in your editor and run it to see your first Java application in action! Do not worry, your expectation will be fulfilled by the end of this tutorial. But before we move on, I would like to do through several elements that you need to know to fully understand what you are doing.
Even if you are familiar with some other programming language, know about compilation, know what an executable file is you may be interested in the following because Java works in a way that differs from C or C++.

[java_020] Java - Getting Started & Tooling

Getting Started with Java: Compiling and Executing Java Code

Sumber: ht

There are several steps that you need to follow to create a Java application. This tutorial shows you how to create a very simple Java application. If you need to create an enterprise application, the creation process is more complex but at its core you will find these simple steps.
The first of these steps is to write some Java code in a text editor.
Then this code has to be transformed to another format, which can be executed by your computer. This transformation is conducted by a special piece of software called a _compiler_. Some languages do not have a compiler; Java does. Every compiler is specific to a language.
The file produced by a compiler is often called a binary file or an executable file. Whereas you can read a source code and understand it, binary or executable files are not meant to be read by a human person. Only your computer can make sense of it.
This code contains special binary codes called _byte code_. This is a technical term that you may come across. The precise description of what is this _byte code_ is beyond the scope of this tutorial.
Compiling some code may fail; your code has to be correct for the compiler to produce an executable version of it. Do not worry, this page gives you the code you are going to compile. All you need to do is copy it and paste it in your text editor.
Once the compiler produced the binary file that you need, you can execute this binary file, that will your program.
These two steps: compilation and execution require two specific pieces of software that are part of the Java Development Kit, also known as the JDK. You will see how to download the JDK for free and how to install it later in this tutorial.
Note that starting with Java SE 11 you can also merge these two steps into one, by executing a `.java` file directly. You can use these feature only if you are executing a program that is written in a single file. This way of executing your java application does not work if your java code spans more than one file.

[java_021] Java - Getting Started & Tooling

Getting Started with Java: Creating a First Java Class

Sumber: ht

The first step you need to know is that the Java code you are writing is saved in plain text files. In this tutorial, your application will be written in a single text file. Larger applications may require thousands of such files.
Java is an object-oriented language. If this technical term does not mean anything to you, do not worry, all you need to remember at this point is that all the code you write must be held in a Java class.
A Java class is created by a special declaration in a text file. Just copy the following code and paste it in your text editor. Congratulation! You have created your first Java class!
The name of this Java class is `MyFirstClass`. You need to save this text in a file named `MyFirstClass.java`. A Java class must be saved in a file that has the same name as your class with the extension `.java`. This is mandatory and is in fact very convenient because you do not need to open a file to know what class is written in it.
You can give this class any name as long as it does not start with a number. There is a convention though: the name of a Java class starts with a capital letter. This is not mandatory but all Java developers follow this convention. When you become a seasoned Java developer, seeing a class that does not follow this convention will look weird to you.

If you are following this example to the letter, you should save the `MyFirstClass` class in a text file called `MyFirstClass.java`.
Just a word of warning: you should be using a plain text editor to create and save this file. Using a word processor will not work.

Contoh Kode / Perintah CLI:

```
public class MyFirstClass {
}
```

[java_022] Java - Getting Started & Tooling

Getting Started with Java: Preparing the Compilation of your First Class

Sumber: ht

Compiling is the second step you need to follow after the creation of your first class. It consists of transforming the Java code you wrote in your `MyFirstClass.java` file into another format that can be executed. The result of this transformation will be stored in another file created by the compiler. The name of this file will be `MyFirstClass.class`.
So far the only tool you have been using is a plain text editor. Compiling this class requires a compiler; something you may not have on your computer. Fortunately, you can download this compiler and use it for free. Let me guide you through this process.
As of now, downloading "Java" means downloading the Java Development Kit, also known as the JDK. The JDK contains many tools and among them are the ones you will be using to compile and run a Java application. It is officially distributed by the OpenJDK project and by Oracle.
You may have heard about some other elements, also called "Java".
The JRE stands for Java Runtime Environment. It is a subset of the JDK that is not distributed by the OpenJDK or Oracle anymore. It only contained the tools needed to run a Java application. You cannot compile your code with the tools provided in the JRE.
You may also have heard about J2EE, Java EE or Jakarta EE. All these acronyms refer to the Java Enterprise Edition. It is a set of tools and libraries to create enterprise-class applications. Java EE is different from the JDK and is outside the scope of this tutorial. You do not need Java EE to compile and run the simple application we are creating in this tutorial.

[java_023] Java - Getting Started & Tooling

Getting Started with Java: Setting up a Java Development Kit

Sumber: ht

You can download the JDK from different places. There is a one-stop page that always refers to the latest version of the JDK: https://jdk.java.net/. Selecting the latest "Ready for use" JDK version takes you to a page where you can download the version of the JDK you need.
From this page you can download four versions:
- Linux/AArch64
- Linux/x64
- macOS/x64
- Windows/x64

This page provides production-ready open-source builds of the Java Development Kit, an implementation of the Java SE Platform under the GNU General Public License, version 2, with the Classpath Exception.
Setting up a JDK for Windows/x64
Let us download the Windows version. What you get is a ZIP file of about 200MB that you can open with any ZIP utility software. This ZIP file contains the JDK. You can unzip the content of this file anywhere on your computer.
Once this is done you need to create an environment variable called `JAVA_HOME` that points to the directory where you unzipped the JDK. First you need to open a DOS prompt. If you unzipped a JDK 19 ZIP file in the `D:\jdk\` directory then the command you need to type in this DOS prompt is the following:
Please note that in this example and all the others the leading `>` is there to show you that you need to type this command or paste it in a prompt. You should not type this character or paste it as it is not part of the `set` command.

You can check that the `JAVA_HOME` variable has been properly set by typing the following code:
This command should print the following:
You then need to update your `PATH` environment variable to add the `bin` directory of your JDK directory to it. This can be done with the following command:
You need to be very cautious while setting up these two variables, because a single mistake like an added white space of a missing semicolon will result in failure.
Do not close this command prompt. If you close it and open it again then you will need to create these two variables again.
Setting up a JDK for Linux/x64
Let us download the Linux version. What you get is an archive file with a `.tar.gz` extension that you need to expand.
To expand it, you need to copy it or move it to the right directory. You can then type the following command:
Please note that in this example and all the others, the leading `$` is there to show you that you need to type this command or paste it in a prompt. You should not type this character or paste it, as it is not part of the `tar` command.
This command expands all the files with the extension `.tar.gz` that you have in the current directory. You can use the exact name of this file if you just need to expand it.
Executing this command may take several seconds or more, depending on your system. It creates a new directory in the current directory with the content of the JDK in it.
Once this is done you need to create an environment variable called `JAVA_HOME` that points to the directory where you expanded the JDK. If you expanded a JDK 19 archive file in the `/home/javauser/jdk` directory then the command you need to type in this shell prompt is the following:
The exact directory depends on the distribution file you have expanded.
You can check that the `JAVA_HOME` variable has been properly set by typing the following code:
This command should print the following:
Then you need to update your `PATH` variable to add the `bin` directory of your JDK directory to it. This can be done with the following command:
You need to be very cautious while setting up these two variables because a single mistake like an added white space of a missing semicolon will result in failure.
Do not close this shell prompt. If you close it and open it again then you will need to create these two variables again.
You can check if everything is ok by typing the following command:
Your shell should print the complete path to the `java` executable file in the `bin` directory of the distribution you just expanded. In this example it will print:
Setting up a JDK for macOS
Let us download the macOS version. What you get is an archive file with a `.tar.gz` extension that you need to expand.
To expand it, you need to copy it or move it to the right directory. You can then type the following command:
Please note that in this example, and all the others, the leading `$` is there to show you that you need to type this command or paste it in a prompt. You should not type this character or paste it as it is not part of the `tar` command.
This command expands all the files with the extension `.tar.gz` that you have in the current directory. You can use the exact name of this file if you just need to expand it.
Executing this command may take several seconds or more, depending on your system. It creates a new directory in the current directory with the content of the JDK in it. This directory has the extension `.jdk`.
Once this is done you need to create an environment variable called `JAVA_HOME` that points to the directory where you expanded the JDK. If you expanded a JDK 19 archive file in the `/Users/javauser/jdk` directory then the command you need to type in this shell prompt is the following:
The exact directory depends on the distribution file you have expanded.
You can check that the `JAVA_HOME` variable has been properly set by typing the following code:
This command should print the following:
You then need to update your `PATH` variable to add the `bin` directory of your JDK directory to it. This can be done with the following command:
You need to be very cautious while setting up these two variables because a single mistake like an added white space of a missing semicolon will result in failure.
Do not close this shell prompt. If you close it and open it again then you will need to create these two variables again.
You can check if everything is ok by typing the following command:

Your shell should print the complete path to the `java` executable file in the `bin` directory of the distribution you just expanded. In this example it will print:

Contoh Kode / Perintah CLI:

```
> set JAVA_HOME=D:\jdk\jdk-19

// ----------------------------------------

> echo %JAVA_HOME%

// ----------------------------------------

D:\jdk\jdk-19

// ----------------------------------------

> set PATH=%JAVA_HOME%\bin;%PATH%

// ----------------------------------------

$ tar xzf *.tar.gz

// ----------------------------------------

$ export JAVA_HOME=/home/javauser/jdk/jdk-19

// ----------------------------------------

$ echo $JAVA_HOME

// ----------------------------------------

/home/javauser/jdk/jdk-19

// ----------------------------------------

$ export PATH=$JAVA_HOME/bin:$PATH

// ----------------------------------------

$ which java

// ----------------------------------------

/home/javauser/jdk/jdk-19/bin/java

// ----------------------------------------

$ tar xzf *.tar.gz

// ----------------------------------------

$ export JAVA_HOME=/Users/javauser/jdk/jdk-19.jdk/Contents/Home

// ----------------------------------------

$ echo $JAVA_HOME

// ----------------------------------------

/Users/javauser/jdk/jdk-19.jdk/Contents/Home
```

```
// ----------------------------------------

$ export PATH=$JAVA_HOME/bin:$PATH

// ----------------------------------------

$ which java

// ----------------------------------------

/Users/javauser/jdk/jdk-19.jdk/Contents/Home/bin/java
```

[java_024] Java - Getting Started & Tooling

Getting Started with Java: Compiling your First Class

Sumber: ht

Once you have properly set up your JDK; the `JAVA_HOME` variable and the `PATH` variable, you are ready to compile your first class.
All the commands you will be typing now should be typed in the same prompt as the one you used to set up these two variables.
Compiling and Running Your First Java Program
Whether you followed the Windows, the Linux or the macOS path, the remaining is the same.
1. Change to the directory where you saved your first class `MyFirstClass.java`. You can check that you are in the right directory by typing `dir`. It will show you the files you have in this directory. You should see your `MyFirstClass.java` file.
2. Check that your compiler is accessible from this directory by typing the following. This command is the same whether you are on Windows or Linux.
It should tell you which version of the `javac` you are currently using. If it gives you an error message then you need to check your `JAVA_HOME` and `PATH` variables as there is most probably something wrong with them.
3. Now you are all set to compile your first code. You can type the following.
Two things may happen at this point. You may have error messages telling you that the compiler cannot compile your code because of errors in your Java code. You will need to fix them before being able to move on.
If the compiler remains silent and does not complain about anything: congratulations! It means that your Java code has been properly compiled. Checking the content of the directory again should show a new file in it: `MyFirstClass.class`

Contoh Kode / Perintah CLI:

```
> java -version

// ----------------------------------------

> javac MyFirstClass.java
```

[java_025] Java - Getting Started & Tooling

Getting Started with Java: Adding Code to Your Class to Run it

Sumber: ht

So far your class is empty; there is no executable code in it. If you were able to compile it properly then you can advance to the next step and execute some code.
Just open your `MyFirstClass.java` file and copy the following code in it.
As you may know, there is a long-standing tradition in computer science, which is to write a program that prints "Hello, World!" on the console of your application. So let us do that!
There is technical code in this class that may not be very clear to you. Do not worry; all you need to do is to compile it following the steps described in the previous section.
Make sure that the compiler created the `MyFirstClass.class` for you. To run it, all you need to type is the following command:
This should print _Hello, World!_ on the console. If this is the case: congratulations! You have been able to run your first

Java program!

Contoh Kode / Perintah CLI:

```
public class MyFirstClass {

    public static void main(String... args) {
        System.out.println("Hello, World!");
    }
}

// ----------------------------------------

> java MyFirstClass
```

[java_026] Java - Getting Started & Tooling

Getting Started with Java: Running the Hello World Program as a Single File Application

Sumber: ht

Starting with Java SE 11, you can run a Java application without going through the compilation step, as long as the program is written in a single file. This is the case of this simple _Hello, World!_ application.
You can just type the following:
And it will print the _Hello, World!_ message on the console.

Contoh Kode / Perintah CLI:

```
> java MyFirstClass.java
```

[java_027] Java - Getting Started & Tooling

Getting Started with Java: Common Problems and Their Solutions

Sumber: ht

Compiler Problems
Common Error Messages on Microsoft Windows Systems
If you receive this error, Windows cannot find the compiler `javac`.
Here's one way to tell Windows where to find `javac`. Suppose you installed the JDK in `C:\jdk19`. At the prompt you would type the following command and press Enter:
If you choose this option, you'll have to precede your `javac` and `java` commands with `C:\jdk19\bin\` each time you compile or run a program. To avoid this extra typing, consult the section Updating the `PATH` variable in the JDK 19 installation instructions.
If you receive this error, you forgot to include the `.java` suffix when compiling the program. Remember, the command is `javac HelloWorldApp.java` not `javac HelloWorldApp`.
Common Error Messages on UNIX Systems
If you receive this error, UNIX cannot find the compiler, `javac`.
Here's one way to tell UNIX where to find javac. Suppose you installed the JDK in `/usr/local/jdk19`. At the prompt you would type the following command and press Return:
Note: If you choose this option, each time you compile or run a program, you'll have to precede your `javac` and `java` commands with `/usr/local/jdk19/`. To avoid this extra typing, you could add this information to your `PATH` variable. The steps for doing so will vary depending on which shell you are currently running.
If you receive this error, you forgot to include the `.java` suffix when compiling the program. Remember, the command is `javac HelloWorldApp.java` not `javac HelloWorldApp`.
Syntax Errors (All Platforms)
If you mistype part of a program, the compiler may issue a syntax error. The message usually displays the type of the error, the line number where the error was detected, the code on that line, and the position of the error within the code. Here's an error caused by omitting a semicolon (`;`) at the end of a statement:
If you see any compiler errors, then your program did not successfully compile, and the compiler did not create a `.class` file. Carefully verify the program, fix any errors that you detect, and try again.

Semantic Errors

In addition to verifying that your program is syntactically correct, the compiler checks for other basic correctness. For example, the compiler warns you each time you use a variable that has not been initialized:

Again, your program did not successfully compile, and the compiler did not create a `.class` file. Fix the error and try again.

Runtime Problems

Error Messages on Microsoft Windows Systems

If you receive this error, `java` cannot find your bytecode file, `HelloWorldApp.class`.

One of the places `java` tries to find your `.class` file is your current directory. So if your `.class` file is in `C:\java`, you should change your current directory to that. To change your directory, type the following command at the prompt and press Enter:

The prompt should change to `C:\java>`. If you enter `dir` at the prompt, you should see your `.java` and `.class` files. Now enter `java HelloWorldApp` again.

If you still have problems, you might have to change your `CLASSPATH` variable. To see if this is necessary, try clobbering the classpath with the following command.

Now enter `java HelloWorldApp` again. If the program works now, you'll have to change your `CLASSPATH` variable. To set this variable, consult the _Updating the PATH variable_ section in the JDK installation instructions. The `CLASSPATH` variable is set in the same manner.

A common mistake made by beginner programmers is to try and run the `java` launcher on the `.class` file that was created by the compiler. For example, you'll get this error if you try to run your program with `java HelloWorldApp.class` instead of `java HelloWorldApp`. Remember, the argument is the name of the class that you want to use, not the filename.

The Java VM requires that the class you execute with it have a `main` method at which to begin execution of your application. A Closer Look at the Adding Code to Your Class to Run it section discusses the main method in detail.

Error Messages on UNIX Systems

If you receive this error, `java` cannot find your bytecode file, `HelloWorldApp.class`.

One of the places java tries to find your bytecode file is your current directory. So, for example, if your bytecode file is in `/home/jdoe/java`, you should change your current directory to that. To change your directory, type the following command at the prompt and press Return:

If you enter `pwd` at the prompt, you should see `/home/jdoe/java`. If you enter `ls` at the prompt, you should see your `.java` and `.class` files. Now enter `java HelloWorldApp` again.

If you still have problems, you might have to change your `CLASSPATH` environment variable. To see if this is necessary, try clobbering the classpath with the following command.

Now enter `java HelloWorldApp` again. If the program works now, you'll have to change your `CLASSPATH` variable in the same manner as the `PATH` variable above.

A common mistake made by beginner programmers is to try and run the java launcher on the `.class` file that was created by the compiler. For example, you'll get this error if you try to run your program with `java HelloWorldApp.class` instead of `java HelloWorldApp`. Remember, the argument is the name of the class that you want to use, not the filename.

The Java VM requires that the class you execute with it have a main method at which to begin execution of your application. A Closer Look at the Adding Code to Your Class to Run it section discusses the main method in detail.

Contoh Kode / Perintah CLI:

```
javac is not recognized as an internal or external command, operable program or batch file

// ----------------------------------------

C:\jdk19\bin\javac HelloWorldApp.java

// ----------------------------------------

Class names, HelloWorldApp, are only accepted if annotation processing is explicitly requested

// ----------------------------------------
```

```
javac: Command not found

// ----------------------------------------

/usr/local/jdk19/javac HelloWorldApp.java

// ----------------------------------------

Class names, 'HelloWorldApp', are only accepted if annotation processing is explicitly requested

// ----------------------------------------

Testing.java:8: error: ';' expected
count++
^
1 error

// ----------------------------------------

Testing.java:8: error: variable count might not have been initialized
count++;
^
Testing.java:9: error: variable count might not have been initialized
System.out.println("Input has " + count + " chars.");
^
2 errors

// ----------------------------------------

cd c:\java

// ----------------------------------------

set CLASSPATH=

// ----------------------------------------

Could not find or load main class HelloWorldApp.class

// ----------------------------------------

Exception in thread "main" java.lang.NoSuchMethodError: main

// ----------------------------------------

Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorldApp

// ----------------------------------------

cd /home/jdoe/java

// ----------------------------------------

unset CLASSPATH

// ----------------------------------------

Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorldApp/class

// ----------------------------------------
```

```
Exception in thread "main" java.lang.NoSuchMethodError: main
```

[java_028] Java - Getting Started & Tooling

Getting Started with Java: Going Further

Sumber: ht

This first Java program showed you the basic steps every Java developer follows to run an application.
1. Create a source in a set of `.java` text files
2. Compile these files to produce a set of corresponding `.class` binary files
3. Run them together as an application.
Developers that work on large applications do not use plain text editors to manage their source code; they use Integrated Development Environments. IDEs are complex software applications, specialized in software development. These applications handle the compilation of your source code automatically, they can help you to track errors in the syntax of your Java code and nail down bugs in its execution, among other things.
Some of these tools are open source and free to use.
- `the Eclipse foundation maintains Eclipse`,
- `the Apache foundation maintains NetBeans`,
- JetBrains publishes `IntelliJ IDEA Community Edition` that is free to use for personal and commercial development.

[java_029] Java - Language Basics - Variables

Creating Variables and Naming Them: Variables

Sumber: ht

As you learned in the previous section, an object stores its state in _fields_.
The What Is an Object? discussion introduced you to fields, but you probably have still a few questions, such as: What are the rules and conventions for naming a field? Besides `int`, what other data types are there? Do fields have to be initialized when they are declared? Are fields assigned a default value if they are not explicitly initialized? We'll explore the answers to such questions in this section, but before we do, there are a few technical distinctions you must first become aware of. In the Java programming language, the terms "field" and "variable" are both used; this is a common source of confusion among new developers, since both often seem to refer to the same thing.
The Java programming language defines the following kinds of variables:
- **Instance Variables (Non-Static Fields)** Technically speaking, objects store their individual states in "non-static fields", that is, fields declared without the `static` keyword. Non-static fields are also known as instance variables because their values are unique to each instance of a class (to each object, in other words); the `currentSpeed` of one bicycle is independent from the `currentSpeed` of another.
- **Class Variables (Static Fields)** A class variable is any field declared with the `static` modifier; this tells the compiler that there is exactly one copy of this variable in existence, regardless of how many times the class has been instantiated. A field defining the number of gears for a particular kind of bicycle could be marked as `static` since conceptually the same number of gears will apply to all instances. The code `static int numGears = 6;` would create such a `static` field. Additionally, the keyword `final` could be added to indicate that the number of gears will never change.
- **Local Variables** Similar to how an object stores its state in fields, a method will often store its temporary state in local variables. The syntax for declaring a local variable is similar to declaring a field (for example, `int count = 0;`). There is no special keyword designating a variable as local; that determination comes entirely from the location in which the variable is declared -- which is between the opening and closing braces of a method. As such, local variables are only visible to the methods in which they are declared; they are not accessible from the rest of the class.
- **Parameters** You've already seen examples of parameters, both in the `Bicycle` class and in the main method of the "Hello World!" application. Recall that the signature for the main method is `public static void main(String[] args)`. Here, the `args` variable is the parameter to this method. The important thing to remember is that parameters are always classified as "variables" not "fields". This applies to other parameter-accepting constructs as well (such as constructors and exception handlers) that you'll learn about later in the tutorial.
Having said that, the remainder of this tutorial uses the following general guidelines when discussing fields and

variables. If we are talking about "fields in general" (excluding local variables and parameters), we may simply say "fields". If the discussion applies to "all of the above", we may simply say "variables". If the context calls for a distinction, we will use specific terms (static field, local variables, etc.) as appropriate. You may also occasionally see the term "member" used as well. A type's fields, methods, and nested types are collectively called its _members_.

Contoh Kode / Perintah CLI:

```
int cadence = 0;
int speed = 0;
int gear = 1;
```

[java_030] Java - Language Basics - Variables

Creating Variables and Naming Them: Naming Variables

Sumber: ht

Every programming language has its own set of rules and conventions for the kinds of names that you're allowed to use, and the Java programming language is no different. The rules and conventions for naming your variables can be summarized as follows:
- Variable names are case-sensitive. A variable's name can be any legal identifier -- an unlimited-length sequence of Unicode letters and digits, beginning with a letter, the dollar sign `$`, or the underscore character `_`. The convention, however, is to always begin your variable names with a letter, not `$` or `_`. Additionally, the dollar sign character, by convention, is never used at all. You may find some situations where auto-generated names will contain the dollar sign, but your variable names should always avoid using it. A similar convention exists for the underscore character; while it's technically legal to begin your variable's name with `_`, this practice is discouraged. White space is not permitted.
- Subsequent characters may be letters, digits, dollar signs, or underscore characters. Conventions (and common sense) apply to this rule as well. When choosing a name for your variables, use full words instead of cryptic abbreviations. Doing so will make your code easier to read and understand. In many cases it will also make your code self-documenting; fields named cadence, speed, and gear, for example, are much more intuitive than abbreviated versions, such as s, c, and g. Also keep in mind that the name you choose must not be a keyword or reserved word.
- If the name you choose consists of only one word, spell that word in all lowercase letters. If it consists of more than one word, capitalize the first letter of each subsequent word. The names `gearRatio` and `currentGear` are prime examples of this convention. If your variable stores a constant value, such as static final int NUM_GEARS = 6, the convention changes slightly, capitalizing every letter and separating subsequent words with the underscore character. By convention, the underscore character is never used elsewhere.

[java_031] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Overview

Sumber: ht

You have already learned that objects store their state in fields. However, the Java programming language also uses the term _variable_ as well. This section discusses this relationship, plus variable naming rules and conventions, basic data types (primitive types, character strings, and arrays), default values, and literals.

[java_032] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Primitive Types

Sumber: ht

The Java programming language is statically-typed, which means that all variables must first be declared before they can be used. This involves stating the variable's type and name, as you have already seen:
Doing so tells your program that a field named `gear` exists, holds numerical data, and has an initial value of `1`. A variable's data type determines the values it may contain, plus the operations that may be performed on it. In addition to `int`, the Java programming language supports seven other primitive data types. A primitive type is predefined by the language and is named by a reserved keyword. Primitive values do not share state with other primitive values. The eight primitive data types supported by the Java programming language are:
- `byte`: The `byte` data type is an 8-bit signed two's complement integer. It has a minimum value of -128 and a

maximum value of 127 (inclusive). The `byte` data type can be useful for saving memory in large arrays, where the memory savings actually matters. They can also be used in place of `int` where their limits help to clarify your code; the fact that a variable's range is limited can serve as a form of documentation.
- `short`: The `short` data type is a 16-bit signed two's complement integer. It has a minimum value of -32,768 and a maximum value of 32,767 (inclusive). As with `byte`, the same guidelines apply: you can use a short to save memory in large arrays, in situations where the memory savings actually matters.
- `int`: By default, the `int` data type is a 32-bit signed two's complement integer, which has a minimum value of -2³¹ and a maximum value of 2³¹-1. In Java SE 8 and later, you can use the `int` data type to represent an unsigned 32-bit integer, which has a minimum value of 0 and a maximum value of 2³²-1. Use the `Integer` class to use `int` data type as an unsigned integer. See the section The Number Classes for more information. Static methods like `compareUnsigned()`) have been added to the `Integer` class to support the arithmetic operations for unsigned integers.
- `long`: The `long` data type is a 64-bit two's complement integer. The signed long has a minimum value of -2⁶³ and a maximum value of 2⁶³-1. In Java SE 8 and later, you can use the `long` data type to represent an unsigned 64-bit long, which has a minimum value of 0 and a maximum value of 2⁶⁴-1. Use this data type when you need a range of values wider than those provided by `int`. The `Long` class also contains methods like `compareUnsigned()`), `divideUnsigned()`) etc to support arithmetic operations for unsigned long.
- `float`: The `float` data type is a single-precision 32-bit IEEE 754 floating point. Its range of values is beyond the scope of this discussion, but is specified in the Floating-Point Types, Formats, and Values section of the Java Language Specification. As with the recommendations for `byte` and `short`, use a `float` (instead of `double`) if you need to save memory in large arrays of floating point numbers. This data type should never be used for precise values, such as currency. For that, you will need to use the `java.math.BigDecimal` class instead. Numbers and Strings covers `BigDecimal` and other useful classes provided by the Java platform.
- `double`: The `double` data type is a double-precision 64-bit IEEE 754 floating point. Its range of values is beyond the scope of this discussion, but is specified in the Floating-Point Types, Formats, and Values section of the Java Language Specification. For decimal values, this data type is generally the default choice. As mentioned above, this data type should never be used for precise values, such as currency.
- `boolean`: The `boolean` data type has only two possible values: `true` and `false`. Use this data type for simple flags that track true/false conditions. This data type represents one bit of information, but its "size" isn't something that's precisely defined.
- `char`: The `char` data type is a single 16-bit Unicode character. It has a minimum value of `\u0000` (or 0) and a maximum value of `\uffff` (or 65,535 inclusive).
In addition to the eight primitive data types listed above, the Java programming language also provides special support for character strings via the `java.lang.String` class. Enclosing your character string within double quotes will automatically create a new `String` object; for example:
`String` objects are immutable, which means that once created, their values cannot be changed. The `String` class is not technically a primitive data type, but considering the special support given to it by the language, you will probably tend to think of it as such. You will learn more about the `String` class in the section Strings.

Contoh Kode / Perintah CLI:

```
int gear = 1;

// ----------------------------------------

String s = "this is a string";
```

[java_033] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Initializing a Variable with a Default Value

Sumber: ht

It is not always necessary to assign a value when a field is declared. Fields that are declared but not initialized will be set to a reasonable default by the compiler. Generally speaking, this default will be zero or null, depending on the data type. Relying on such default values, however, is generally considered bad programming style.
The following table summarizes the default values for the above data types.
Data Type	Default Value (for fields)
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	`\u0000`
String (or any object)	null
boolean	`false`
Local variables are slightly different; the compiler never assigns a default value to an uninitialized local variable. If you cannot initialize your local variable where it is declared, make sure to assign it a value before you attempt to use it. Accessing an uninitialized local variable will result in a compile-time error.

[java_034] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Creating Values with Literals

Sumber: ht

You may have noticed that the `new` keyword is not used when initializing a variable of a primitive type. Primitive types are special data types built into the language; they are not objects created from a class. A literal is the source code representation of a fixed value; literals are represented directly in your code without requiring computation. As shown below, it is possible to assign a literal to a variable of a primitive type:

Contoh Kode / Perintah CLI:

```
boolean result = true;
char capitalC = 'C';
byte b = 100;
short s = 10000;
int i = 100000;
```

[java_035] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Integer Literals

Sumber: ht

An integer literal is of type `long` if it ends with the letter `L` or `l`; otherwise it is of type `int`. It is recommended that you use the upper case letter `L` because the lower case letter `l` is hard to distinguish from the digit `1`.
Values of the integral types `byte`, `short`, `int`, and `long` can be created from `int` literals. Values of type `long` that exceed the range of `int` can be created from `long` literals. Integer literals can be expressed by these number systems:
- Decimal: Base 10, whose digits consists of the numbers 0 through 9; this is the number system you use every day
- Hexadecimal: Base 16, whose digits consist of the numbers 0 through 9 and the letters A through F
- Binary: Base 2, whose digits consists of the numbers 0 and 1 (you can create binary literals in Java SE 7 and later)
For general-purpose programming, the decimal system is likely to be the only number system you will ever use. However, if you need to use another number system, the following example shows the correct syntax. The prefix `0x` indicates hexadecimal and `0b` indicates binary:

Contoh Kode / Perintah CLI:

```
// The number 26, in decimal
int decimalValue = 26;

//  The number 26, in hexadecimal
int hexadecimalValue = 0x1a;

// The number 26, in binary
int binaryValue = 0b11010;
```

[java_036] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Floating-Point Literals

Sumber: ht

A floating-point literal is of type `float` if it ends with the letter `F` or `f`; otherwise its type is `double` and it can optionally end with the letter `D` or `d`.
The floating point types (`float` and `double`) can also be expressed using `E` or `e` (for scientific notation), `F` or `f` (32-bit float literal) and `D` or `d` (64-bit double literal; this is the default and by convention is omitted).

Contoh Kode / Perintah CLI:

```
double d1 = 123.4;

// same value as d1, but in scientific notation
double d2 = 1.234e2;
float f1  = 123.4f;
```

[java_037] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Character and String Literals

Sumber: ht

Literals of types `char` and `String` may contain any Unicode (UTF-16) characters. If your editor and file system allow it, you can use such characters directly in your code. If not, you can use a "Unicode escape" such as `\u0108` (capital C with circumflex), or "S\u00ED Se\u00F1or" (Sí Señor in Spanish). Always use 'single quotes' for `char` literals and "double quotes" for `String` literals. Unicode escape sequences may be used elsewhere in a program (such as in field names, for example), not just in `char` or `String` literals.
The Java programming language also supports a few special escape sequences for `char` and `String` literals: `\b` (backspace), `\t` (tab), `\n` (line feed), `\f` (form feed), `\r` (carriage return), `\"` (double quote), `\'` (single quote), and `\\` (backslash).
There is also a special `null` literal that can be used as a value for any reference type. The `null` literal may be assigned to any variable, except variables of primitive types. There is little you can do with a `null` value beyond testing for its presence. Therefore, `null` is often used in programs as a marker to indicate that some object is unavailable.
Finally, there is also a special kind of literal called a _class literal_, formed by taking a type name and appending `.class`; for example, `String.class`. This refers to the object that represents the type itself, of type `Class`.

[java_038] Java - Language Basics - Primitives

Creating Primitive Type Variables in Your Programs: Using Underscore Characters in Numeric Literals

Sumber: ht

In Java SE 7 and later, any number of underscore characters (`_`) can appear anywhere between digits in a numerical literal. This feature enables you, for example. to separate groups of digits in numeric literals, which can improve the readability of your code.
For instance, if your code contains numbers with many digits, you can use an underscore character to separate digits in groups of three, similar to how you would use a punctuation mark like a comma, or a space, as a separator.
The following example shows other ways you can use the underscore in numeric literals:

You can place underscores only between digits; you cannot place underscores in the following places:
- At the beginning or end of a number
- Adjacent to a decimal point in a floating point literal
- Prior to an `F` or `L` suffix
- In positions where a string of digits is expected
The following examples demonstrate valid and invalid underscore placements in numeric literals:

Contoh Kode / Perintah CLI:

```
long creditCardNumber = 1234_5678_9012_3456L;
long socialSecurityNumber = 999_99_9999L;
float pi =  3.14_15F;
long hexBytes = 0xFF_EC_DE_5E;
long hexWords = 0xCAFE_BABE;
long maxLong = 0x7fff_ffff_ffff_ffffL;
byte nybbles = 0b0010_0101;
long bytes = 0b11010010_01101001_10010100_10010010;

// ----------------------------------------

// Invalid: cannot put underscores
// adjacent to a decimal point
float pi1 = 3_.1415F;
// Invalid: cannot put underscores
// adjacent to a decimal point
float pi2 = 3._1415F;
// Invalid: cannot put underscores
// prior to an L suffix
long socialSecurityNumber1 = 999_99_9999_L;

// OK (decimal literal)
int x1 = 5_2;
// Invalid: cannot put underscores
// At the end of a literal
int x2 = 52_;
// OK (decimal literal)
int x3 = 5_______2;

// Invalid: cannot put underscores
// in the 0x radix prefix
int x4 = 0_x52;
// Invalid: cannot put underscores
// at the beginning of a number
int x5 = 0x_52;
// OK (hexadecimal literal)
int x6 = 0x5_2;
// Invalid: cannot put underscores
// at the end of a number
int x7 = 0x52_;
```

[java_039] Java - Language Basics - Arrays

Creating Arrays in Your Programs: Arrays

Sumber: ht

An _array_ is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed. You have seen an example of arrays already, in the main method of the "Hello World!" application. This section discusses arrays in greater detail.
<figure>
<p align="center">
<img src="/assets/images/language-basics/01_array.png"

alt="An array of 8 elements."
width="60%"/>
</p>
<figcaption align="center">An array of 8 elements.</figcaption>
</figure>
Each item in an array is called an _element_, and each element is accessed by its numerical _index_. As shown in the preceding illustration, numbering begins with 0. The 6th element, for example, would therefore be accessed at index 5.
The following program, `ArrayDemo`, creates an array of integers, puts some values in the array, and prints each value to standard output.
The output from this program is:
In a real-world programming situation, you would probably use one of the supported looping constructs to iterate through each element of the array, rather than write each line individually as in the preceding example. However, the example clearly illustrates the array syntax. You will learn about the various looping constructs (for, while, and do-while) in the Control Flow section.

Contoh Kode / Perintah CLI:

```
class ArrayDemo {
    public static void main(String[] args) {
        // declares an array of integers
        int[] anArray;

        // allocates memory for 10 integers
        anArray = new int[10];

        // initialize first element
        anArray[0] = 100;
        // initialize second element
        anArray[1] = 200;
        // and so forth
        anArray[2] = 300;
        anArray[3] = 400;
        anArray[4] = 500;
        anArray[5] = 600;
        anArray[6] = 700;
        anArray[7] = 800;
        anArray[8] = 900;
        anArray[9] = 1000;

        System.out.println("Element at index 0: "
                           + anArray[0]);
        System.out.println("Element at index 1: "
                           + anArray[1]);
        System.out.println("Element at index 2: "
                           + anArray[2]);
        System.out.println("Element at index 3: "
                           + anArray[3]);
        System.out.println("Element at index 4: "
                           + anArray[4]);
        System.out.println("Element at index 5: "
                           + anArray[5]);
        System.out.println("Element at index 6: "
                           + anArray[6]);
        System.out.println("Element at index 7: "
                           + anArray[7]);
        System.out.println("Element at index 8: "
                           + anArray[8]);
        System.out.println("Element at index 9: "
                           + anArray[9]);
    }
}
```

```
// ----------------------------------------

Element at index 0: 100
Element at index 1: 200
Element at index 2: 300
Element at index 3: 400
Element at index 4: 500
Element at index 5: 600
Element at index 6: 700
Element at index 7: 800
Element at index 8: 900
Element at index 9: 1000
```

[java_040] Java - Language Basics - Arrays

Creating Arrays in Your Programs: Declaring a Variable to Refer to an Array

Sumber: ht

The preceding program declares an array (named `anArray`) with the following line of code:
Like declarations for variables of other types, an array declaration has two components: the array's type and the array's name. An array's type is written as `type[]`, where `type` is the data type of the contained elements; the brackets are special symbols indicating that this variable holds an array. The size of the array is not part of its type (which is why the brackets are empty). An array's name can be anything you want, provided that it follows the rules and conventions as discussed in the Classes section. As with variables of other types, the declaration does not actually create an array; it simply tells the compiler that this variable will hold an array of the specified type.
Similarly, you can declare arrays of other types:
You can also place the brackets after the array's name:
However, convention discourages this form; the brackets identify the array type and should appear with the type designation.

Contoh Kode / Perintah CLI:

```
// declares an array of integers
int[] anArray;

// ----------------------------------------

byte[] anArrayOfBytes;
short[] anArrayOfShorts;
long[] anArrayOfLongs;
float[] anArrayOfFloats;
double[] anArrayOfDoubles;
boolean[] anArrayOfBooleans;
char[] anArrayOfChars;
String[] anArrayOfStrings;

// ----------------------------------------

// this form is discouraged
float anArrayOfFloats[];
```

[java_041] Java - Language Basics - Arrays

Creating Arrays in Your Programs: Creating, Initializing, and Accessing an Array

Sumber: ht

One way to create an array is with the `new` operator. The next statement in the `ArrayDemo` program allocates an array with enough memory for 10 integer elements and assigns the array to the `anArray` variable.
If this statement is missing, then the compiler prints an error like the following, and compilation fails:
The next few lines assign values to each element of the array:
Each array element is accessed by its numerical index:
Alternatively, you can use the shortcut syntax to create and initialize an array:
Here the length of the array is determined by the number of values provided between braces and separated by commas.
You can also declare an array of arrays (also known as a multidimensional array) by using two or more sets of brackets, such as `String[][]` names. Each element, therefore, must be accessed by a corresponding number of index values.
In the Java programming language, a multidimensional array is an array whose components are themselves arrays. This is unlike arrays in C or Fortran. A consequence of this is that the rows are allowed to vary in length, as shown in the following `MultiDimArrayDemo` program:
The output from this program is:
Finally, you can use the built-in `length` property to determine the size of any array. The following code prints the array's size to standard output:

Contoh Kode / Perintah CLI:

```
// create an array of integers
anArray = new int[10];

// ----------------------------------------

ArrayDemo.java:4: Variable anArray may not have been initialized.

// ----------------------------------------

anArray[0] = 100; // initialize first element
anArray[1] = 200; // initialize second element
anArray[2] = 300; // and so forth

// ----------------------------------------

System.out.println("Element 1 at index 0: " + anArray[0]);
System.out.println("Element 2 at index 1: " + anArray[1]);
System.out.println("Element 3 at index 2: " + anArray[2]);

// ----------------------------------------

int[] anArray = {
    100, 200, 300,
    400, 500, 600,
    700, 800, 900, 1000
};

// ----------------------------------------

class MultiDimArrayDemo {
    public static void main(String[] args) {
        String[][] names = {
            {"Mr. ", "Mrs. ", "Ms. "},
            {"Smith", "Jones"}
        };
        // Mr. Smith
        System.out.println(names[0][0] + names[1][0]);
```

```
        // Ms. Jones
        System.out.println(names[0][2] + names[1][1]);
    }
}

// ----------------------------------------

Mr. Smith
Ms. Jones

// ----------------------------------------

System.out.println(anArray.length);
```

[java_042] Java - Language Basics - Arrays

Creating Arrays in Your Programs: Copying Arrays

Sumber: ht

The `System` class has an `arraycopy()`) method that you can use to efficiently copy data from one array into another: The two `Object` arguments specify the array to copy from and the array to copy to. The three `int` arguments specify the starting position in the source array, the starting position in the destination array, and the number of array elements to copy.
The following program, `ArrayCopyDemo`, declares an array of `String` elements. It uses the `System.arraycopy()`) method to copy a subsequence of array components into a second array:
The output from this program is:

Contoh Kode / Perintah CLI:

```
public static void arraycopy(Object src, int srcPos,
                             Object dest, int destPos, int length)

// ----------------------------------------

class ArrayCopyDemo {
    public static void main(String[] args) {
        String[] copyFrom = {
            "Affogato", "Americano", "Cappuccino", "Corretto", "Cortado",
            "Doppio", "Espresso", "Frappucino", "Freddo", "Lungo", "Macchiato",
            "Marocchino", "Ristretto" };

        String[] copyTo = new String[7];
        System.arraycopy(copyFrom, 2, copyTo, 0, 7);
        for (String coffee : copyTo) {
            System.out.print(coffee + " ");
        }
    }
}

// ----------------------------------------

Cappuccino Corretto Cortado Doppio Espresso Frappucino Freddo
```

[java_043] Java - Language Basics - Arrays

Creating Arrays in Your Programs: Array Manipulations

Sumber: ht

Arrays are a powerful and useful concept used in programming. Java SE provides methods to perform some of the most common manipulations related to arrays. For instance, the `ArrayCopyDemo` example uses the `arraycopy()`) method of the `System` class instead of manually iterating through the elements of the source array and placing each one into the destination array. This is performed behind the scenes, enabling the developer to use just one line of code to call the method.

For your convenience, Java SE provides several methods for performing array manipulations (common tasks, such as copying, sorting and searching arrays) in the `java.util.Arrays` class. For instance, the previous example can be modified to use the `java.util.Arrays` method of the `java.util.Arrays` class, as you can see in the `ArrayCopyOfDemo` example. The difference is that using the `java.util.Arrays` method does not require you to create the destination array before calling the method, because the destination array is returned by the method:

As you can see, the output from this program is the same, although it requires fewer lines of code. Note that the second parameter of the `java.util.Arrays` method is the initial index of the range to be copied, inclusively, while the third parameter is the final index of the range to be copied, exclusively. In this example, the range to be copied does not include the array element at index 9 (which contains the string `Lungo`).

Some other useful operations provided by methods in the `java.util.Arrays` class are:

- Searching an array for a specific value to get the index at which it is placed (the `binarySearch()`) method).
- Comparing two arrays to determine if they are equal or not (the `equals()`) method).
- Filling an array to place a specific value at each index (the `fill()`) method).
- Sorting an array into ascending order. This can be done either sequentially, using the `sort()`) method, or concurrently, using the `parallelSort()`) method introduced in Java SE 8. Parallel sorting of large arrays on multiprocessor systems is faster than sequential array sorting.
- Creating a stream that uses an array as its source (the `stream()`) method). For example, the following statement prints the contents of the `copyTo` array in the same way as in the previous example:

See Aggregate Operations for more information about streams.

- Converting an array to a string. The `toString()`) method converts each element of the array to a string, separates them with commas, then surrounds them with brackets. For example, the following statement converts the `copyTo` array to a string and prints it:

This statement prints the following:

Contoh Kode / Perintah CLI:

```
class ArrayCopyOfDemo {
    public static void main(String[] args) {
        String[] copyFrom = {
            "Affogato", "Americano", "Cappuccino", "Corretto", "Cortado",
            "Doppio", "Espresso", "Frappucino", "Freddo", "Lungo", "Macchiato",
            "Marocchino", "Ristretto" };

        String[] copyTo = java.util.Arrays.copyOfRange(copyFrom, 2, 9);
        for (String coffee : copyTo) {
            System.out.print(coffee + " ");
        }
    }
}

// ----------------------------------------

java.util.Arrays.stream(copyTo).map(coffee -> coffee + " ").forEach(System.out::print);

// ----------------------------------------

System.out.println(java.util.Arrays.toString(copyTo));

// ----------------------------------------
```

```
[Cappuccino, Corretto, Cortado, Doppio, Espresso, Frappucino, Freddo]
```

[java_044] Java - Language Basics - Arrays

Creating Arrays in Your Programs: Wrapping-up Variables and Arrays

Sumber: ht

The Java programming language uses both "fields" and "variables" as part of its terminology. Instance variables (non-static fields) are unique to each instance of a class. Class variables (static fields) are fields declared with the static modifier; there is exactly one copy of a class variable, regardless of how many times the class has been instantiated. Local variables store temporary state inside a method. Parameters are variables that provide extra information to a method; both local variables and parameters are always classified as "variables" (not "fields"). When naming your fields or variables, there are rules and conventions that you should (or must) follow.
The eight primitive data types are: `byte`, `short`, `int`, `long`, `float`, `double`, `boolean`, and `char`. The `java.lang.String` class represents character strings. The compiler will assign a reasonable default value for fields of the above types; for local variables, a default value is never assigned.
A literal is the source code representation of a fixed value. An array is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed.

[java_045] Java - Modern Features - Type Inference (var)

Local Variable Type Inference: Overview

Sumber: ht

Local Variable Type Inference

[java_046] Java - Modern Features - Type Inference (var)

Local Variable Type Inference: Style Guidelines {.subtitle}

Sumber: ht

Stuart W. Marks {.author}
March 2018 {.date}

[java_047] Java - Modern Features - Type Inference (var)

Local Variable Type Inference: Introduction

Sumber: ht

Java SE 10 introduced [type inference for local variables][jep286]. Previously,
all local variable declarations required an explicit (manifest) type
on the left-hand side. With type inference, the explicit type can be
replaced by the reserved type name `var` for local variable
declarations that have initializers. The type of the variable is
inferred from the type of the initializer.
There is a certain amount of controversy over this feature. Some
welcome the concision it enables; others fear that it deprives readers
of important type information, impairing readability. And both groups
are right. It can make code more readable by eliminating redundant
information, and it can also make code less readable by eliding useful
information. Another group worries that it will be overused,
resulting in more bad Java code being written. This is also true, but
it's also likely to result in more *good* Java code being written.
Like all features, it must be used with judgment. There's no blanket
rule for when it should and shouldn't be used.
Local variable declarations do not exist in isolation; the surrounding

code can affect or even overwhelm the effects of using `var`. The goal of this document is to examine the impact that surrounding code has on `var` declarations, to explain some of the tradeoffs, and to provide guidelines for the effective use of `var`.

[java_048] Java - Modern Features - Type Inference (var)

Local Variable Type Inference: Principles

Sumber: ht

P1. Reading code is more important than writing code. {#P1}

Code is read much more often than it is written. Further, when writing code, we usually have the whole context in our head, and take our time; when reading code, we are often context-switching, and may be in more of a hurry. Whether and how particular language features are used ought to be determined by their impact on *future readers* of the program, not its original author. Shorter programs can be preferable to longer ones, but shortening a program too much can omit information that's useful for understanding the program. The central issue here is to find the right size for the program such that understandability is maximized.

We are specifically unconcerned here with the amount of keyboarding that's necessary to input or to edit a program. While concision may be a nice bonus for the author, focusing on it misses the main goal, which is to improve the understandability of the resulting program.

P2. Code should be clear from local reasoning. {#P2}

The reader should be able to look at a `var` declaration, along with uses of the declared variable, and understand almost immediately what's going on. Ideally, the code should be readily understandable using only the context from a snippet or a patch. If understanding a `var` declaration requires the reader to look at several locations around the code, it might not be a good situation in which to use `var`. Then again, it might indicate a problem with the code itself.

P3. Code readability shouldn't depend on IDEs. {#P3}

Code is often written and read within an IDE, so it's tempting to rely heavily on code analysis features of IDEs. For type declarations, why not just use `var` everywhere, since one can always point at a variable to determine its type?

There are two reasons. The first is that code is often read outside an IDE. Code appears in many places where IDE facilities aren't available, such as snippets within a document, browsing a repository on the internet, or in a patch file. It is counterproductive to have to import code into an IDE simply to understand what the code does.

The second reason is that even when one is reading code within an IDE, explicit actions are often necessary to query the IDE for further information about a variable. For instance, to query the type of a variable declared using `var`, one might have to hover the pointer over the variable and wait for a popup. This might take only a moment, but it disrupts the flow of reading.

Code should be self-revealing. It should be understandable on its face, without the need for assistance from tools.

P4. Explicit types are a tradeoff. {#P4}

Java has historically required local variable declarations to include

the type explicitly. While explicit types can be very helpful, they are sometimes not very important, and are sometimes just in the way. Requiring an explicit type can add clutter that crowds out useful information.
Omitting an explicit type can reduce clutter, but only if its omission doesn't impair understandability. The type isn't the only way to convey information to the reader. Other means include the variable's name and the initializer expression. We should take all the available channels into account when determining whether it's OK to mute one of these channels.

[java_049] Java - Modern Features - Type Inference (var)

Local Variable Type Inference: Guidelines

Sumber: ht

G1. Choose variable names that provide useful information. {#G1}
This is good practice in general, but it's much more important in the context of `var`. In a `var` declaration, information about the meaning and use of the variable can be conveyed using the variable's name. Replacing an explicit type with var should often be accompanied by improving the variable name. For example:
// ORIGINAL
List<Customer> x = dbconn.executeQuery(query);
// GOOD
var custList = dbconn.executeQuery(query);
In this case, a useless variable name has been replaced with a name that is evocative of the type of the variable, which is now implicit in the `var` declaration.
Encoding the variable's type in its name, taken to its logical conclusion, results in "[Hungarian Notation]". Just as with explicit types, this is sometimes helpful, and sometimes just clutter. In this example the name `custList` implies that a `List` is being returned. That might not be significant. Instead of the exact type, it's sometimes better for a variable's name to express the role or the nature of the variable, such as "customers":
// ORIGINAL
try (Stream<Customer> result = dbconn.executeQuery(query)) {
return result.map(...)
.filter(...)
.findAny();
}
// GOOD
try (var customers = dbconn.executeQuery(query)) {
return customers.map(...)
.filter(...)
.findAny();
}
G2. Minimize the scope of local variables. {#G2}
Limiting the scope of local variables is good practice in general. This practice is described in [*Effective Java (3rd Edition)*][Bloch], Item 57. It applies with extra force if `var` is in use.
In the following example, the `add` method clearly adds the special

item as the last list element, so it's processed last, as expected.

```
var items = new ArrayList<Item>(...);
items.add(MUST_BE_PROCESSED_LAST);
for (var item : items) ...
```

Now suppose that in order to remove duplicate items, a programmer were to modify this code to use a `HashSet` instead of an `ArrayList`:

```
var items = new HashSet<Item>(...);
items.add(MUST_BE_PROCESSED_LAST);
for (var item : items) ...
```

This code now has a bug, since sets don't have a defined iteration order. However, the programmer is likely to fix this bug immediately, as the uses of the `items` variable are adjacent to its declaration.

Now suppose that this code is part of a large method, with a correspondingly large scope for the `items` variable:

```
var items = new HashSet<Item>(...);
// ... 100 lines of code ...
items.add(MUST_BE_PROCESSED_LAST);
for (var item : items) ...
```

The impact of changing from an `ArrayList` to a `HashSet` is no longer readily apparent, since `items` is used so far away from its declaration. It seems likely that this bug could survive for much longer.

If `items` had been declared explicitly as `List<String>`, changing the initializer would also require changing the type to `Set<String>`. This might prompt the programmer to inspect the rest of the method for code that would be impacted by such a change. (Then again, it might not.) Use of `var` would remove this prompting, possibly increasing the risk of a bug being introduced in code like this.

This might seem like an argument against using `var`, but it really isn't. The initial example that uses `var` is perfectly fine. The problem only occurs when the variable's scope is large. Instead of simply avoiding `var` in these cases, one should change the code to reduce the scope of the local variables, and only then declare them with `var`.

G3. Consider `var` when the initializer provides sufficient information to the reader. {#G3}

Local variables are often initialized with constructors. The name of the class being constructed is often repeated as the explicit type on the left-hand side. If the type name is long, use of `var` provides concision without loss of information:

```
// ORIGINAL
ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
// GOOD
var outputStream = new ByteArrayOutputStream();
```

It's also reasonable to use `var` in cases where the initializer is a method call, such as a static factory method, instead of a constructor, and when its name contains enough type information:

```
// ORIGINAL
BufferedReader reader = Files.newBufferedReader(...);
List<String> stringList = List.of("a", "b", "c");
// GOOD
var reader = Files.newBufferedReader(...);
```

```
var stringList = List.of("a", "b", "c");
```

In these cases, the methods' names strongly imply a particular return type, which is then used for inferring the type of the variable.

G4. Use `var` to break up chained or nested expressions with local variables. {#G4}

Consider code that takes a collection of strings and finds the string that occurs most often. This might look like the following:

```
return strings.stream()
.collect(groupingBy(s -> s, counting()))
.entrySet()
.stream()
.max(Map.Entry.comparingByValue())
.map(Map.Entry::getKey);
```

This code is correct, but it's potentially confusing, as it looks like a single stream pipeline. In fact, it's a short stream, followed by a second stream over the result of the first stream, followed by a mapping of the `Optional` result of the second stream. The most readable way to express this code would have been as two or three statements; first group entries into a map, then reduce over that map, then extract the key from the result (if present), as shown below:

```
Map<String, Long> freqMap = strings.stream()
.collect(groupingBy(s -> s, counting()));
Optional<Map.Entry<String, Long>> maxEntryOpt = freqMap.entrySet()
.stream()
.max(Map.Entry.comparingByValue());
return maxEntryOpt.map(Map.Entry::getKey);
```

But the author probably resisted doing that because writing the types of the intermediate variables seemed too burdensome, so instead they distorted the control flow. Using `var` allows us to express the code more naturally without paying the high price of explicitly declaring the types of the intermediate variables:

```
var freqMap = strings.stream()
.collect(groupingBy(s -> s, counting()));
var maxEntryOpt = freqMap.entrySet()
.stream()
.max(Map.Entry.comparingByValue());
return maxEntryOpt.map(Map.Entry::getKey);
```

One might legitimately prefer the first snippet with its single long chain of method calls. However, in some cases it's better to break up long method chains. Using `var` for these cases is a viable approach, whereas using full declarations of the intermediate variables in the second snippet makes it an unpalatable alternative. As with many other situations, the correct use of `var` might involve both taking something out (explicit types) and adding something back (better variable names, better structuring of code).

G5. Don't worry too much about "programming to the interface" with local variables. {#G5}

A common idiom in Java programming is to construct an instance of a concrete type but to assign it to a variable of an interface type. This binds the code to the abstraction instead of the implementation, which preserves flexibility during future maintenance of the code. For example:

```
// ORIGINAL
List<String> list = new ArrayList<>();
```

If `var` is used, however, the concrete type is inferred instead of the interface:

```
// Inferred type of list is ArrayList<String>
var list = new ArrayList<String>();
```

It must be reiterated here that `var` can *only* be used for local variables. It cannot be used to infer field types, method parameter types, and method return types. The principle of "programming to the interface" is still as important as ever in those contexts.

The main issue is that code that uses the variable can form dependencies on the concrete implementation. If the variable's initializer were to change in the future, this might cause its inferred type to change, causing errors or bugs to occur in subsequent code that uses the variable.

If, as recommended in guideline G2, the scope of the local variable is small, the risks from "leakage" of the concrete implementation that can impact the subsequent code are limited. If the variable is used only in code that's a few lines away, it should be easy to avoid problems or to mitigate them if they arise.

In this particular case, `ArrayList` only contains a couple of methods that aren't on `List`, namely `ensureCapacity` and `trimToSize`. These methods don't affect the contents of the list, so calls to them don't affect the correctness of the program. This further reduces the impact of the inferred type being a concrete implementation rather than an interface.

G6. Take care when using `var` with diamond or generic methods. {#G6}

Both `var` and the "diamond" feature allow you to omit explicit type information when it can be derived from information already present. Can you use both in the same declaration?

Consider the following:

```
PriorityQueue<Item> itemQueue = new PriorityQueue<Item>();
```

This can be rewritten using either diamond or `var`, without losing type information:

```
// OK: both declare variables of type PriorityQueue<Item>
PriorityQueue<Item> itemQueue = new PriorityQueue<>();
var itemQueue = new PriorityQueue<Item>();
```

It is legal to use both `var` and diamond, but the inferred type will change:

```
// DANGEROUS: infers as PriorityQueue<Object>
var itemQueue = new PriorityQueue<>();
```

For its inference, diamond can use the target type (typically, the left-hand side of a declaration) or the types of constructor arguments. If neither is present, it falls back to the broadest applicable type, which is often `Object`. This is usually not what was intended.

Generic methods have employed type inference so successfully that it's quite rare for programmers to provide explicit type arguments. Inference for generic methods relies on the target type if there are no actual method arguments that provide sufficient type information. In a `var` declaration, there is no target type, so a similar issue can occur as with diamond. For example,

```
// DANGEROUS: infers as List<Object>
var list = List.of();
```

With both diamond and generic methods, additional type information can

be provided by actual arguments to the constructor or method, allowing the intended type to be inferred. Thus,

```
// OK: itemQueue infers as PriorityQueue<String>
Comparator<String> comp = ... ;
var itemQueue = new PriorityQueue<>(comp);
// OK: infers as List<BigInteger>
var list = List.of(BigInteger.ZERO);
```

If you decide to use `var` with diamond or a generic method, you should ensure that method or constructor arguments provide enough type information so that the inferred type matches your intent. Otherwise, avoid using both `var` with diamond or a generic method in the same declaration.

G7. Take care when using `var` with literals. {#G7}

Primitive literals can be used as initializers for `var` declarations. It's unlikely that using `var` in these cases will provide much advantage, as the type names are generally short. However, `var` is sometimes useful, for example, to align variable names.

There is no issue with boolean, character, `long`, and string literals. The type inferred from these literals is precise, and so the meaning of `var` is unambiguous:

```
// ORIGINAL
boolean ready = true;
char ch = '\ufffd';
long sum = 0L;
String label = "wombat";
// GOOD
var ready = true;
var ch    = '\ufffd';
var sum   = 0L;
var label = "wombat";
```

Particular care should be taken when the initializer is a numeric value, especially an integer literal. With an explicit type on the left-hand side, the numeric value may be silently widened or narrowed to types other than `int`. With `var`, the value will be inferred as an `int`, which may be unintended.

```
// ORIGINAL
byte flags = 0;
short mask = 0x7fff;
long base = 17;
// DANGEROUS: all infer as int
var flags = 0;
var mask = 0x7fff;
var base = 17;
```

Floating point literals are mostly unambiguous:

```
// ORIGINAL
float f = 1.0f;
double d = 2.0;
// GOOD
var f = 1.0f;
var d = 2.0;
```

Note that `float` literals can be widened silently to `double`. It is

somewhat obtuse to initialize a `double` variable using an explicit `float` literal such as `3.0f`, however, cases may arise where a `double` variable is initialized from a `float` field. Caution with `var` is advised here:

```
// ORIGINAL
static final float INITIAL = 3.0f;
...
double temp = INITIAL;
// DANGEROUS: now infers as float
var temp = INITIAL;
```

(Indeed, this example violates guideline G3, because there isn't enough information in the initializer for a reader to see the inferred type.)

[java_050] Java - Modern Features - Type Inference (var)

Local Variable Type Inference: Examples

Sumber: ht

This section contains some examples of where `var` can be used to greatest benefit.

The following code removes at most `max` matching entries from a Map. Wildcarded type bounds are used for improving the flexibility of the method, resulting in considerable verbosity. Unfortunately, this requires the type of the Iterator to be a nested wildcard, making its declaration more verbose. This declaration is so long that the header of the for-loop no longer fits on a single line, making the code even harder to read.

```
// ORIGINAL
void removeMatches(Map<? extends String, ? extends Number> map, int max) {
for (Iterator<? extends Map.Entry<? extends String, ? extends Number>> iterator =
map.entrySet().iterator(); iterator.hasNext();) {
Map.Entry<? extends String, ? extends Number> entry = iterator.next();
if (max > 0 && matches(entry)) {
iterator.remove();
max--;
}
}
}
```

Use of `var` here removes the noisy type declarations for the local variables. Having explicit types for the Iterator and Map.Entry locals in this kind of loop is largely unnecessary. This also allows the for-loop control to fit on a single line, further improving readability.

```
// GOOD
void removeMatches(Map<? extends String, ? extends Number> map, int max) {
for (var iterator = map.entrySet().iterator(); iterator.hasNext();) {
var entry = iterator.next();
if (max > 0 && matches(entry)) {
iterator.remove();
max--;
}
}
```

```
}
```

Consider code that reads a single line of text from a socket using
the try-with-resources statement. The networking and I/O APIs use an
object wrapping idiom. Each intermediate object must be declared as a
resource variable so that it will be closed properly if an error
occurs while opening a subsequent wrapper. The conventional code for
this requires the class name to be repeated on the left and right
sides of the variable declaration, resulting in a lot of clutter:

```
// ORIGINAL
try (InputStream is = socket.getInputStream();
InputStreamReader isr = new InputStreamReader(is, charsetName);
BufferedReader buf = new BufferedReader(isr)) {
return buf.readLine();
}
```

Using `var` reduces the noise considerably:

```
// GOOD
try (var inputStream = socket.getInputStream();
var reader = new InputStreamReader(inputStream, charsetName);
var bufReader = new BufferedReader(reader)) {
return bufReader.readLine();
}
```

[java_051] Java - Modern Features - Type Inference (var)

Local Variable Type Inference: Conclusion

Sumber: ht

Using `var` for declarations can improve code by reducing clutter,
thereby letting more important information stand out. On the other
hand, applying `var` indiscriminately can make things worse. Used
properly, `var` can help improve good code, making it shorter and
clearer without compromising understandability.

[jep286]: https://openjdk.java.net/jeps/286
[Hungarian Notation]: https://en.wikipedia.org/wiki/Hungarian_notation
[Bloch]: https://www.pearson.com/us/higher-education/program/Bloch-Effective-Java-3rd-Edition/PGM1763855.html

[java_052] Java - Modern Features - Text Blocks

Programmer's Guide to Text Blocks: Overview

Sumber: ht

Programmer's Guide to Text Blocks
Jim Laskey and Stuart Marks {.author}
Version 11 (August 2020) {.date}

JEP 378 introduces _text blocks_
into Java SE 15 as a Standard feature. While the JEP explains the feature
in great detail, it's not always apparent how the feature can and should
be used. This guide assembles practical usage advice for text blocks,
along with some style guidelines.

Contents

[java_053] Java - Modern Features - Text Blocks

Programmer's Guide to Text Blocks: Introduction

Sumber: ht

A text block's _principalis munus_ is to provide clarity by way of minimizing the Java syntax required to render a string that spans multiple lines.
In earlier releases of the JDK, embedding multi-line code snippets required a tangled mess of explicit line terminators, string concatenations, and delimiters. Text blocks eliminate most of these obstructions, allowing you to embed code snippets and text sequences more or less as-is.
A text block is an alternative form of Java string representation that can be used anywhere a traditional double-quoted string literal can be used. For example:

```
// Using a literal string
String dqName = "Pat Q. Smith";
// Using a text block
String tbName = """
Pat Q. Smith""";
```

The object produced from a text block is a `java.lang.String` with the same characteristics as a traditional double-quoted string. This includes object representation and interning. Continuing with `dqName` and `tbName` from the examples above,

```
// Both dqName and tbName are strings of equal value
dqName.equals(tbName)    // true
// Both dqName and tbName intern to the same string
dqName == tbName         // true
```

Text blocks can be used anywhere a string literal can be used. For example, text blocks may be intermixed with string literals in a string concatenation expression:

```
String str = "The old";
String tb = """
the new""";
String together = str + " and " + tb + ".";
```

Text blocks may be used as a method argument:

```
System.out.println("""
This is the first line
This is the second line
This is the third line
""");
```

String methods may be applied to a text block:

```
"""
John Q. Smith""".substring(8).equals("Smith")    // true
```

A text block can be used in place of a string literal to improve the readability and clarity of the code. This primarily occurs when a string literal is used to represent a multi-line string. In this case there is considerable clutter from quotation marks, newline escapes, and concatenation operators:

```
// ORIGINAL
String message = "'The time has come,' the Walrus said,\n" +
"'To talk of many things:\n" +
```

```
"Of shoes -- and ships -- and sealing-wax --\n" +
"Of cabbages -- and kings --\n" +
"And why the sea is boiling hot --\n" +
"And whether pigs have wings.'\n";
```

Using text blocks removes much of the clutter:

```
// BETTER
String message = """
'The time has come,' the Walrus said,
'To talk of many things:
Of shoes -- and ships -- and sealing-wax --
Of cabbages -- and kings --
And why the sea is boiling hot --
And whether pigs have wings.'
""";
```

[java_054] Java - Modern Features - Text Blocks

Programmer's Guide to Text Blocks: Using Text Blocks

Sumber: ht

Preview Feature in Java SE 13 and 14

Text blocks existed as a Preview
feature of the Java Language in Java SE 13 and 14. In order to use text
blocks in those versions of Java SE, you must use the `--enable-preview`
and `-source` flags on the `javac` command line and the
`--enable-preview` flag on the `java` command line:

```
javac --enable-preview -source 14 -d classes TextBlockExample.java
java --enable-preview -cp classes com.example.TextBlockExample
```

Using Java SE 13 or 14 `jshell` to experiment with text blocks requires
using the `--enable-preview` flag as well:

```
jshell --enable-preview
```

Use `--enable-preview` and `--source` flags (note: two hyphens) with the
Java SE 13 and 14 `java` command to compile and run a single source file
with a main program:

```
java --enable-preview --source 14 TextBlockExample.java
```

(Single-file source code programs are described in
JEP 330.)

Standard Feature in Java SE 15 and Beyond

In Java SE 15 and beyond, the `--enable-preview` and `-source` flags are
no longer required to use text blocks:

```
javac -d classes TextBlockExample.java
java -cp classes com.example.TextBlockExample
```

To use `jshell` to experiment with text blocks simply requires using the
`jshell` command with no flags:

```
jshell
```

The `java` command can be used to compile and run a single source file
with a main program, again with no additional flags:

```
java TextBlockExample.java
```

(Single-file source code programs are described in
JEP 330.)

Text Block Syntax

A text block begins with three double-quote characters followed by a
line terminator. You can't put a text block on a single line, nor can

the contents of the text block follow the three opening double-quotes
without an intervening line terminator. The reason for this is that text
blocks are primarily designed to support multi-line strings, and
requiring the initial line terminator simplifies the indentation
handling rules (see the section below, _Incidental White Space_).
// ERROR
String name = """Pat Q. Smith""";
// ERROR
String name = """red
green
blue
""";
// OK
String name = """
red
green
blue
""";
This last example is equivalent to the following string literal:
String name = "red\n" +
"green\n" +
"blue\n";
Here's an example of a snippet of Java code within a text block:
String source = """
String message = "Hello, World!";
System.out.println(message);
""";
Note that there is no need to escape the embedded double quotes. The
equivalent string literal would be:
String source = "String message = \"Hello, World!\";\n" +
"System.out.println(message);\n";
That Final New Line
Note that the example above,
String name = """
red
green
blue
""";
is equivalent to `"red\ngreen\nblue\n"`. What if you want to represent a
multi-line string without that final `\n`?
String name = """
red
green
blue""";
This text block is equivalent to `"red\ngreen\nblue"`. Thus, placing
the closing delimiter on the last visible line effectively drops the
last `\n`. However, as described in guideline G12 below,
a better alternative is to use the `\<line-terminator>` escape sequence
(see New Escape Sequences).
Incidental White Space
Ideally, a text block would be indented to match the indentation of the
surrounding code. For example:

```
void writeHTML() {
String html = """
<html>
<body>
<p>Hello World.</p>
</body>
</html>
""";
writeOutput(html);
}
```

However, this raises the question of how spaces used for indentation affect the contents of the string. A naïve interpretation would include all of this whitespace in the text block. The consequence would be that reindenting the code would affect the contents of the text block. This is quite likely to be an error.

To avoid this problem, a text block differentiates _incidental_ white space from _essential_ white space. The Java compiler automatically strips away the incidental white space. The indentation to the left of `<html>` and `</html>` is considered incidental, since these lines are indented the least. Thus, they effectively determine the left margin of the text in the text block. However, the indentation of `<body>` relative to `<html>` is _not_ considered to be incidental white space. Presumably, this relative indentation is intended to be part of the string's contents.

The example below uses "`*`" to visualize the incidental white space, with essential white space shown as actual white space.

```
void writeHTML() {
String html = """
********<html>
********    <body>
********        <p>Hello World.</p>
********    </body>
********</html>
********""";
writeOutput(html);
}
```

After the incidental white space is stripped, the resulting contents of the text block are as follows:

```
<html>
<body>
<p>Hello World.</p>
</body>
</html>
```

The algorithm for determining incidental white space is described in JEP 355 in scary detail. Nevertheless, the net effect is quite simple. The entire contents of the text block is shifted to the left until the line with the least leading white space has no leading white space.

To preserve some white space and not have it be considered incidental white space, simply shift the content lines of the text block to the right, while keeping the closing triple-quote delimiter at the indentation appropriate for the surrounding code. For example:

```
void writeHTML() {
```

```
String html = """
********    <html>
********        <body>
********            <p>Hello World.</p>
********        </body>
********    </html>
********""";
writeOutput(html);
}
```

results in the following:

```
<html>
<body>
<p>Hello World.</p>
</body>
</html>
```

A text block can _opt out_ of incidental white space stripping by positioning the closing delimiter in the first character position of a source line:

```
void writeHTML() {
String html = """
<html>
<body>
<p>Hello World.</p>
</body>
</html>
""";
writeOutput(html);
}
```

The result is that there is no incidental white space that is stripped, and the string includes leading white space on each line.

```
<html>
<body>
<p>Hello World.</p>
</body>
</html>
```

This technique for controlling the amount of indentation that is preserved only works if the last line of the text block ends with a line terminator. If the last line does _not_ end with a line terminator, you need to use `String::indent` to control the indentation explicitly. In the following example,

```
String colors = """
red
green
blue""";
```

all of the indentation is treated as incidental and is stripped away:

```
red
green
blue
```

To include some indentation in the string's contents, invoke the `indent` method on the text block:

```
String colors = """
red
```

green
blue""".indent(4);
This results in:
red
green
blue
Trailing White Space
Trailing white space on each line in a text block is also considered incidental and is stripped away by the Java compiler. This is done so that the contents of the text block are always visually discernible. If this were not done, a text editor that automatically strips trailing white space could invisibly change the contents of a text block.
If you need to have trailing white space in a text block, then you can use one of the following strategies:
// new escape sequence for space, see below
String q = """
trailing\s\s\s
white space
""";
// character substitution
String r = """
trailing$$$
white space
""".replace('$', ' ');
// character fence
String s = """
trailing |
white space|
""".replace("|\n", "\n");
// octal escape sequence for space
String t = """
trailing\040\040\040
white space
""";
Note: `\u0020` cannot be used because Unicode escapes are translated early during source file reading, prior to lexical analysis. By contrast, character and string escapes such as `\040` are processed after lexical analysis has divided the source file into tokens and has identified string literals and text blocks.
Detecting Potential Issues with White Space
In the preceding examples, all indentation consisted of space characters. However, sometimes people use TAB `\t` characters. Unfortunately it is not possible for the Java compiler to know how tab characters are displayed in different editors. Therefore, the rule is that each individual white space character is treated equally. A single space character is treated the same as a single tab character, even though the latter might result in white space equivalent up to eight spaces when displayed on some particular system.
It follows that mixing white space characters can have inconsistent and unintended effects. Consider the following example, in which some lines are indented with spaces and some with tabs (which are visualized with

` |-> `):
String colors = """
**********red
 |-> |-> |-> |-> |->   green
**********blue""";
In this case, stripping of incidental indentation would be uneven since the second line only has five white space characters and the others have twenty. The result would look something like this:
red
green
blue
It is possible to detect issues related to incidental white space by turning on text block lint detection with a Java compiler lint flag, `-Xlint:text-blocks`. If lint detection is on, then the above example will generate a warning, "inconsistent white space indentation". This lint flag also enables another warning, "trailing white space will be removed", which will be emitted if there is trailing white space on any line within a text block. If you need to preserve trailing white space, use one of the escaping or replacement techniques described in the section above.
Normalization Of Line Terminators
One of the complications of a multi-line string literal is that the line terminator (`\n`, `\r`, or `\r\n`) used in the source file varies from platform to platform. Editors on different platforms may invisibly change line terminators. Or, if a source file is edited on different platforms, a text block might contain a mixture of different line terminators. This is likely to produce confusing and inconsistent results.
To avoid these problems, the Java compiler normalizes all line terminators in a text block to be `\n`, regardless of what line terminators actually appear in the source file. The following text block (where ` | ` and ` <- ` represent `\n` and `\r`):
String colors = """
red |
green <-
blue <- |
""";
is equivalent to this string literal:
String colors = "red\ngreen\nblue\n";
If the platform line terminator is required then `String::replaceAll("\n", System.lineSeparator())` can be used.
Translation Of Escape Sequences
As with string literals, text blocks recognize the escape sequences, `\b`, `\f`, `\n`, `\t`, `\r`, `\"`, `\'`, `\\`, and octal escapes. Unlike string literals, escapes sequences are often not required. Under most circumstances, the actual characters `\n`, `\t`, `\"`, and `\'` can be used instead of escape sequences. The following text block (where ` |-> ` and ` | ` represent `\t` and `\n`):
String s = """
Color |->   Shape |
Red |-> |->   Circle |
Green |->   Square |

```
Blue |->  |->    Triangle |
""";
```

results in:

```
Color |->   Shape |
Red |->   |->   Circle |
Green |->   Square |
Blue |->  |->   Triangle |
```

Escaping is required when three or more double quotes occur consecutively.

```
String code = """
String source = \"""
String message = "Hello, World!";
System.out.println(message);
\""";
""";
```

Escape translation occurs as the last step of processing by the Java compiler, so you can bypass the line terminator normalization and whitespace stripping steps by using explicit escape sequences. For example:

```
String s = """
red  \040
green\040
blue \040
""";
```

would guarantee that all lines are of equal length since the `\040` does not get translated to a space until after stripping of trailing white space ("`*`" is used to show trailing space). The result is:

```
red***
green*
blue**
```

Note: As noted previously, the Unicode escape sequence `\u0020` _cannot_ be used as a substitute for `\040`.

New Escape Sequences

The `\<line-terminator>` escape sequence explicitly suppresses the inclusion of an implicit new line character.

For example, it is common practice to split very long string literals into concatenations of smaller substrings and then hard wrapping the resulting string expression onto multiple lines.

```
String literal = "Lorem ipsum dolor sit amet, consectetur adipiscing " +
"elit, sed do eiusmod tempor incididunt ut labore " +
"et dolore magna aliqua.";
```

With the `\<line-terminator>` escape sequence this could be expressed as;

```
String text = """
Lorem ipsum dolor sit amet, consectetur adipiscing \
elit, sed do eiusmod tempor incididunt ut labore \
et dolore magna aliqua.\
""";
```

The `\s` escape sequence simply translates to space (`\040`, ASCII character 32, white space). Since escape sequences don't get translated until after incidental space stripping, `\s` can act as fence to prevent the stripping of trailing white space. Using `\s` at the end of each line in the following example, guarantees that each line is exactly six characters long.

```
String colors = """
```

```
red  \s
green\s
blue \s
""";
```

[java_055] Java - Modern Features - Text Blocks

Programmer's Guide to Text Blocks: Style Guidelines for Text Blocks {toc=omit-children}

Sumber: ht

G1. You should use a text block when it improves the clarity of the code, particularly with multi-line strings. {#G1}

```
// ORIGINAL
String message = "'The time has come,' the Walrus said,\n" +
"'To talk of many things:\n" +
"Of shoes -- and ships -- and sealing-wax --\n" +
"Of cabbages -- and kings --\n" +
"And why the sea is boiling hot --\n" +
"And whether pigs have wings.'\n";
// BETTER
String message = """
'The time has come,' the Walrus said,
'To talk of many things:
Of shoes -- and ships -- and sealing-wax --
Of cabbages -- and kings --
And why the sea is boiling hot --
And whether pigs have wings.'
""";
```

G2. If a string fits on a single line, without concatenation and escaped newlines, you should probably continue to use a string literal. {#G2}

```
// ORIGINAL - is a text block helpful here?
String name = """
Pat Q. Smith""";
// BETTER - a string literal works fine
String name = "Pat Q. Smith";
```

G3. Use embedded escape sequences when they maintain readability. {#G3}

```
var data = """
Name | Address | City
Bob Smith | 123 Anytown St\nApt 100 | Vancouver
Jon Brown | 1000 Golden Place\nSuite 5 | Santa Ana
""";
```

G4. For most multi-line strings, place the opening delimiter at the right end of the previous line, and place the closing delimiter on its own line, at the left margin of the text block. {#G4}

```
String string = """
red
green
blue
""";
```

G5. Avoid aligning the opening and closing delimiters and the text block's left margin. This requires reindentation of the text block if the variable name or modifiers are changed. {#G5}

```
// ORIGINAL
String string = """
red
```

```
green
blue
""";
// ORIGINAL - after variable declaration changes
static String rgbNames = """
red
green
blue
""";
// BETTER
String string = """
red
green
blue
""";
// BETTER - after variable declaration changes
static String rgbNames = """
red
green
blue
""";
```

G6. Avoid in-line text blocks within complex expressions, as doing so can distort readability. Consider refactoring to a local variable or to a static final field. {#G6}

```
// ORIGINAL
String poem = new String(Files.readAllBytes(Paths.get("jabberwocky.txt")));
String middleVerses = Pattern.compile("\\n\\n")
.splitAsStream(poem)
.match(verse -> !"""
'Twas brillig, and the slithy toves
Did gyre and gimble in the wabe;
All mimsy were the borogoves,
And the mome raths outgrabe.
""".equals(verse))
.collect(Collectors.joining("\n\n"));
// BETTER
String firstLastVerse = """
'Twas brillig, and the slithy toves
Did gyre and gimble in the wabe;
All mimsy were the borogoves,
And the mome raths outgrabe.
""";
String poem = new String(Files.readAllBytes(Paths.get("jabberwocky.txt")));
String middleVerses = Pattern.compile("\\n\\n")
.splitAsStream(poem)
.match(verse -> !firstLastVerse.equals(verse))
.collect(Collectors.joining("\n\n"));
```

G7. Either use only spaces or only tabs for the indentation of a text block. Mixing white space will lead to a result with irregular indentation. {#G7}

```
// ORIGINAL
String colors = """
********red
 |->       green
```

```
********blue""";    // result: "*******red\ngreen\n*******blue"
// PROBABLY WHAT WAS INTENDED
String colors = """
********red
********green
********blue""";    // result: "red\ngreen\nblue"
```

G8. When a text block contains sequences of three or more double quotes, escape the first double quote of every run of three double quotes. {#G8}

```
// ORIGINAL
String code = """
String source = \"\"\"
String message = "Hello, World!";
System.out.println(message);
\"\"\";
""";
// BETTER
String code = """
String source = \"""
String message = "Hello, World!";
System.out.println(message);
\""";
""";
```

G9. Most text blocks should be indented to align with neighbouring Java code. {#G9}

```
// ORIGINAL - odd indentation
void printPoem() {
String poem = """
'Twas brillig, and the slithy toves
Did gyre and gimble in the wabe;
All mimsy were the borogoves,
And the mome raths outgrabe.
""";
System.out.print(poem);
}
// BETTER
void printPoem() {
String poem = """
'Twas brillig, and the slithy toves
Did gyre and gimble in the wabe;
All mimsy were the borogoves,
And the mome raths outgrabe.
""";
System.out.print(poem);
}
```

G10. It is recommended to fully left justify a wide string in order to avoid horizontal scrolling or line wrapping. {#G10}

```
// ORIGINAL
class Outer {
class Inner {
void printPoetry() {
String lilacs = """
Over the breast of the spring, the land, amid cities,
Amid lanes and through old woods, where lately the violets peep'd from the ground, spotting the gray debris,
```

```
Amid the grass in the fields each side of the lanes, passing the endless grass,
Passing the yellow-spear'd wheat, every grain from its shroud in the dark-brown fields uprisen,
Passing the apple-tree blows of white and pink in the orchards,
Carrying a corpse to where it shall rest in the grave,
Night and day journeys a coffin.
""";
System.out.println(lilacs);
}
}
}
// BETTER
class Outer {
class Inner {
void printPoetry() {
String lilacs = """
Over the breast of the spring, the land, amid cities,
Amid lanes and through old woods, where lately the violets peep'd from the ground, spotting the gray debris,
Amid the grass in the fields each side of the lanes, passing the endless grass,
Passing the yellow-spear'd wheat, every grain from its shroud in the dark-brown fields uprisen,
Passing the apple-tree blows of white and pink in the orchards,
Carrying a corpse to where it shall rest in the grave,
Night and day journeys a coffin.
""";
System.out.println(lilacs);
}
}
}
```

G11. Similarly, it is also reasonable to fully left justify a text block when a high line count causes the closing delimiter is likely to vertically scroll out of view. This allows the reader to track indentation with the left margin when the closing delimiter is out of view. {#G11}

```
// ORIGINAL
String validWords = """
aa
aah
aahed
aahing
aahs
aal
aalii
aaliis
...
zythum
zythums
zyzzyva
zyzzyvas
zzz
zzzs
""";
// BETTER
String validWords = """
aa
aah
```

```
aahed
aahing
aahs
aal
aalii
aaliis
...
zythum
zythums
zyzzyva
zyzzyvas
zzz
zzzs
""";
```

G12. The `\<line-terminator>` escape sequence should be used when a text block's final new line needs to be excluded. This better frames the text block and allows the closing delimiter to manage indentation. {#G12}

```
// ORIGINAL
String name = """
red
green
blue""".indent(4);
// BETTER
String name = """
red
green
blue\
""";
```

[java_056] Java - Modern Features - Text Blocks

Programmer's Guide to Text Blocks: New String Methods

Sumber: ht

Several new methods are included in the `String` class as part of the text blocks feature. (Note that these methods are marked deprecated, for removal, in Java SE 13 and 14, to indicate that they are part of a preview feature.)

`String formatted(Object... args)`

This method is equivalent to `String.format(this, args)`. The advantage is that, as an instance method, it can be chained off the end of a text block:

```
String output = """
Name: %s
Phone: %s
Address: %s
Salary: $%.2f
""".formatted(name, phone, address, salary);
```

`String stripIndent()`

The `stripIndent` method removes incidental white space from a multi-line string, using the same algorithm used by the Java compiler. This is useful if you have a program that reads text as input data and you want to strip indentation in the same manner as is done for text blocks.

`String translateEscapes()`

The `translateEscapes` method performs the translation of escape
sequences (`\b`, `\f`, `\n`, `\t`, `\r`, `\"`, `\'`, `\\`, `\s`,
`\<line-terminator>` and octal
escapes) and is used by the Java compiler to process text blocks and
string literals. This is useful if you have a program that reads text
as input data and you want to perform escape sequence processing. Note
that Unicode escapes (`\uNNNN`) are _not_ processed.

[java_057] Java - Modern Features - Text Blocks

Programmer's Guide to Text Blocks: References

Sumber: ht

"The Walrus and the Carpenter"
: Lewis Carroll, _Through the Looking-Glass and What Alice Found There,_ 1872.
"Jabberwocky"
: Lewis Carroll, _Mischmasch,_ 1855.
"When Lilacs Last in the Dooryard Bloom'd"
: Walt Whitman, _Sequel to Drum-Taps_, 1865.

[java_058] Java - Object-Oriented - Enums

Enums: What are enums?

Sumber: ht

Enums are classes where all instances are known to the compiler.
They are used for creating types that can only have few possible values.
Enums can be created similar to classes but use the `enum` keyword instead of `class`.
In the body, there is a list of instances of the enum called enum constants which are seperated by `,`.
No instances of the enum can be created outside of enum constants.
All enums implicitly extend `java.lang.Enum` and cannot have any subclasses.

Contoh Kode / Perintah CLI:

```
public enum DayOfWeek {
    // enum constants are listed here:
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
}
```

[java_059] Java - Object-Oriented - Enums

Enums: Accessing, evaluating, and comparing enums

Sumber: ht

The values of an enum can be used as constants.
In order to check whether two instances of an enum are the same, the `==` operator can be used.
It is also possible to use `switch` for performing actions depending on the value of the enum.
With switch expressions,
the compiler can check whether all values of the enum are handled.
If any possible value is missing in a switch expression, there will be a compiler error.
This is referred to as exhaustiveness checking and can also be achieved with regular classes
through sealed classes and pattern matching.

Contoh Kode / Perintah CLI:

```
DayOfWeek weekStart = DayOfWeek.MONDAY;

if (weekStart == DayOfWeek.MONDAY) {
    System.out.println("The week starts on Monday.");
}
```

```
// ----------------------------------------

DayOfWeek someDay = DayOfWeek.FRIDAY;

switch (someDay) {
    case MONDAY ->
        System.out.println("The week just started.");
    case TUESDAY, WEDNESDAY, THURSDAY ->
        System.out.println("We are somewhere in the middle of the week.");
    case FRIDAY ->
        System.out.println("The weekend is near.");
    case SATURDAY, SUNDAY ->
        System.out.println("Weekend");
    default ->
        throw new AssertionError("Should not happen");
}

// ----------------------------------------

DayOfWeek someDay = DayOfWeek.FRIDAY;

String text = switch (someDay) {
    case MONDAY -> "The week just started.";
    case TUESDAY, WEDNESDAY, THURSDAY -> "We are somewhere in the middle of the week.";
    case FRIDAY -> "The weekend is near.";
    case SATURDAY, SUNDAY -> "Weekend";
};

System.out.println(text);
```

[java_060] Java - Object-Oriented - Enums

Enums: Adding members to enums

Sumber: ht

Just like classes, enums can have constructors, methods and fields.
In order to add these, it is necessary to add a `;` after the list of enum constants.
Arguments to the constructor are passed in parenthesis after the declaration of the enum constant.

Contoh Kode / Perintah CLI:

```
public enum DayOfWeek {
    MONDAY("MON"), TUESDAY("TUE"), WEDNESDAY("WED"), THURSDAY("THU"), FRIDAY("FRI"),
    SATURDAY("SAT"), SUNDAY("SUN");

    private final String abbreviation;

    DayOfWeek(String abbreviation) {
        this.abbreviation = abbreviation;
    }

    public String getAbbreviation() {
        return abbreviation;
    }
}
```

[java_061] Java - Object-Oriented - Enums

Enums: Special methods

Sumber: ht

All enums have a few methods that are added implicitly.
For example, the method `name()` is present in all enum instances and can be used to get the name of the enum constant.
Similarly, a method named `ordinal()` returns the position of the enum constant in the declaration.
Aside from instance methods, there are also static methods added to all enums.
The method `values()` returns an array containing all instances of the enum and the method `valueOf(String)` can be used to get a specific instance by its name.
Furthermore, enums implement the interface `Comparable`.
By default, enums are ordered according to their ordinal number
i.e. in the order of occurrence of the enum constant.
This allows for comparing instances of enums as well as sorting or searching.

Contoh Kode / Perintah CLI:

```
System.out.println(DayOfWeek.MONDAY.name());    // prints "MONDAY"
System.out.println(DayOfWeek.MONDAY.ordinal()); // prints "0" because MONDAY is the first constant in the DayOfWeek enum

// ----------------------------------------

DayOfWeek[] days = DayOfWeek.values(); // all days of the week
DayOfWeek monday = DayOfWeek.valueOf("MONDAY");

// ----------------------------------------

public void compareDayOfWeek(DayOfWeek dayOfWeek){
    int comparison = dayOfWeek.compareTo(DayOfWeek.WEDNESDAY);
    if (comparison < 0) {
        System.out.println("It's before the middle of the work week.");
    } else if (comparison > 0) {
        System.out.println("It's after the middle of the work week.");
    } else {
        System.out.println("It's the middle of the work week.");
    }
}

// ----------------------------------------

List<DayOfWeek> days = new ArrayList<>(List.of(DayOfWeek.FRIDAY, DayOfWeek.TUESDAY, DayOfWeek.SATURDAY));
Collections.sort(days);
```

[java_062] Java - Object-Oriented - Enums

Enums: Using enums as singletons

Sumber: ht

Since enums can only have a specific number of instances, it is possible to create a singleton by creating an enum with only a single enum constant.

Contoh Kode / Perintah CLI:

```
public enum SomeSingleton {
    INSTANCE;
    //fields, methods, etc.
}
```

[java_063] Java - Object-Oriented - Enums

Enums: Abstract methods in enums

Sumber: ht

Even though enums cannot be extended, they can still have `abstract` methods. In that case, an implementation must be present in each enum constant.

Contoh Kode / Perintah CLI:

```
enum MyEnum {
    A() {
        @Override
        void doSomething() {
            System.out.println("a");
        }
    },
    B() {
        @Override
        void doSomething() {
            System.out.println("b");
        }
    };

    abstract void doSomething();
}
```

[java_064] Java - Object-Oriented - Enums

Enums: Precautions

Sumber: ht

Care should be taken when using enums where the number (or names) of instances is subject to change.
Whenever enum constants are changed, other code expecting the old version of the enum might not work as expected.
This may manifest in compilation errors (e.g. when referencing a removed enum constant),
runtime errors (e.g. if there is a `default` case even though the new enum constant should be handled separately)
or other inconsistencies (e.g. if the value of the enum was saved to a file which is then read and expecting that value to still exist).
When changing enum constants, it is recommended to review all code using the enum.
This is especially important in cases where the enum is also used by other people's code.
Furthermore, it might be worth considering to use other options
in case of many instances since listing a lot of instances at a single location in code can be inflexible.
For example, it may be better to use a configuration file for listing all instances
and reading these configuration files in the program in cases like this.

[java_065] Java - Object-Oriented - Enums

Enums: Conclusion

Sumber: ht

Enums provide a simple and safe way of representing a fixed set of constants while keeping most of the flexibilities of classes. They are a special type of class that can be used to write code that is elegant, readable, maintainable and works well with other modern Java features like switch expressions. Another special class is the Record class introduced in JDK 16. Visit our records tutorial to learn more.
To learn more about enums, visit the `java.lang.Enum` javadoc.

[java_066] Java - Modern OOP - Records

JEP 395: Records: Overview & Metadata

Sumber: ht

<noscript></noscript>
TABLE { border-collapse: collapse; padding: 0px; margin: 1em 0 1em 2em; }
TR:first-child TH, TR:first-child TD { padding-top: 0; }
TH, TD { padding: 0px; padding-top: .5ex; vertical-align: baseline; text-align: left; }
TD + TD, TH + TH { padding-left: 1em; }
TD:first-child, TH:first-child, TD.jep { text-align: right; }
TABLE.head TD:first-child { font-style: italic; padding-left: 2em; white-space: nowrap; }
PRE { padding-left: 2em; margin: 1ex 0; font-size: inherit; }
TABLE PRE { padding-left: 0; margin: 0; }
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { padding-left: .5em; }
TABLE.jeps TD:first-child,
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { font-size: smaller; }
TABLE.jeps TD.cl { font-size: smaller; padding-right: 0; text-align: right; }
TABLE.jeps TD.cm { font-size: smaller; padding-left: .1em; padding-right: .1em; }
TABLE.jeps TD.cr { font-size: smaller; padding-left: 0; }
TABLE.jeps TD.z { padding-left: 0; padding-right: 0; }
TABLE.head TD { padding-top: 0; }
JEP 395: Records
Owner
Gavin Bierman
Type
Feature
Scope
SE
Status
Closed / Delivered
Release
16
Component
specification / language
Discussion
amber dash dev at openjdk dot java dot net
Relates to
JEP 359: Records (Preview)
JEP 384: Records (Second Preview)
Reviewed by
Alex Buckley, Brian Goetz
Endorsed by
Brian Goetz
Created
2020/06/08 16:07
Updated
2026/09/03 21:20
Issue
8246771

[java_067] Java - Modern OOP - Records

JEP 395: Records: Summary

Sumber: ht

Enhance the Java programming language with
records
, which are classes
that act as transparent carriers for immutable data. Records can be thought of
as
nominal tuples
.

[java_068] Java - Modern OOP - Records

JEP 395: Records: History

Sumber: ht

Records were proposed by
JEP 359
and delivered in
JDK 14
as a
preview feature
.
In response to feedback, the design was refined by
JEP 384
and delivered in
JDK 15
as a
preview feature for a second time. The refinements for the second preview were as follows:
In the first preview, canonical constructors were required to be
public
.
In the second preview, if the canonical constructor is
implicitly declared then its access modifier is the same as the record class; if
the canonical constructor is explicitly declared then its access modifier must provide
at least as much access as the record class.
The meaning of the
@Override
annotation was extended to include
the case where the annotated method is an explicitly declared accessor method for
a record component.
To enforce the intended use of compact constructors, it became a compile-time
error to assign to any of the instance fields in the constructor body.
The ability to declare local record classes, local enum classes, and local interfaces
was introduced.
This JEP proposes to finalize the feature in JDK 16, with the following refinement:
Relax the longstanding restriction whereby an inner class
cannot declare a member that is explicitly or implicitly static. This will become legal
and, in particular, will allow an inner class to declare a member that is a record class.
Additional refinements may be incorporated based on further feedback.

[java_069] Java - Modern OOP - Records

JEP 395: Records: Goals

Sumber: ht

Devise an object-oriented construct that expresses a simple aggregation of values.
Help developers to focus on modeling immutable data rather than extensible behavior.
Automatically implement data-driven methods such as
equals
and accessors.
Preserve long-standing Java principles such as nominal typing and migration compatibility.

[java_070] Java - Modern OOP - Records

JEP 395: Records: Non-Goals

Sumber: ht

While records do offer improved concision when declaring data carrier classes, it is not a goal to declare a "war on boilerplate". In particular, it is not a goal to address the problems of mutable classes which use the JavaBeans naming conventions.
It is not a goal to add features such as properties or annotation-driven code generation, which are often proposed to streamline the declaration of classes for "Plain Old Java Objects".

[java_071] Java - Modern OOP - Records

JEP 395: Records: Motivation

Sumber: ht

It is a common complaint that "Java is too verbose" or has "too much ceremony". Some of the worst offenders are classes that are nothing more than immutable data carriers
for a handful of values. Properly writing such a data-carrier class involves a lot of low-value, repetitive, error-prone code: constructors, accessors,
equals
,
hashCode
,
toString
, etc. For example, a class to carry
x and y coordinates inevitably ends up like this:
Developers are sometimes tempted to cut corners by omitting methods such as
equals
, leading to surprising behavior or poor debuggability, or by pressing an alternate but not entirely appropriate class into service because it has the "right shape" and they don't want to declare yet another class.
IDEs help us to
write
most of the code in a data-carrier class, but don't do anything to help the
reader

distill the design intent of "I'm a data carrier
for
x
and
y
" from the dozens of lines of boilerplate. Writing Java code
that models a handful of values should be easier to write, to read, and to
verify as correct.
While it is superficially tempting to treat records as primarily being about
boilerplate reduction, we instead choose a more semantic goal:
modeling data as
data
. (If the semantics are right, the boilerplate will take care of itself.)
It should be easy and concise to declare data-carrier classes that
by default
make their data immutable and provide idiomatic implementations of methods that
produce and consume the data.

Contoh Kode / Perintah CLI:

```
class Point {
    private final int x;
    private final int y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    int x() { return x; }
    int y() { return y; }

    public boolean equals(Object o) {
        if (!(o instanceof Point)) return false;
        Point other = (Point) o;
        return other.x == x && other.y == y;
    }

    public int hashCode() {
        return Objects.hash(x, y);
    }

    public String toString() {
        return String.format("Point[x=%d, y=%d]", x, y);
    }
}
```

[java_072] Java - Modern OOP - Records

JEP 395: Records: Description

Sumber: ht

Record classes
are a new kind of class in the Java language. Record classes
help to model plain data aggregates with less ceremony than normal classes.
The declaration of a record class primarily consists of a declaration of its
state
; the record class then commits to an API that matches that state. This
means that record classes give up a freedom that classes usually enjoy

-- the ability to decouple a class's API from its internal representation -- but, in return, record class declarations become significantly more concise.
More precisely, a record class declaration consists of a name, optional type parameters, a header, and a body. The header lists the components of the record class, which are the variables that make up its state. (This list of components is sometimes referred to as the state description.) For example:
Because record classes make the semantic claim of being transparent carriers for their data, a record class acquires many standard members automatically:
For each component in the header, two members: a public accessor method with the same name and return type as the component, and a private final field with the same type as the component;
A canonical constructor whose signature is the same as the header, and which assigns each private field to the corresponding argument from a new expression which instantiates the record;
equals and hashCode methods which ensure that two record values are equal if they are of the same type and contain equal component values; and
A toString method that returns a string representation of all the record components, along with their names.
In other words, the header of a record class describes its state, i.e., the types and names of its components, and the API is derived mechanically and completely from that state description. The API includes protocols for construction, member access, equality, and display. (We expect a future version to support deconstruction patterns to allow powerful pattern matching.)

Contoh Kode / Perintah CLI:

```
record Point(int x, int y) { }
```

[java_073] Java - Modern OOP - Records

JEP 395: Records: Constructors for record classes

Sumber: ht

The rules for constructors in a record class are different than in a normal class. A normal class without any constructor declarations is automatically given a
default constructor
.
In contrast, a record class without any constructor declarations is automatically given a
canonical constructor
that assigns all the private fields to the
corresponding arguments of the
new
expression which instantiated the record.
For example, the record declared earlier --
record Point(int x, int y) { }
--
is compiled as if it were:
The canonical constructor may be declared explicitly with a list of formal parameters which match the record header, as shown above. It may also be declared more compactly, by eliding the list of formal parameters. In such a
compact canonical constructor
the parameters are declared implicitly, and the
private fields corresponding to record components cannot be assigned in the body but are automatically assigned to the corresponding formal parameter
(
this.x = x;
) at the end of the constructor. The compact form helps
developers focus on validating and normalizing parameters without the tedious work of assigning parameters to fields.
For example, here is a compact canonical constructor that validates its implicit formal parameters:
Here is a compact canonical constructor that normalizes its formal parameters:
This declaration is equivalent to the conventional constructor form:
Record classes with implicitly declared constructors and methods satisfy important, and intuitive, semantic properties. For example, consider a record class
R
declared as follows:
If an instance
r1
of
R
is copied in the following way:
then, assuming
r1
is not the null reference, it is
always
the case that the
expression

r1.equals(r2)
will evaluate to
true
. Explicitly declared accessor
and
equals
methods should respect this invariant. However, it is not generally
possible for a compiler to check that explicitly declared methods respect this
invariant.
As an example, the following declaration of a record class should be considered
bad style because its accessor methods "silently" adjust the state of a record
instance, and the invariant above is not satisfied:
In addition, for all record classes the implicitly declared
equals
method is implemented so that it is
reflexive and that it behaves consistently with
hashCode
for record classes
that have floating point components. Again, explicitly declared
equals
and
hashCode
methods should behave similarly.

Contoh Kode / Perintah CLI:

```
record Point(int x, int y) {
    // Implicitly declared fields
    private final int x;
    private final int y;

    // Other implicit declarations elided ...

    // Implicitly declared canonical constructor
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

// ----------------------------------------

record Range(int lo, int hi) {
    Range {
        if (lo > hi)  // referring here to the implicit constructor parameters
            throw new IllegalArgumentException(String.format("(%d,%d)", lo, hi));
    }
}

// ----------------------------------------

record Rational(int num, int denom) {
    Rational {
        int gcd = gcd(num, denom);
        num /= gcd;
        denom /= gcd;
    }
}
```

```
// ----------------------------------------

record Rational(int num, int denom) {
    Rational(int num, int denom) {
        // Normalization
        int gcd = gcd(num, denom);
        num /= gcd;
        denom /= gcd;
        // Initialization
        this.num = num;
        this.denom = denom;
    }
}

// ----------------------------------------

record R(T1 c1, ..., Tn cn){ }

// ----------------------------------------

R r2 = new R(r1.c1(), r1.c2(), ..., r1.cn());

// ----------------------------------------

record SmallPoint(int x, int y) {
  public int x() { return this.x < 100 ? this.x : 100; }
  public int y() { return this.y < 100 ? this.y : 100; }
}
```

[java_074] Java - Modern OOP - Records

JEP 395: Records: Rules for record classes

Sumber: ht

There are numerous restrictions on the declaration of a record class in comparison to a normal class:
A record class declaration does not have an
extends
clause. The superclass
of a record class is always
java.lang.Record
, similar to how the superclass of
an enum class is always
java.lang.Enum
. Even though a normal class can explicitly
extend its implicit superclass
Object
, a record cannot explicitly extend any
class, even its implicit superclass
Record
.
A record class is implicitly
final
, and cannot be
abstract
. These
restrictions emphasize that the API of a record class is defined solely by its

state description, and cannot be enhanced later by another class.
The fields derived from the record components are
final
. This restriction
embodies an
immutable by default
policy that is widely applicable for
data-carrier classes.
A record class cannot explicitly declare instance fields, and cannot contain instance initializers. These restrictions ensure that the record header alone defines the state of a record value.
Any explicit declarations of a member that would otherwise be automatically derived must match the type of the automatically derived member exactly, disregarding any annotations on the explicit declaration. Any explicit implementation of accessors or the
equals
or
hashCode
methods should be
careful to preserve the semantic invariants of the record class.
A record class cannot declare
native
methods. If a record class could
declare a
native
method then the behavior of the record class would by
definition depend on external state rather than the record class's explicit state. No class with native methods is likely to be a good candidate for migration to a record.
Beyond the restrictions above, a record class behaves like a normal class:
Instances of record classes are created using a
new
expression.
A record class can be declared top level or nested, and can be generic.
A record class can declare
static
methods, fields, and initializers.
A record class can declare instance methods.
A record class can implement interfaces. A record class cannot specify a superclass since that would mean inherited state, beyond the state described in the header. A record class can, however, freely specify superinterfaces and declare instance methods to implement them. Just as for classes, an interface can usefully characterize the behavior of many records. The behavior may be domain-independent (e.g.,
Comparable
) or domain-specific, in which case
records can be part of a
sealed
hierarchy which captures the domain (see
below).
A record class can declare nested types, including nested record classes. If a record class is itself nested, then it is implicitly static; this avoids an

immediately enclosing instance which would silently add state to the record class.
A record class, and the components in its header, may be decorated with annotations. Any annotations on the record components are propagated to the automatically derived fields, methods, and constructor parameters, according to the set of applicable targets for the annotation. Type annotations on the types of record components are also propagated to the corresponding type uses in the automatically derived members.
Instances of record classes can be serialized and deserialized. However, the process cannot be customized by providing
writeObject
,
readObject
,
readObjectNoData
,
writeExternal
, or
readExternal
methods. The components
of a record class govern serialization, while the canonical constructor of a record class governs deserialization.

[java_075] Java - Modern OOP - Records

JEP 395: Records: Local record classes

Sumber: ht

A program that produces and consumes instances of a record class is likely to deal with many intermediate values that are themselves simple groups of variables. It will often be convenient to declare record classes to model those intermediate values. One option is to declare "helper" record classes that are static and nested, much as many programs declare helper classes today. A more convenient option would be to declare a record
inside a method
, close to the
code which manipulates the variables. Accordingly we define
local
record classes
, akin to the existing construct of
local classes
.
In the following example, the aggregation of a merchant and a monthly sales figure is modeled with a local record class,
MerchantSales
. Using this record class
improves the readability of the stream operations which follow:
Local record classes are a particular case of nested record classes. Like nested record classes, local record classes are
implicitly static
. This means that
their own methods cannot access any variables of the enclosing method; in turn, this avoids capturing an immediately enclosing instance which would silently add

state to the record class. The fact that local record classes are implicitly static is in contrast to local classes, which are not implicitly static. In fact, local classes are never static -- implicitly or explicitly -- and can always access variables in the enclosing method.

Contoh Kode / Perintah CLI:

```
List<Merchant> findTopMerchants(List<Merchant> merchants, int month) {
    // Local record
    record MerchantSales(Merchant merchant, double sales) {}

    return merchants.stream()
        .map(merchant -> new MerchantSales(merchant, computeSales(merchant, month)))
        .sorted((m1, m2) -> Double.compare(m2.sales(), m1.sales()))
        .map(MerchantSales::merchant)
        .collect(toList());
}
```

[java_076] Java - Modern OOP - Records

JEP 395: Records: Local enum classes and local interfaces

Sumber: ht

The addition of local record classes is an opportunity to add other kinds of implicitly-static local declarations.
Nested enum classes and nested interfaces are already implicitly static, so for consistency we define local enum classes and local interfaces, which are also implicitly static.

[java_077] Java - Modern OOP - Records

JEP 395: Records: Static members of inner classes

Sumber: ht

It is
currently specified
to be a compile-time error if an inner class declares a member that is explicitly or implicitly static, unless the member is a constant variable. This means that, for example, an inner class cannot declare a record class member, since nested record classes are implicitly static.
We relax this restriction in order to allow an inner class
to declare members that are either explicitly or implicitly static. In particular, this allows an inner class to declare a static member that is a record class.

[java_078] Java - Modern OOP - Records

JEP 395: Records: Annotations on record components

Sumber: ht

Record components have multiple roles in record declarations. A record component is a first-class concept, but each component also corresponds to a field of the same name and type, an accessor method of the same name and return type, and a formal parameter of the canonical constructor of the same name and type.
This raises the question: When a component is annotated, what actually is being annotated? The answer is, "all of the elements to which this particular annotation is applicable." This enables classes that use annotations on their fields, constructor parameters, or accessor methods to be migrated to records

without having to redundantly declare these members. For example, a class such
as the following
can be migrated to the equivalent, and considerably more readable, record declaration:
The applicability of an annotation is declared using a
@Target
meta-annotation.
Consider the following:
This declares the annotation
@I1
that it is applicable to field
declarations. We can declare that an annotation is applicable to more than one
declaration; for example:
This declares an annotation
@I2
that it is applicable to both field
declarations and method declarations.
Returning to annotations on a record component, these annotations appear at the
corresponding program points where they are applicable. In other words, the
propagation is under the control of the developer using the
@Target
meta-annotation. The propagation rules are systematic and intuitive, and all
that apply are followed:
If an annotation on a record component is applicable to a field declaration,
then the annotation appears on the corresponding private field.
If an annotation on a record component is applicable to a method declaration,
then the annotation appears on the corresponding accessor method.
If an annotation on a record component is applicable to a formal parameter,
then the annotation appears on the corresponding formal parameter of the canonical
constructor if one is not declared explicitly, or else to the corresponding formal
parameter of the compact constructor if one is declared explicitly.
If an annotation on a record component is applicable to a type, the annotation
will be propagated to all of the following:
the type of the corresponding field
the return type of the corresponding accessor method
the type of the corresponding formal parameter of the canonical constructor
the type of the record component (which is accessible at runtime via reflection)
If a public accessor method or (non-compact) canonical constructor is declared
explicitly, then it only has the annotations which appear on it directly;
nothing is propagated from the corresponding record component to these members.
A declaration annotation on a record component will not be amongst those associated
with the record component at run time via the
reflection API
unless the annotation is meta-annotated with
@Target(RECORD_COMPONENT)
.

Contoh Kode / Perintah CLI:

```
public final class Card {
    private final @MyAnno Rank rank;
    private final @MyAnno Suit suit;
    @MyAnno Rank rank() { return this.rank; }
    @MyAnno Suit suit() { return this.suit; }
    ...
}
```

```
// ----------------------------------------

public record Card(@MyAnno Rank rank, @MyAnno Suit suit) { ... }

// ----------------------------------------

@Target(ElementType.FIELD)
    public @interface I1 {...}

// ----------------------------------------

@Target({ElementType.FIELD, ElementType.METHOD})
    public @interface I2 {...}
```

[java_079] Java - Modern OOP - Records

JEP 395: Records: Compatibility and migration

Sumber: ht

The abstract class
java.lang.Record
is the common superclass of all record
classes. Every Java source file implicitly imports the
java.lang.Record
class,
as well as all other types in the
java.lang
package, regardless of whether you
enable or disable preview features. However, if your application imports
another class named
Record
from a different package, you might get a compiler
error.
Consider the following class declaration of
com.myapp.Record
:
The following example,
org.example.MyappPackageExample
, imports
com.myapp.Record
with a wildcard but doesn't compile:
The compiler generates an error message similar to the following:
Both
Record
in the
com.myapp
package and
Record
in the
java.lang
package
are imported with wildcards. Consequently, neither class takes precedence, and
the compiler generates an error message when it encounters the use of the simple
name

Record
.
To enable this example to compile, the
import
statement can be changed so that
it imports the fully qualified name of
Record
:
The introduction of classes in the
java.lang
package is rare but sometimes
necessary. Previous examples are
Enum
in Java 5,
Module
in
Java 9, and
Record
in Java 14.

Contoh Kode / Perintah CLI:

```
package com.myapp;

public class Record {
    public String greeting;
    public Record(String greeting) {
        this.greeting = greeting;
    }
}

// ----------------------------------------

package org.example;
import com.myapp.*;

public class MyappPackageExample {
    public static void main(String[] args) {
       Record r = new Record("Hello world!");
    }
}

// ----------------------------------------

./org/example/MyappPackageExample.java:6: error: reference to Record is ambiguous
       Record r = new Record("Hello world!");
       ^
  both class com.myapp.Record in com.myapp and class java.lang.Record in java.lang match

./org/example/MyappPackageExample.java:6: error: reference to Record is ambiguous
       Record r = new Record("Hello world!");
                      ^
  both class com.myapp.Record in com.myapp and class java.lang.Record in java.lang match

// ----------------------------------------

import com.myapp.Record;
```

[java_080] Java - Modern OOP - Records

JEP 395: Records: Java grammar

Sumber: ht

Contoh Kode / Perintah CLI:

```
RecordDeclaration:
  {ClassModifier} `record` TypeIdentifier [TypeParameters]
    RecordHeader [SuperInterfaces] RecordBody

RecordHeader:
 `(` [RecordComponentList] `)`

RecordComponentList:
 RecordComponent { `,` RecordComponent}

RecordComponent:
 {Annotation} UnannType Identifier
 VariableArityRecordComponent

VariableArityRecordComponent:
 {Annotation} UnannType {Annotation} `...` Identifier

RecordBody:
  `{` {RecordBodyDeclaration} `}`

RecordBodyDeclaration:
  ClassBodyDeclaration
  CompactConstructorDeclaration

CompactConstructorDeclaration:
  {ConstructorModifier} SimpleTypeName ConstructorBody
```

[java_081] Java - Modern OOP - Records

JEP 395: Records: Class-file representation

Sumber: ht

The
class
file of a record uses a
Record
attribute to store information
about the record's components:
If the record component has a generic signature that is different from the
erased descriptor then there must be a
Signature
attribute in the
record_component_info
structure.

Contoh Kode / Perintah CLI:

```
Record_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 components_count;
    record_component_info components[components_count];
}

record_component_info {
    u2 name_index;
```

```
    u2 descriptor_index;
    u2 attributes_count;
    attribute_info attributes[attributes_count];
}
```

[java_082] Java - Modern OOP - Records

JEP 395: Records: Reflection API

Sumber: ht

We add two public methods to
java.lang.Class
:
RecordComponent[] getRecordComponents()
 -- Returns an array of
java.lang.reflect.RecordComponent
objects. The elements of this array
correspond to the record's components, in the same order as they appear in the
record declaration. Additional information can be extracted from each element of
the array, including its name, annotations, and accessor method.
boolean isRecord()
 -- Returns true if the given class was declared as a
record. (Compare with
isEnum
.)

[java_083] Java - Modern OOP - Records

JEP 395: Records: Alternatives

Sumber: ht

Record classes can be considered a nominal form of
tuples
. Instead of record
classes, we could implement structural tuples. However, while tuples might offer
a lightweight means of expressing some aggregates, the result is often inferior
aggregates:
A central aspect of Java's design philosophy is that
names matter
. Classes
and their members have meaningful names, while tuples and tuple components do
not. That is, a
Person
record class with components
firstName
and
lastName
is
clearer and safer than an anonymous tuple of two strings.
Classes allow for state validation through their constructors; tuples typically
do not. Some data aggregates (such as numeric ranges) have invariants that, if
enforced by the constructor, can thereafter be relied upon. Tuples do not offer
this ability.
Classes can have behavior that is based on their state; co-locating the state
and behavior makes the behavior more discoverable and easier to access. Tuples,

being raw data, offer no such facility.

[java_084] Java - Modern OOP - Records

JEP 395: Records: Dependencies

Sumber: ht

Record classes work well with another feature currently in preview, namely
sealed
classes
(
JEP 360
). For example, a family
of record classes can be explicitly declared to implement the same sealed
interface:
The combination of record classes and sealed classes is sometimes referred to as
algebraic data types
.
Record classes allow us to express
products
, and sealed classes allow us to
express
sums
.
In addition to the combination of record classes and sealed classes, record
classes lend themselves naturally to
pattern matching
. Because
record classes couple their API to their state description, we will eventually
be able to derive deconstruction patterns for record classes as well, and use
sealed type information to determine exhaustiveness in
switch
expressions with
type patterns or deconstruction patterns.
Installing
Contributing
Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds
Mailing lists
Wiki
*
IRC
Mastodon
Bluesky
Bylaws
*
Census
Legal
*
AI
Workshop

JEP Process
Source code
GitHub
Mercurial
Tools
Git
jtreg harness
Groups
(overview
,
archive
)
Adoption
Build
Client Libraries
Compatibility & Specification Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
Members
Networking
Porters
Quality
Security
Serviceability
Vulnerability
Web
Projects
(
overview
,
archive
)
Amber
Babylon
Brisbane
CRaC
Code Tools
Coin
Common VM Interface
Detroit
Developers' Guide
Duke
IcedTea
JDK 8 Updates
JDK 9
JDK
(...,

26
,
27
,
28
)
JDK Updates
JMC
Jigsaw
Lanai
Leyden
Lilliput
Loom
Memory Model Update
Multi-Language VM
Nashorn
New I/O
OpenJFX
Panama
Port: AArch32
Port: AArch64
Port: BSD
Port: Haiku
Port: MIPS
Port: Mobile
Port: PowerPC/AIX
Port: RISC-V
Port: s390x
SCTP
Shenandoah
Skara
Sumatra
Tsan
Valhalla
Wakefield
Zero
ZGC
(c) 2026 Oracle Corporation and/or its affiliates
Terms of Use
*

License:
GPLv2
*
Privacy
*
Trademarks

Contoh Kode / Perintah CLI:

```
package com.example.expression;

public sealed interface Expr
    permits ConstantExpr, PlusExpr, TimesExpr, NegExpr {...}
```

```
public record ConstantExpr(int i)       implements Expr {...}
public record PlusExpr(Expr a, Expr b)  implements Expr {...}
public record TimesExpr(Expr a, Expr b) implements Expr {...}
public record NegExpr(Expr e)           implements Expr {...}
```

[java_085] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Overview & Metadata

Sumber: ht

<noscript></noscript>
TABLE { border-collapse: collapse; padding: 0px; margin: 1em 0 1em 2em; }
TR:first-child TH, TR:first-child TD { padding-top: 0; }
TH, TD { padding: 0px; padding-top: .5ex; vertical-align: baseline; text-align: left; }
TD + TD, TH + TH { padding-left: 1em; }
TD:first-child, TH:first-child, TD.jep { text-align: right; }
TABLE.head TD:first-child { font-style: italic; padding-left: 2em; white-space: nowrap; }
PRE { padding-left: 2em; margin: 1ex 0; font-size: inherit; }
TABLE PRE { padding-left: 0; margin: 0; }
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { padding-left: .5em; }
TABLE.jeps TD:first-child,
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { font-size: smaller; }
TABLE.jeps TD.cl { font-size: smaller; padding-right: 0; text-align: right; }
TABLE.jeps TD.cm { font-size: smaller; padding-left: .1em; padding-right: .1em; }
TABLE.jeps TD.cr { font-size: smaller; padding-left: 0; }
TABLE.jeps TD.z { padding-left: 0; padding-right: 0; }
TABLE.head TD { padding-top: 0; }
JEP 409: Sealed Classes
Owner
Gavin Bierman
Type
Feature
Scope
SE
Status
Closed / Delivered
Release
17
Component
specification / language
Discussion
amber dash dev at openjdk dot java dot net
Relates to
JEP 360: Sealed Classes (Preview)
JEP 397: Sealed Classes (Second Preview)
Reviewed by
Alex Buckley
Endorsed by
Brian Goetz
Created
2021/01/27 13:40

Updated
2024/01/03 01:48
Issue
8260514

[java_086] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Summary

Sumber: ht

Enhance the Java programming language with
sealed classes and interfaces
.
Sealed classes and interfaces restrict which other classes or interfaces may extend or implement them.

[java_087] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: History

Sumber: ht

Sealed Classes were proposed by
JEP 360
and
delivered in
JDK 15
as a
preview feature
. They were proposed again, with refinements,
by
JEP 397
and delivered in
JDK 16
as a preview feature.
This JEP proposes to finalize Sealed Classes in JDK 17, with no changes from JDK 16.

[java_088] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Goals

Sumber: ht

Allow the author of a class or interface to control which code is responsible
for implementing it.
Provide a more declarative way than access modifiers to restrict the use of a
superclass.
Support future directions in
pattern matching
by
providing a foundation for the exhaustive analysis of patterns.

[java_089] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Non-Goals

Sumber: ht

It is not a goal to provide new forms of access control such as "friends".
It is not a goal to change
final
in any way.

[java_090] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Motivation

Sumber: ht

The object-oriented data model of inheritance hierarchies of classes and interfaces has proven to be highly effective in modeling the real-world data processed by modern applications. This expressiveness is an important aspect of the Java language.
There are, however, cases where such expressiveness can usefully be tamed. For example, Java supports
enum classes
to model the situation where a given class has only a fixed number of instances. In the following code, an enum class lists a fixed set of planets. They are the only values of the class, therefore you can switch over them exhaustively -- without having to write a
default
clause:
Using enum classes to model fixed sets of values is often helpful, but sometimes we want to model a fixed set of
kinds
of values. We can do this by using a class hierarchy not as a mechanism for code inheritance and reuse but, rather, as a way to list kinds of values. Building on our planetary example, we might model the kinds of values in the astronomical domain as follows:
This hierarchy does not, however, reflect the important domain knowledge that there are only three kinds of celestial objects in our model. In these situations, restricting the set of subclasses or subinterfaces can streamline the modeling.
Consider another example: In a graphics library, the author of a class
Shape
may intend that only particular classes can extend
Shape
, since much of the library's work involves handling each kind of shape in the appropriate way. The author is interested in the clarity of code that handles known subclasses of
Shape
, and not interested in writing code to defend against unknown subclasses of
Shape
. Allowing arbitrary classes to extend
Shape
, and thus inherit its code for reuse, is not a goal in this case. Unfortunately, Java assumes that code reuse is always a goal: If

Shape
can be extended at all,
then it can be extended by any number of classes. It would be helpful to relax this assumption so that an author can declare a class hierarchy that is not open for extension by arbitrary classes. Code reuse would still be possible within such a closed class hierarchy, but not beyond.
Java developers are familiar with the idea of restricting the set of subclasses because it often crops up in API design. The language provides limited tools in this area: Either make a class
final
, so it has zero subclasses, or make the
class or its constructor package-private, so it can only have subclasses in the same package. An example of a package-private superclass
appears in the JDK
:
The package-private approach is useful when the goal is code reuse, such as having the subclasses of
AbstractStringBuilder
share its code for
append
.
However, the approach is useless when the goal is modeling alternatives, since user code cannot access the key abstraction -- the superclass -- in order to
switch
over it. Allowing users to access the superclass without also allowing them to extend it cannot be specified without resorting to brittle tricks involving non-
public
constructors -- which do not work for interfaces. In a
graphics library that declares
Shape
and its subclasses, it would be
unfortunate if only one package could access
Shape
.
In summary, it should be possible for a superclass to be widely
accessible
(since it represents an important abstraction for users) but not widely
extensible
(since its subclasses should be restricted to those known to the author). The author of such a superclass should be able to express that it is co-developed with a given set of subclasses, both to document intent for the reader and to allow enforcement by the Java compiler. At the same time, the superclass should not unduly constrain its subclasses by, e.g., forcing them to
be
final
or preventing them from defining their own state.

Contoh Kode / Perintah CLI:

```
enum Planet { MERCURY, VENUS, EARTH }

Planet p = ...
switch (p) {
```

```
  case MERCURY: ...
  case VENUS: ...
  case EARTH: ...
}

// ----------------------------------------

interface Celestial { ... }
final class Planet implements Celestial { ... }
final class Star   implements Celestial { ... }
final class Comet  implements Celestial { ... }

// ----------------------------------------

package java.lang;

abstract class AbstractStringBuilder { ... }
public final class StringBuffer  extends AbstractStringBuilder { ... }
public final class StringBuilder extends AbstractStringBuilder { ... }
```

[java_091] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Description

Sumber: ht

A
sealed
class or interface can be extended or implemented only by those
classes and interfaces permitted to do so.
A class is sealed by applying the
sealed
modifier to its declaration. Then,
after any
extends
and
implements
clauses, the
permits
clause specifies the
classes that are permitted to extend the sealed class. For example, the
following declaration of
Shape
specifies three permitted subclasses:
The classes specified by
permits
must be located near the superclass: either
in the same module (if the superclass is in a named module) or in the same
package (if the superclass is in the unnamed module). For example, in the
following declaration of
Shape
its permitted subclasses are all located in
different packages of the same named module:
When the permitted subclasses are small in size and number, it may be convenient
to declare them in the same source file as the sealed class. When they are
declared in this way, the
sealed

class may omit the
permits
clause and the
Java compiler will infer the permitted subclasses from the declarations in the source file. (The subclasses may be auxiliary or nested classes.) For example, if the following code is found in
Root.java
then the sealed class
Root
is inferred to have three permitted subclasses:
Classes specified by
permits
must have a canonical name, otherwise a
compile-time error is reported. This means that anonymous classes and local classes cannot be permitted subtypes of a sealed class.
A sealed class imposes three constraints on its permitted subclasses:
The sealed class and its permitted subclasses must belong to the same module, and, if declared in an unnamed module, to the same package.
Every permitted subclass must directly extend the sealed class.
Every permitted subclass must use a modifier to describe how it propagates the sealing initiated by its superclass:
A permitted subclass may be declared
final
to prevent its part of the
class hierarchy from being extended further.
(
Record classes
are implicitly declared
final
.)
A permitted subclass may be declared
sealed
to allow its part of the
hierarchy to be extended further than envisaged by its sealed superclass, but in a restricted fashion.
A permitted subclass may be declared
non-sealed
so that its part of the
hierarchy reverts to being open for extension by unknown subclasses. A sealed class cannot prevent its permitted subclasses from doing this.
(The modifier
non-sealed
is the first
hyphenated keyword
proposed for Java.)
As an example of the third constraint,
Circle
and
Square
may be
final
while

Rectangle
is
sealed
and we add a new subclass,
WeirdShape
, that is
non-sealed
:
Even though the
WeirdShape
is open to extension by unknown classes, all
instances of those subclasses are also instances of
WeirdShape
. Therefore
code written to test whether an instance of
Shape
is either a
Circle
, a
Rectangle
, a
Square
, or a
WeirdShape
remains exhaustive.
Exactly one of the modifiers
final
,
sealed
, and
non-sealed
must be used by
each permitted subclass. It is not possible for a class to be both
sealed
(implying subclasses) and
final
(implying no subclasses), or both
non-sealed
(implying subclasses) and
final
(implying no subclasses), or both
sealed
(implying restricted subclasses) and
non-sealed
(implying
unrestricted subclasses).
(The
final
modifier can be considered a special case of sealing, where
extension/implementation is prohibited completely. That is,
final
is

conceptually equivalent to
sealed
plus a
permits
clause which specifies
nothing, though such a
permits
clause cannot be written.)
A class which is
sealed
or
non-sealed
may be
abstract
, and have
abstract
members. A
sealed
class may permit subclasses which are
abstract
, providing
they are then
sealed
or
non-sealed
, rather than
final
.
It is a compile-time error if any class extends a
sealed
class but is not
permitted to do so.

Contoh Kode / Perintah CLI:

```
package com.example.geometry;

public abstract sealed class Shape
    permits Circle, Rectangle, Square { ... }

// ----------------------------------------

package com.example.geometry;

public abstract sealed class Shape
    permits com.example.polar.Circle,
            com.example.quad.Rectangle,
            com.example.quad.simple.Square { ... }

// ----------------------------------------

abstract sealed class Root { ...
    final class A extends Root { ... }
    final class B extends Root { ... }
    final class C extends Root { ... }
}
```

```
// ----------------------------------------

package com.example.geometry;

public abstract sealed class Shape
    permits Circle, Rectangle, Square, WeirdShape { ... }

public final class Circle extends Shape { ... }

public sealed class Rectangle extends Shape
    permits TransparentRectangle, FilledRectangle { ... }
public final class TransparentRectangle extends Rectangle { ... }
public final class FilledRectangle extends Rectangle { ... }

public final class Square extends Shape { ... }

public non-sealed class WeirdShape extends Shape { ... }
```

[java_092] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Class accessibility

Sumber: ht

Because
extends
and
permits
clauses make use of class names, a permitted
subclass and its sealed superclass must be accessible to each other. However, permitted subclasses need not have the same accessibility as each other, or as the sealed class. In particular, a subclass may be less accessible than the sealed class. This means that, in a future release when pattern matching is supported by switches, some code will not be able to exhaustively
switch
over
the subclasses unless a
default
clause (or other total pattern) is used. Java compilers will be encouraged to detect when
switch
is not as exhaustive as its original author imagined it would be, and customize the error message to recommend a
default
clause.

[java_093] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Sealed interfaces

Sumber: ht

As for classes, an interface can be sealed by applying the
sealed
modifier to
the interface. After any
extends
clause to specify superinterfaces, the

implementing classes and subinterfaces are specified with a
permits
clause.
For example, the planetary example from above can be rewritten as follows:
Here is another classic example of a class hierarchy where there is a known set of subclasses: modeling mathematical expressions.

Contoh Kode / Perintah CLI:

```
sealed interface Celestial
    permits Planet, Star, Comet { ... }

final class Planet implements Celestial { ... }
final class Star   implements Celestial { ... }
final class Comet  implements Celestial { ... }

// --------------------------------------

package com.example.expression;

public sealed interface Expr
    permits ConstantExpr, PlusExpr, TimesExpr, NegExpr { ... }

public final class ConstantExpr implements Expr { ... }
public final class PlusExpr     implements Expr { ... }
public final class TimesExpr    implements Expr { ... }
public final class NegExpr      implements Expr { ... }
```

[java_094] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Sealing and record classes

Sumber: ht

Sealed classes work well with
record classes
. Record classes are
implicitly
final
, so a sealed hierarchy of record classes is slightly more concise than the example above:
The combination of sealed classes and record classes is sometimes referred to as
algebraic data types
:
Record classes allow us to express
product types
, and sealed classes allow us to express
sum types
.

Contoh Kode / Perintah CLI:

```
package com.example.expression;

public sealed interface Expr
    permits ConstantExpr, PlusExpr, TimesExpr, NegExpr { ... }

public record ConstantExpr(int i)       implements Expr { ... }
public record PlusExpr(Expr a, Expr b)  implements Expr { ... }
public record TimesExpr(Expr a, Expr b) implements Expr { ... }
```

```
public record NegExpr(Expr e)           implements Expr { ... }
```

[java_095] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Sealed classes and conversions

Sumber: ht

A cast expression converts a value to a type. A type
instanceof
expression
tests a value against a type. Java is extremely permissive about the types that are allowed in these kinds of expressions. For example:
This program is legal even though it is currently not possible for a
C
object to
implement the interface
I
. Of course, as the program evolves, it might be:
The type conversion rules capture a notion of
open extensibility
. The Java
type system does not assume a closed world. Classes and interfaces can be extended at some future time, and casting conversions compile to runtime tests, so we can safely be flexible.
However, at the other end of the spectrum the conversion rules do address the case where a class can definitely not be extended, i.e., when it is a
final
class.
The method
test
fails to compile, since the compiler knows that there can be
no subclass of
C
, so since
C
does not implement
I
then it is never
possible for a
C
value to implement
I
. This is a compile-time error.
What if
C
is not
final
, but
sealed
? Its direct subclasses
are explicitly enumerated, and -- by the definition of being
sealed
 -- in the same

module, so we expect the compiler to look to see if it can spot a similar compile-time error. Consider the following code:

Class C does not implement I, and is not final, so by the existing rules we might conclude that a conversion is possible. C is sealed, however, and there is one permitted direct subclass of C, namely D. By the definition of sealed types, D must be either final, sealed, or non-sealed. In this example, all the direct subclasses of C are final and do not implement I. This program should therefore be rejected, since there cannot be a subtype of C that implements I.

In contrast, consider a similar program where one of the direct subclasses of the sealed class is non-sealed:

This is type-correct, since it is possible for a subtype of the non-sealed type D to implement I.

Consequently, supporting

sealed
classes leads to a change in the definition of
narrowing reference conversion
to navigate sealed hierarchies to determine at compile time which
conversions are not possible.

Contoh Kode / Perintah CLI:

```
interface I {}
class C {} // does not implement I

void test (C c) {
    if (c instanceof I)
        System.out.println("It's an I");
}

// ----------------------------------------

...
class B extends C implements I {}

test(new B());
// Prints "It's an I"

// ----------------------------------------

interface I {}
final class C {}

void test (C c) {
    if (c instanceof I)     // Compile-time error!
        System.out.println("It's an I");
}

// ----------------------------------------

interface I {}
sealed class C permits D {}
final class D extends C {}

void test (C c) {
    if (c instanceof I)     // Compile-time error!
        System.out.println("It's an I");
}

// ----------------------------------------

interface I {}
sealed class C permits D, E {}
non-sealed class D extends C {}
final class E extends C {}

void test (C c) {
    if (c instanceof I)
        System.out.println("It's an I");
}
```

[java_096] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Sealed classes in the JDK

Sumber: ht

An example of how sealed classes might be used in the JDK is in the
java.lang.constant
package that models
descriptors for JVM entities
:

Contoh Kode / Perintah CLI:

```
package java.lang.constant;

public sealed interface ConstantDesc
    permits String, Integer, Float, Long, Double,
            ClassDesc, MethodTypeDesc, DynamicConstantDesc { ... }

// ClassDesc is designed for subclassing by JDK classes only
public sealed interface ClassDesc extends ConstantDesc
    permits PrimitiveClassDescImpl, ReferenceClassDescImpl { ... }
final class PrimitiveClassDescImpl implements ClassDesc { ... }
final class ReferenceClassDescImpl implements ClassDesc { ... }

// MethodTypeDesc is designed for subclassing by JDK classes only
public sealed interface MethodTypeDesc extends ConstantDesc
    permits MethodTypeDescImpl { ... }
final class MethodTypeDescImpl implements MethodTypeDesc { ... }

// DynamicConstantDesc is designed for subclassing by user code
public non-sealed abstract class DynamicConstantDesc implements ConstantDesc { ... }
```

[java_097] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Sealed classes and pattern matching

Sumber: ht

A significant benefit of sealed classes will be realized in
JEP 406
,
which proposes to extend
switch
with pattern matching. Instead of inspecting
an instance of a sealed class with
if
-
else
chains, user code will be able to
use a
switch
enhanced with patterns. The use of sealed classes will allow
the Java compiler to check that the patterns are exhaustive.
For example, consider this code using the
sealed
hierarchy declared earlier:
The Java compiler cannot ensure that the
instanceof
tests cover all the

permitted subclasses of
Shape
. The final
else
clause is actually
unreachable, but this cannot be verified by the compiler. More importantly, no
compile-time error message would be issued if the
instanceof Rectangle
test
was omitted.
In contrast, with pattern matching for
switch
(
JEP 406
)the compiler can confirm that
every permitted subclass of
Shape
is covered, so no
default
clause or other
total pattern is needed. The compiler will, moreover, issue an error message if
any of the three cases is missing:

Contoh Kode / Perintah CLI:

```
Shape rotate(Shape shape, double angle) {
        if (shape instanceof Circle) return shape;
        else if (shape instanceof Rectangle) return shape;
        else if (shape instanceof Square) return shape;
        else throw new IncompatibleClassChangeError();
}

// ----------------------------------------

Shape rotate(Shape shape, double angle) {
    return switch (shape) {   // pattern matching switch
        case Circle c    -> c;
        case Rectangle r -> shape.rotate(angle);
        case Square s    -> shape.rotate(angle);
        // no default needed!
    }
}
```

[java_098] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Java grammar

Sumber: ht

The grammar for class declarations is amended to the following:

Contoh Kode / Perintah CLI:

```
NormalClassDeclaration:
  {ClassModifier} class TypeIdentifier [TypeParameters]
    [Superclass] [Superinterfaces] [PermittedSubclasses] ClassBody

ClassModifier:
  (one of)
  Annotation public protected private
  abstract static sealed final non-sealed strictfp
```

```
PermittedSubclasses:
  permits ClassTypeList

ClassTypeList:
  ClassType {, ClassType}
```

[java_099] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: JVM support for sealed classes

Sumber: ht

The Java Virtual Machine recognizes
sealed
classes and interfaces at runtime,
and prevents extension by unauthorized subclasses and subinterfaces.
Although
sealed
is a class modifier, there is no
ACC_SEALED
flag in the
ClassFile
structure. Instead, the
class
file of a sealed class has a
PermittedSubclasses
attribute which implicitly indicates the
sealed
modifier
and explicitly specifies the permitted subclasses:
The list of permitted subclasses is mandatory. Even when the permitted
subclasses are inferred by the compiler, those inferred subclasses are
explicitly included in the
PermittedSubclasses
attribute.
The
class
file of a permitted subclass carries no new attributes.
When the JVM attempts to define a class whose superclass or superinterface has a
PermittedSubclasses
attribute, the class being defined must be named by the
attribute. Otherwise, an
IncompatibleClassChangeError
is thrown.

Contoh Kode / Perintah CLI:

```
PermittedSubclasses_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 number_of_classes;
    u2 classes[number_of_classes];
}
```

[java_100] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Reflection API

Sumber: ht

We add the following
public
methods to
java.lang.Class
:
Class<?>[] getPermittedSubclasses()
boolean isSealed()
The method
getPermittedSubclasses()
returns an array containing
java.lang.Class
objects representing the permitted subclasses of the class, if
the class is sealed. It returns an empty array if the class is not sealed.
The method
isSealed
returns true if the given class or interface is sealed.
(Compare with
isEnum
.)

[java_101] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Future Work

Sumber: ht

A common pattern, especially when writing APIs, is to define a public type as an interface and implement it with a single private class. With sealed classes this can be expressed more precisely, as a sealed public interface with a single permitted private implementation. Thus the type is widely accessible but the implementation is not, and cannot be extended in any way.
A clumsiness of this approach is that implementation methods that accept
Foo
objects require explicit casts, for example:
The cast here seems unnecessary, since we know it should always succeed. Yet there is an implicit semantic assumption in the cast, which is that the class
MyFooImpl
is the only implementation of
Foo
. There is no way for the author
to capture this intuition so that it can be checked at compile time. Should, in time,
Foo
permit an additional implementation, this cast would remain
type-correct but may fail at runtime. In other words, the semantic assumption would be broken but the compiler cannot alert the developer of that fact.
With the precision of sealed hierarchies it may be worth providing developers with the means to express such semantic assumptions, and for the compiler to check them. This could be achieved by adding a new form of reference conversion for assignment contexts which allows the conversion of a sealed supertype to its only subtype, for example:

Alternatively, we could provide a new form of cast, for example:
In both cases, should the interface
Foo
be changed to permit another
implementation, then both would cause compile-time errors upon recompilation.

Contoh Kode / Perintah CLI:

```
public sealed interface Foo permits MyFooImpl { }
private final class MyFooImpl implements Foo { }

// ----------------------------------------

void m(Foo f) {
    MyFooImpl mfi = (MyFooImpl) f;
    ...
}

// ----------------------------------------

MyFooImpl mfi = f; // Allowed because the compiler sees that MyFooImpl
                   // is the only permitted subtype of Foo.
                   // (A synthetic cast would be added for safety.)

// ----------------------------------------

MyFooImpl mfi = (total MyFooImpl) f;
```

[java_102] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Alternatives

Sumber: ht

Some languages have direct support for
algebraic data types
(ADTs), such as Haskell's
data
feature. It would be possible to express ADTs
more directly and in a manner familiar to Java developers through a variant of
the
enum
feature, where a sum of products could be defined in a single
declaration. However, this would not support all the desired use cases, such as
those where sums range over classes in more than one compilation unit, or sums
that range over classes that are not products.
The
permits
clause allows a sealed class, such as the
Shape
class shown
earlier, to be accessible-for-invocation by code in any module, but
accessible-for-implementation by code in only the same module as the sealed
class (or same package if in the unnamed module). This makes the type system
more expressive than the access-control system. With access control alone, if
Shape
is accessible-for-invocation by code in any module (because its package
is exported), then
Shape

is also accessible-for-implementation in any module;
and if
Shape
is not accessible-for-implementation in any other module, then
Shape
is also not accessible-for-invocation in any other module.

[java_103] Java - Modern OOP - Sealed Classes

JEP 409: Sealed Classes: Dependencies

Sumber: ht

Sealed classes do not depend on any other JEPs. As mentioned earlier,
JEP 406
proposes to extend
switch
with pattern matching, and builds on sealed classes to
improve the exhaustiveness checking of
switch
.
Installing
Contributing
Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds
Mailing lists
Wiki
*
IRC
Mastodon
Bluesky
Bylaws
*
Census
Legal
*
AI
Workshop
JEP Process
Source code
GitHub
Mercurial
Tools
Git
jtreg harness
Groups
(overview
,
archive
)
Adoption
Build

Client Libraries
Compatibility & Specification Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
Members
Networking
Porters
Quality
Security
Serviceability
Vulnerability
Web
Projects
(
overview
,
archive
)
Amber
Babylon
Brisbane
CRaC
Code Tools
Coin
Common VM Interface
Detroit
Developers' Guide
Duke
IcedTea
JDK 8 Updates
JDK 9
JDK
(...,
26
,
27
,
28
)
JDK Updates
JMC
Jigsaw
Lanai
Leyden
Lilliput
Loom
Memory Model Update

Multi-Language VM
Nashorn
New I/O
OpenJFX
Panama
Port: AArch32
Port: AArch64
Port: BSD
Port: Haiku
Port: MIPS
Port: Mobile
Port: PowerPC/AIX
Port: RISC-V
Port: s390x
SCTP
Shenandoah
Skara
Sumatra
Tsan
Valhalla
Wakefield
Zero
ZGC
(c) 2026 Oracle Corporation and/or its affiliates
Terms of Use
*

License:
GPLv2
*
Privacy
*
Trademarks

[java_104] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Overview & Metadata

Sumber: ht

<noscript></noscript>
TABLE { border-collapse: collapse; padding: 0px; margin: 1em 0 1em 2em; }
TR:first-child TH, TR:first-child TD { padding-top: 0; }
TH, TD { padding: 0px; padding-top: .5ex; vertical-align: baseline; text-align: left; }
TD + TD, TH + TH { padding-left: 1em; }
TD:first-child, TH:first-child, TD.jep { text-align: right; }
TABLE.head TD:first-child { font-style: italic; padding-left: 2em; white-space: nowrap; }
PRE { padding-left: 2em; margin: 1ex 0; font-size: inherit; }
TABLE PRE { padding-left: 0; margin: 0; }
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { padding-left: .5em; }
TABLE.jeps TD:first-child,
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { font-size: smaller; }

```
    TABLE.jeps TD.cl { font-size: smaller; padding-right: 0; text-align: right; }
    TABLE.jeps TD.cm { font-size: smaller; padding-left: .1em; padding-right: .1em; }
    TABLE.jeps TD.cr { font-size: smaller; padding-left: 0; }
    TABLE.jeps TD.z { padding-left: 0; padding-right: 0; }
    TABLE.head TD { padding-top: 0; }
```

JEP 441: Pattern Matching for switch
Owner
Gavin Bierman
Type
Feature
Scope
SE
Status
Closed / Delivered
Release
21
Component
specification / language
Discussion
amber dash dev at openjdk dot org
Relates to
JEP 433: Pattern Matching for switch (Fourth Preview)
Reviewed by
Brian Goetz
Endorsed by
Brian Goetz
Created
2023/01/18 14:43
Updated
2023/09/19 13:38
Issue
8300542

[java_105] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Summary

Sumber: ht

Enhance the Java programming language with pattern matching for
switch
expressions and statements. Extending pattern matching to
switch
allows an
expression to be tested against a number of patterns, each with a specific action, so that complex data-oriented queries can be expressed concisely and safely.

[java_106] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: History

Sumber: ht

This feature was originally proposed by
JEP 406
(JDK 17) and subsequently
refined by JEPs
420
(JDK 18),
427
(JDK 19), and
433
(JDK 20). It has co-evolved with the
Record Patterns
feature (
JEP 440
),
with which it has considerable interaction. This JEP proposes to finalize the feature with further small refinements based upon continued experience and feedback.
Apart from various editorial changes, the main changes from the previous JEP are to:
Remove parenthesized patterns, since they did not have sufficient value, and
Allow qualified enum constants as
case
constants in
switch
expressions and
statements.

[java_107] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Goals

Sumber: ht

Expand the expressiveness and applicability of
switch
expressions and
statements by allowing patterns to appear in
case
labels.
Allow the historical null-hostility of
switch
to be relaxed when desired.
Increase the safety of
switch
statements by requiring that pattern
switch
statements cover all possible input values.
Ensure that all existing
switch
expressions and statements continue to
compile with no changes and execute with identical semantics.

[java_108] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Motivation

Sumber: ht

In Java 16,
JEP 394
extended the
instanceof
operator to take a
type pattern
and perform
pattern matching
. This modest extension allows the
familiar instanceof-and-cast idiom to be simplified, making it both more concise
and less error-prone:
In the new code,
obj
matches the type pattern
String s
if, at run time, the
value of
obj
is an instance of
String
. If the pattern matches then the
instanceof
expression is
true
and the pattern variable
s
is initialized to
the value of
obj
cast to
String
, which can then be used in the contained
block.
We often want to compare a variable such as
obj
against multiple alternatives.
Java supports multi-way comparisons with
switch
statements and, since
Java 14,
switch
expressions (
JEP 361
), but unfortunately
switch
is very limited. We can only switch on values of a few types --
integral primitive types (excluding
long

), their corresponding boxed forms,
enum
types, and
String
-- and we can only test for exact equality against
constants. We might like to use patterns to test the same variable against a
number of possibilities, taking a specific action on each, but since the
existing
switch
does not support that we end up with a chain of
if...else
tests such as:
This code benefits from using pattern
instanceof
expressions, but it is far
from perfect. First and foremost, this approach allows coding errors to remain
hidden because we have used an overly general control construct. The intent is
to assign something to
formatted
in each arm of the
if...else
chain, but
there is nothing that enables the compiler to identify and enforce this
invariant. If some "then" block -- perhaps one that is executed rarely -- does not
assign to
formatted
, we have a bug. (Declaring
formatted
as a blank local
would at least enlist the compiler's definite-assignment analysis in this
effort, but developers do not always write such declarations.) In addition, the
above code is not optimizable; absent compiler heroics it will have
O
(
n
)
time complexity, even though the underlying problem is often
O
(1).
But
switch
is a perfect match for pattern matching! If we extend
switch
statements and expressions to work on any type, and allow
case
labels with
patterns rather than just constants, then we can rewrite the above code more
clearly and reliably:
The semantics of this
switch
are clear: A
case

label with a pattern applies
if the value of the selector expression
obj
matches the pattern. (We have shown
a
switch
expression for brevity but could instead have shown a
switch
statement; the switch block, including the
case
labels, would be unchanged.)
The intent of this code is clearer because we are using the right control construct: We are saying, "the parameter
obj
matches at most one of the
following conditions, figure it out and evaluate the corresponding arm." As a bonus, it is more optimizable; in this case we are more likely to be able to perform the dispatch in
O
(1) time.

Contoh Kode / Perintah CLI:

```
// Prior to Java 16
if (obj instanceof String) {
    String s = (String)obj;
    ... use s ...
}

// As of Java 16
if (obj instanceof String s) {
    ... use s ...
}

// ----------------------------------------

// Prior to Java 21
static String formatter(Object obj) {
    String formatted = "unknown";
    if (obj instanceof Integer i) {
        formatted = String.format("int %d", i);
    } else if (obj instanceof Long l) {
        formatted = String.format("long %d", l);
    } else if (obj instanceof Double d) {
        formatted = String.format("double %f", d);
    } else if (obj instanceof String s) {
        formatted = String.format("String %s", s);
    }
    return formatted;
}

// ----------------------------------------

// As of Java 21
static String formatterPatternSwitch(Object obj) {
    return switch (obj) {
        case Integer i -> String.format("int %d", i);
        case Long l    -> String.format("long %d", l);
        case Double d  -> String.format("double %f", d);
```

```
        case String s  -> String.format("String %s", s);
        default        -> obj.toString();
    };
}
```

[java_109] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Switches and null

Sumber: ht

Traditionally,
switch
statements and expressions throw
NullPointerException
if the selector expression evaluates to
null
, so testing for
null
must be
done outside of the
switch
:
This was reasonable when
switch
supported only a few reference types. However,
if
switch
allows a selector expression of any reference type, and
case
labels can have type patterns, then the standalone
null
test feels like an
arbitrary distinction which invites needless boilerplate and opportunities for
error. It would be better to integrate the
null
test into the
switch
by
allowing a new
null
case label:
The behavior of the
switch
when the value of the selector expression is
null
is always determined by its
case
labels. With a
case null
, the
switch
executes the code associated with that label; without a
case null
, the

switch
throws
NullPointerException
, just as before. (To maintain backward
compatibility with the current semantics of
switch
, the
default
label does
not match a
null
selector.)

Contoh Kode / Perintah CLI:

```
// Prior to Java 21
static void testFooBarOld(String s) {
    if (s == null) {
        System.out.println("Oops!");
        return;
    }
    switch (s) {
        case "Foo", "Bar" -> System.out.println("Great");
        default           -> System.out.println("Ok");
    }
}

// ----------------------------------------

// As of Java 21
static void testFooBarNew(String s) {
    switch (s) {
        case null         -> System.out.println("Oops");
        case "Foo", "Bar" -> System.out.println("Great");
        default           -> System.out.println("Ok");
    }
}
```

[java_110] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Case refinement

Sumber: ht

In contrast to
case
labels with constants, a pattern
case
label can apply to
many values. This can often lead to conditional code on the right-hand side of a
switch rule. For example, consider the following code:
The problem here is that using a single pattern to discriminate among cases does
not scale beyond a single condition. We would prefer to write multiple patterns
but we then need some way to express a refinement to a pattern. We therefore
allow
when
clauses in switch blocks to specify guards to pattern
case
labels, e.g.,

case String s when s.equalsIgnoreCase("YES")
. We refer to such a
case
label as a
guarded
case
label, and to the boolean expression as the
guard
.
With this approach, we can rewrite the above code using guards:
This leads to a more readable style of
switch
programming where the complexity
of the test appears on the left of a switch rule, and the logic that applies if
that test is satisfied is on the right of a switch rule.
We can further enhance this example with extra rules for other known constant
strings:
These examples shows how
case
constants,
case
patterns, and the
null
label
combine to showcase the new power of
switch
programming: We can simplify
complicated conditional logic that was formerly mixed with business logic into a
readable, sequential list of switch labels with the business logic to the right
of the switch rules.

Contoh Kode / Perintah CLI:

```
// As of Java 21
static void testStringOld(String response) {
    switch (response) {
        case null -> { }
        case String s -> {
            if (s.equalsIgnoreCase("YES"))
                System.out.println("You got it");
            else if (s.equalsIgnoreCase("NO"))
                System.out.println("Shame");
            else
                System.out.println("Sorry?");
        }
    }
}

// ----------------------------------------

// As of Java 21
static void testStringNew(String response) {
    switch (response) {
        case null -> { }
        case String s
        when s.equalsIgnoreCase("YES") -> {
            System.out.println("You got it");
        }
```

```
        case String s
        when s.equalsIgnoreCase("NO") -> {
            System.out.println("Shame");
        }
        case String s -> {
            System.out.println("Sorry?");
        }
    }
}

// ----------------------------------------

// As of Java 21
static void testStringEnhanced(String response) {
    switch (response) {
        case null -> { }
        case "y", "Y" -> {
            System.out.println("You got it");
        }
        case "n", "N" -> {
            System.out.println("Shame");
        }
        case String s
        when s.equalsIgnoreCase("YES") -> {
            System.out.println("You got it");
        }
        case String s
        when s.equalsIgnoreCase("NO") -> {
            System.out.println("Shame");
        }
        case String s -> {
            System.out.println("Sorry?");
        }
    }
}
```

[java_111] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Switches and enum constants

Sumber: ht

The use of enum constants in
case
labels is, at present, highly constrained:
The selector expression of the
switch
must be of the enum type, and the labels
must be simple names of the enum's constants. For example:
Even after adding pattern labels, this constraint leads to unnecessarily verbose
code. For example:
This code would be more readable if we could have a separate
case
for each
enum constant rather than lots of guarded patterns. We therefore relax the
requirement that the selector expression be of the enum type and we allow
case
constants to use qualified names of enum constants. This allows the above code
to be rewritten as:
Now we have a direct case for each enum constant without using guarded type

patterns, which were previously used simply to work around the present constraint of the type system.

Contoh Kode / Perintah CLI:

```
// Prior to Java 21
public enum Suit { CLUBS, DIAMONDS, HEARTS, SPADES }

static void testforHearts(Suit s) {
    switch (s) {
        case HEARTS -> System.out.println("It's a heart!");
        default -> System.out.println("Some other suit");
    }
}

// ----------------------------------------

// As of Java 21
sealed interface CardClassification permits Suit, Tarot {}
public enum Suit implements CardClassification { CLUBS, DIAMONDS, HEARTS, SPADES }
final class Tarot implements CardClassification {}

static void exhaustiveSwitchWithoutEnumSupport(CardClassification c) {
    switch (c) {
        case Suit s when s == Suit.CLUBS -> {
            System.out.println("It's clubs");
        }
        case Suit s when s == Suit.DIAMONDS -> {
            System.out.println("It's diamonds");
        }
        case Suit s when s == Suit.HEARTS -> {
            System.out.println("It's hearts");
        }
        case Suit s -> {
            System.out.println("It's spades");
        }
        case Tarot t -> {
            System.out.println("It's a tarot");
        }
    }
}

// ----------------------------------------

// As of Java 21
static void exhaustiveSwitchWithBetterEnumSupport(CardClassification c) {
    switch (c) {
        case Suit.CLUBS -> {
            System.out.println("It's clubs");
        }
        case Suit.DIAMONDS -> {
            System.out.println("It's diamonds");
        }
        case Suit.HEARTS -> {
            System.out.println("It's hearts");
        }
        case Suit.SPADES -> {
            System.out.println("It's spades");
        }
        case Tarot t -> {
            System.out.println("It's a tarot");
        }
    }
```

```
}
```

[java_112] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Description

Sumber: ht

We enhance
switch
statements and expressions in four ways:
Improve enum constant
case
labels,
Extend
case
labels to include patterns and
null
in addition to constants,
Broaden the range of types permitted for the selector expressions of both
switch
statements and
switch
expressions (along with the required richer
analysis of exhaustiveness of switch blocks), and
Allow optional
when
clauses to follow
case
labels.

[java_113] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Improved enum constant case labels

Sumber: ht

It has long been a requirement that, when switching over an enum type, the only
valid
case
constants were enum constants. But this is a strong requirement
that becomes burdensome with the new, richer forms of switch.
To maintain compatibility with existing Java code, when switching over an enum
type a
case
constant can still use the simple name of a constant of the enum
type being switched over.
For new code, we extend the treatment of enums. First, we allow qualified names
of enum constants to appear as
case
constants. These qualified names can be
used when switching over an enum type.
Second, we drop the requirement that the selector expression be of an enum type
when the name of one of that enum's constants is used as a
case
constant. In

that situation we require the name to be qualified and its value to be
assignment compatible with the type of the selector expression. (This aligns the
treatment of enum
case
constants with the numerical
case
constants.)
For example, the following two methods are allowed:
The following example is not allowed:

Contoh Kode / Perintah CLI:

```
// As of Java 21
sealed interface Currency permits Coin {}
enum Coin implements Currency { HEADS, TAILS }

static void goodEnumSwitch1(Currency c) {
    switch (c) {
        case Coin.HEADS -> {    // Qualified name of enum constant as a label
            System.out.println("Heads");
        }
        case Coin.TAILS -> {
            System.out.println("Tails");
        }
    }
}

static void goodEnumSwitch2(Coin c) {
    switch (c) {
        case HEADS -> {
            System.out.println("Heads");
        }
        case Coin.TAILS -> {    // Unnecessary qualification but allowed
            System.out.println("Tails");
        }
    }
}

// ----------------------------------------

// As of Java 21
static void badEnumSwitch(Currency c) {
    switch (c) {
        case Coin.HEADS -> {
            System.out.println("Heads");
        }
        case TAILS -> {         // Error - TAILS must be qualified
            System.out.println("Tails");
        }
        default -> {
            System.out.println("Some currency");
        }
    }
}
```

[java_114] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Patterns in switch labels

Sumber: ht

We revise the grammar for switch labels in a switch block to read (compare
JLS Section 14.11.1
):
The main enhancement is to introduce a new
case
label,
case p
, where
p
is a
pattern. The essence of
switch
is unchanged: The value of the selector
expression is compared to the switch labels, one of the labels is selected, and
the code associated with that label is executed or evaluated. The difference now
is that for
case
labels with patterns, the label selected is determined by the
result of pattern matching rather than by an equality test. For example, in the
following code, the value of
obj
matches the pattern
Long l
, and the
expression associated with the label
case Long l
is evaluated:
After a successful pattern match we often further test the result of the
match. This can lead to cumbersome code, such as:
The desired test -- that
obj
is a
String
of length 1 -- is unfortunately split
between the pattern
case
label and the following
if
statement.
To address this we introduce
guarded pattern
case
labels
by allowing an
optional
guard
, which is a boolean expression, to follow the pattern
label. This permits the above code to be rewritten so that all the conditional

logic is lifted into the switch label:
The first clause matches if
obj
is both a
String
and
of length 1. The second
case matches if
obj
is a
String
of any length.
Only pattern labels can have guards. For example, it is not valid to write a
label with a
case
constant and a guard; e.g.,
case "Hello" when callRandomBooleanExpression()
.
There are five major language design areas to consider when supporting
patterns in
switch
:
Enhanced type checking
Exhaustiveness of
switch
expressions and statements
Scope of pattern variable declarations
Dealing with
null
Errors

Contoh Kode / Perintah CLI:

```
SwitchLabel:
  case CaseConstant { , CaseConstant }
  case null [, default]
  case Pattern [ Guard ]
  default

// ----------------------------------------

// As of Java 21
static void patternSwitchTest(Object obj) {
    String formatted = switch (obj) {
        case Integer i -> String.format("int %d", i);
        case Long l    -> String.format("long %d", l);
        case Double d  -> String.format("double %f", d);
        case String s  -> String.format("String %s", s);
        default        -> obj.toString();
    };
}

// ----------------------------------------

// As of Java 21
static void testOld(Object obj) {
    switch (obj) {
        case String s:
```

```
            if (s.length() == 1) { ... }
            else { ... }
            break;
        ...
    }
}

// ----------------------------------------

// As of Java 21
static void testNew(Object obj) {
    switch (obj) {
        case String s when s.length() == 1 -> ...
        case String s                      -> ...
        ...
    }
}
```

[java_115] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Enhanced type checking

Sumber: ht

Selector expression typing
Supporting patterns in
switch
means that we can relax the restrictions on the
type of the selector expression. Currently the type of the selector expression
of a normal
switch
must be either an integral primitive type (excluding
long
), the corresponding boxed form (i.e.,
Character
,
Byte
,
Short
, or
Integer
),
String
, or an
enum
type. We extend this and require that the
type of the selector expression be either an integral primitive type (excluding
long
) or any reference type.
For example, in the following pattern
switch
the selector expression
obj
is
matched with type patterns involving a class type, an
enum
type, a record type,

and an array type, along with a
null
case
label and a
default
:
Every
case
label in the switch block must be compatible with the selector
expression. For a
case
label with a pattern, known as a
pattern label
, we
use the existing notion of
compatibility of an expression with a pattern
(
JLS Section 14.30.1
).
Dominance of
case
labels
Supporting pattern
case
labels means that for a given value of the selector
expression it is now possible for more than one
case
label to apply, whereas
previously at most one
case
label could apply. For example, if the selector
expression evaluates to a
String
then both the
case
labels
case String s
and
case CharSequence cs
would apply.
The first issue to resolve is deciding exactly which label should apply in this
circumstance. Rather than attempt a complicated best-fit approach, we adopt a
simpler semantics: The first
case
label appearing in a switch block that
applies to a value is chosen.
In this example, if the value of
obj
is of type
String
then the first
case

label will apply; if it is of type
CharSequence
but not of type
String
then
the second pattern label will apply.
But what happens if we swap the order of these two
case
labels?
Now if the value of
obj
is of type
String
the
CharSequence
case
label
applies, since it appears first in the switch block. The
String
case
label
is unreachable in the sense that there is no value of the selector expression that would cause it to be chosen. By analogy to unreachable code, this is treated as a programmer error and results in a compile-time error.
More precisely, we say that the first
case
label
case CharSequence cs
dominates
the second
case
label
case String s
because every value that
matches the pattern
String s
also matches the pattern
CharSequence cs
, but
not vice versa. This is because the type of the second pattern,
String
, is a
subtype of the type of the first pattern,
CharSequence
.
An unguarded pattern
case
label dominates a guarded pattern
case
label that
has the same pattern. For example, the (unguarded) pattern
case

label
case String s
dominates the guarded pattern
case
label
case String s when s.length() > 0
, since every value that matches the
case
label
case String s when s.length() > 0
must match the
case
label
case String s
.
A guarded pattern
case
label dominates another pattern
case
label (guarded
or unguarded) only when both the former's pattern dominates the latter's pattern
and
when its guard is a constant expression of value
true
. For example, the
guarded pattern
case
label
case String s when true
dominates the pattern
case
label
case String s
. We do not analyze the guarding expression any
further in order to determine more precisely which values match the pattern
label -- a problem which is undecidable in general.
A pattern
case
label can dominate a constant
case
label. For example, the
pattern
case
label
case Integer i
dominates the constant
case
label
case 42
, and the pattern
case
label

case E e
dominates the constant
case
label
case A
when
A
is a member of
enum
class type
E
. A guarded pattern
case
label dominates a constant
case
label if the same pattern
case
label
without the guard does. In other words, we do not check the guard, since this is
undecidable in general. For example, the pattern
case
label
case String s when s.length() > 1
dominates the constant
case
label
case "hello"
, as
expected; but
case Integer i when i != 0
dominates the
case
label
case 0
.
All of this suggests a simple, predictable, and readable ordering of
case
labels
in which the constant
case
labels should appear before the guarded pattern
case
labels, and those should appear before the unguarded pattern
case
labels:
The compiler checks all
case
labels. It is a compile-time error for a
case
label in a switch block to be dominated by any preceding
case
label in that

switch block. This dominance requirement ensures that if a switch block
contains only type pattern
case
labels, they will appear in subtype order.
(The notion of dominance is analogous to conditions on the
catch
clauses of a
try
statement, where it is an error if a
catch
clause that catches an
exception class
E
is preceded by a
catch
clause that can catch
E
or a
superclass of
E
(
JLS Section 11.2.3
). Logically, the preceding
catch
clause dominates the subsequent
catch
clause.)
It is also a compile-time error for a switch block of a
switch
expression or
switch
statement to have more than one match-all switch label. The match-all
labels are
default
and pattern
case
labels where the pattern unconditionally
matches the selector expression. For example, the type pattern
String s
unconditionally matches a selector expression of type
String
, and the type
pattern
Object o
unconditionally matches a selector expression of any
reference type:

Contoh Kode / Perintah CLI:

```
// As of Java 21
record Point(int i, int j) {}
enum Color { RED, GREEN, BLUE; }

static void typeTester(Object obj) {
```

```
    switch (obj) {
        case null     -> System.out.println("null");
        case String s -> System.out.println("String");
        case Color c  -> System.out.println("Color: " + c.toString());
        case Point p  -> System.out.println("Record class: " + p.toString());
        case int[] ia -> System.out.println("Array of ints of length" + ia.length);
        default       -> System.out.println("Something else");
    }
}

// ----------------------------------------

// As of Java 21
static void first(Object obj) {
    switch (obj) {
        case String s ->
            System.out.println("A string: " + s);
        case CharSequence cs ->
            System.out.println("A sequence of length " + cs.length());
        default -> {
            break;
        }
    }
}

// ----------------------------------------

// As of Java 21
static void error(Object obj) {
    switch (obj) {
        case CharSequence cs ->
            System.out.println("A sequence of length " + cs.length());
        case String s ->    // Error - pattern is dominated by previous pattern
            System.out.println("A string: " + s);
        default -> {
            break;
        }
    }
}

// ----------------------------------------

// As of Java 21
Integer i = ...
switch (i) {
    case -1, 1 -> ...                    // Special cases
    case Integer j when j > 0 -> ...     // Positive integer cases
    case Integer j -> ...                // All the remaining integers
}

// ----------------------------------------

// As of Java 21
static void matchAll(String s) {
    switch(s) {
        case String t:
            System.out.println(t);
            break;
        default:
            System.out.println("Something else");  // Error - dominated!
    }
}
```

```
static void matchAll2(String s) {
    switch(s) {
        case Object o:
            System.out.println("An Object");
            break;
        default:
            System.out.println("Something else");  // Error - dominated!
    }
}
```

[java_116] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Exhaustiveness of switch expressions and statements

Sumber: ht

Type coverage
A
switch
expression requires that all possible values of the selector
expression be handled in the switch block; in other words, it must be
exhaustive
.
This maintains the property that successful evaluation of a
switch
expression
always yields a value.
For normal
switch
expressions, this property is enforced by a straightforward
set of extra conditions on the switch block.
For pattern
switch
expressions and statements, we achieve this by defining a
notion of
type coverage
of switch labels in a switch block. The type coverage
of all the switch labels in the switch block is then combined to determine if
the switch block exhausts all the possibilities of the selector expression.
Consider this (erroneous) pattern
switch
expression:
The switch block has only one switch label,
case String s
. This matches any
value of
obj
whose type is a subtype of
String
. We
therefore say that the type coverage of this switch label is every subtype of
String
. This pattern

switch
expression is not exhaustive because the type
coverage of its switch block (all subtypes of
String
) does not include the
type of the selector expression (
Object
).
Consider this (still erroneous) example:
The type coverage of this switch block is the union of the coverage of its two
switch labels. In other words, the type coverage is the set of all subtypes of
String
and the set of all subtypes of
Integer
. But, again, the type coverage
still does not include the type of the selector expression, so this pattern
switch
expression is also not exhaustive and causes a compile-time error.
The type coverage of a
default
label is every type, so this example is (at
last!) legal:
Exhaustiveness in practice
The notion of type coverage already exists in non-pattern
switch
expressions. For example:
This
switch
expression over an enum class is not exhaustive because the
anticipated input
YELLOW
is not covered. As expected, adding a
case
label to
handle the
YELLOW
enum constant is sufficient to make the
switch
exhaustive:
That a
switch
written this way is exhaustive has two important benefits.
First, it would be cumbersome to have to write a
default
clause, which
probably just throws an exception, since we have already handled all the cases:
Manually writing a
default
clause in this situation is not only irritating
but actually pernicious, since the compiler can do a better job of checking
exhaustiveness without one. (The same is true of any other match-all clause
such as

default
,
case null, default
, or an unconditional type pattern.) If
we omit the
default
clause then we will discover at compile time if we have
forgotten a
case
label, rather than finding out at run time -- and maybe not
even then.
More importantly, what happens if someone later adds another constant to the
Color
enum? If we have an explicit match-all clause then we will only
discover the new constant value if it shows up at run time. But if we code the
switch
to cover all the constants known at compile time, and omit the
match-all clause, then we will find out about this change the next time we
recompile the class containing the
switch
. A match-all clause risks sweeping
exhaustiveness errors under the rug.
In conclusion: An exhaustive
switch
without a match-all clause is better than
an exhaustive
switch
with one, when possible.
Looking to run time, what happens if a new
Color
constant is added, and the
class containing the
switch
is not recompiled? There is a risk that the new
constant will be exposed to our
switch
. Because this risk is always present
with enums, if an exhaustive enum
switch
does not have a match-all clause
then the compiler will synthesize a
default
clause that throws an
exception. This guarantees that the
switch
cannot complete normally without
selecting one of the clauses.
The notion of exhaustiveness is designed to strike a balance between covering all reasonable cases while not forcing you to write possibly many rare corner cases that will pollute or even dominate your code for little actual value. Put another way: Exhaustiveness is a compile-time approximation of true run-time exhaustiveness.

Exhaustiveness and sealed classes

If the type of the selector expression is a sealed class
(
JEP 409
) then the type coverage check can take into account the
permits
clause of the sealed class to determine whether a switch block is
exhaustive. This can sometimes remove the need for a
default
clause, which as
argued above is good practice. Consider the following example of a
sealed
interface
S
with three permitted subclasses
A
,
B
, and
C
:

The compiler can determine that the type coverage of the switch block is the
types
A
,
B
, and
C
. Since the type of the selector expression,
S
, is a
sealed interface whose permitted subclasses are exactly
A
,
B
, and
C
, this
switch block is exhaustive. As a result, no
default
label is needed.

Some extra care is needed when a permitted direct subclass only implements a
specific parameterization of a (generic)
sealed
superclass. For example:

The only permitted subclasses of
I
are
A
and
B
, but the compiler can

detect that the switch block need only cover the class
B
to be exhaustive
since the selector expression is of type
I<Integer>
and no parameterization
of
A
is a subtype of
I<Integer>
.
Again, the notion of exhaustiveness is an approximation. Because of separate compilation, it is possible for a novel implementation of the interface
I
to
show up at runtime, so the compiler will in this case insert a synthetic
default
clause that throws.
The notion of exhaustiveness is made more complicated by record patterns
(
JEP 440
) since record patterns can be
nested. Accordingly, the notion of exhaustiveness must reflect this potentially recursive structure.
Exhaustiveness and compatibility
The requirement of exhaustiveness applies to both pattern
switch
expressions
and pattern
switch
statements. To ensure backward compatibility, all
existing
switch
statements will compile unchanged. But if a
switch
statement
uses any of the
switch
enhancements described in this JEP then the compiler will check that it is exhaustive. (Future compilers of the Java language may emit warnings for legacy
switch
statements that are not exhaustive.)
More precisely, exhaustiveness is required of any
switch
statement that uses
pattern or
null
labels or whose selector expression is not one of the legacy types (
char
,

byte
,
short
,
int
,
Character
,
Byte
,
Short
,
Integer
,
String
, or an
enum
type). For example:

Contoh Kode / Perintah CLI:

```
// As of Java 21
static int coverage(Object obj) {
    return switch (obj) {           // Error - not exhaustive
        case String s -> s.length();
    };
}

// ----------------------------------------

// As of Java 21
static int coverage(Object obj) {
    return switch (obj) {           // Error - still not exhaustive
        case String s  -> s.length();
        case Integer i -> i;
    };
}

// ----------------------------------------

// As of Java 21
static int coverage(Object obj) {
    return switch (obj) {
        case String s  -> s.length();
        case Integer i -> i;
        default -> 0;
    };
}

// ----------------------------------------

// As of Java 20
enum Color { RED, YELLOW, GREEN }

int numLetters = switch (color) {   // Error - not exhaustive!
    case RED -> 3;
    case GREEN -> 5;
}

// ----------------------------------------
```

```
// As of Java 20
int numLetters = switch (color) {   // Exhaustive!
    case RED -> 3;
    case GREEN -> 5;
    case YELLOW -> 6;
}

// ----------------------------------------

int numLetters = switch (color) {
    case RED -> 3;
    case GREEN -> 5;
    case YELLOW -> 6;
    default -> throw new ArghThisIsIrritatingException(color.toString());
}

// ----------------------------------------

// As of Java 21
sealed interface S permits A, B, C {}
final class A implements S {}
final class B implements S {}
record C(int i) implements S {}    // Implicitly final

static int testSealedExhaustive(S s) {
    return switch (s) {
        case A a -> 1;
        case B b -> 2;
        case C c -> 3;
    };
}

// ----------------------------------------

// As of Java 21
sealed interface I<T> permits A, B {}
final class A<X> implements I<String> {}
final class B<Y> implements I<Y> {}

static int testGenericSealedExhaustive(I<Integer> i) {
    return switch (i) {
        // Exhaustive as no A case possible!
        case B<Integer> bi -> 42;
    };
}

// ----------------------------------------

// As of Java 21
sealed interface S permits A, B, C {}
final class A implements S {}
final class B implements S {}
record C(int i) implements S {}    // Implicitly final

static void switchStatementExhaustive(S s) {
    switch (s) {                   // Error - not exhaustive;
                                   // missing clause for permitted class B!
        case A a :
            System.out.println("A");
            break;
        case C c :
```

```
            System.out.println("C");
            break;
    };
}
```

[java_117] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Scope of pattern variable declarations

Sumber: ht

Pattern variables
(
JEP 394
) are local variables that are
declared by patterns. Pattern variable declarations are unusual in that their
scope is
flow-sensitive
. As a recap consider the following example, where the
type pattern
String s
declares the pattern variable
s
:
The declaration of
s
is in scope in the parts of the code where the pattern
variable
s
will have been initialized. In this example, that is in the
right-hand operand of the
&&
expression and in the "then" block. However,
s
is not in scope in the "else" block: In order for control to transfer to the
"else" block the pattern match must fail, in which case the pattern variable
will not have been initialized.
We extend this flow-sensitive notion of scope for pattern variable declarations
to encompass pattern declarations occurring in
case
labels with three new
rules:
The scope of a pattern variable declaration which occurs in the pattern of a
guarded
case
label includes the guard, i.e., the
when
expression.
The scope of a pattern variable declaration which occurs in a
case
label of
a
switch
rule includes the expression, block, or
throw

statement that
appears to the right of the arrow.
The scope of a pattern variable declaration which occurs in a
case
label of
a
switch
labeled statement group includes the block statements of the
statement group. Falling through a
case
label that declares a pattern
variable is forbidden.
This example shows the first rule in action:
The scope of the declaration of the pattern variable
c
includes the guard,
i.e., the expression
c.charValue() == 7
.
This variant shows the second rule in action:
Here the scope of the declaration of the pattern variable
c
is the block to
the right of the first arrow. The scope of the declaration of the pattern
variable
i
is the
throw
statement to the right of the second arrow.
The third rule is more complicated. Let us first consider an example where
there is only one
case
label for a
switch
labeled statement group:
The scope of the declaration of the pattern variable
c
includes all the
statements of the statement group, namely the two
if
statements and
the
println
statement. The scope does not include the statements of the
default
statement group, even though the execution of the first statement
group can fall through the
default
switch label and execute these statements.
We forbid the possibility of falling through a
case
label that declares a

pattern variable. Consider this erroneous example:
If this were allowed and the value of
obj
were a
Character
then execution of the switch block could fall through the second
statement group, after
case Integer i:
, where the pattern variable
i
would
not have been initialized. Allowing execution to fall through a
case
label that
declares a pattern variable is therefore a compile-time error.
This is why a switch label consisting of multiple pattern labels, e.g.
case Character c: case Integer i: ...
, is not permitted. Similar reasoning applies
to the prohibition of multiple patterns within a single
case
label: Neither
case Character c, Integer i: ...
nor
case Character c, Integer i -> ...
is
allowed. If such
case
labels were allowed then both
c
and
i
would be in
scope after the colon or arrow, yet only one of them would have been initialized
depending on whether the value of
obj
was a
Character
or an
Integer
.
On the other hand, falling through a label that does not declare a pattern
variable is safe, as this example shows:

Contoh Kode / Perintah CLI:

```
// As of Java 21
static void testFlowScoping(Object obj) {
    if ((obj instanceof String s) && s.length() > 3) {
        System.out.println(s);
    } else {
        System.out.println("Not a string");
    }
}

// ----------------------------------------
```

```
// As of Java 21
static void testScope1(Object obj) {
    switch (obj) {
        case Character c
        when c.charValue() == 7:
            System.out.println("Ding!");
            break;
        default:
            break;
    }
}

// ----------------------------------------

// As of Java 21
static void testScope2(Object obj) {
    switch (obj) {
        case Character c -> {
            if (c.charValue() == 7) {
                System.out.println("Ding!");
            }
            System.out.println("Character");
        }
        case Integer i ->
            throw new IllegalStateException("Invalid Integer argument: "
                                            + i.intValue());
        default -> {
            break;
        }
    }
}

// ----------------------------------------

// As of Java 21
static void testScope3(Object obj) {
    switch (obj) {
        case Character c:
            if (c.charValue() == 7) {
                System.out.print("Ding ");
            }
            if (c.charValue() == 9) {
                System.out.print("Tab ");
            }
            System.out.println("Character");
        default:
            System.out.println();
    }
}

// ----------------------------------------

// As of Java 21
static void testScopeError(Object obj) {
    switch (obj) {
        case Character c:
            if (c.charValue() == 7) {
                System.out.print("Ding ");
            }
            if (c.charValue() == 9) {
                System.out.print("Tab ");
```

```
            }
            System.out.println("character");
        case Integer i:                 // Compile-time error
            System.out.println("An integer " + i);
        default:
            break;
    }
}

// ----------------------------------------

// As of Java 21
void testScope4(Object obj) {
    switch (obj) {
        case String s:
            System.out.println("A string: " + s);  // s in scope here!
        default:
            System.out.println("Done");            // s not in scope here
    }
}
```

[java_118] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Dealing with null

Sumber: ht

Traditionally, a
switch
throws
NullPointerException
if the selector
expression evaluates to
null
. This is well-understood behavior and we do not
propose to change it for any existing
switch
code. There are, however,
reasonable and non-exception-raising semantics for pattern matching and
null
values, so in pattern switch blocks we can treat
null
in a more regular
fashion whilst remaining compatible with existing
switch
semantics.
First, we introduce a new
null
case label. We then lift the blanket rule that
a
switch
immediately throws
NullPointerException
if the value of the
selector expression is
null
. Instead we inspect the

case
labels to determine
the behavior of a
switch
:
If the selector expression evaluates to
null
then any
null
case label is
said to match. If there is no such label associated with the switch block then
the
switch
throws
NullPointerException
, as before.
If the selector expression evaluates to a non-
null
value then we select a
matching
case
label, as normal. If no
case
label matches then any
default
label is considered to match.
For example, given the declaration below, evaluating
nullMatch(null)
will print
null!
rather than throw
NullPointerException
:
A switch block without a
case null
label is treated as if it has a
case null
rule whose body throws
NullPointerException
. In other words, this code:
is equivalent to:
In both examples, evaluating
nullMatch(null)
will cause
NullPointerException
to
be thrown.
We preserve the intuition from the existing
switch
construct that performing a
switch over
null

is an exceptional thing to do. The difference in a pattern
switch
is that you can directly handle this case inside the
switch
. If you
see a
null
label in a switch block then that label will match a
null
value. If you do not see a
null
label in a switch block then switching over a
null
value will throw
NullPointerException
, as before. The treatment of
null
values in switch blocks is thus regularized.
It is meaningful, and not uncommon, to want to combine a
null
case with a
default
. To that end we allow
null
case labels to have an optional
default
; for example:
The value of
obj
matches this label if either it is the null reference value,
or none of the other
case
labels match.
It is a compile-time error for a switch block to have both a
null
case
label with a
default
and a
default
label.

Contoh Kode / Perintah CLI:

```
// As of Java 21
static void nullMatch(Object obj) {
    switch (obj) {
        case null     -> System.out.println("null!");
        case String s -> System.out.println("String");
        default       -> System.out.println("Something else");
    }
}

// ----------------------------------------
```

```
// As of Java 21
static void nullMatch2(Object obj) {
    switch (obj) {
        case String s  -> System.out.println("String: " + s);
        case Integer i -> System.out.println("Integer");
        default        -> System.out.println("default");
    }
}

// ----------------------------------------

// As of Java 21
static void nullMatch2(Object obj) {
    switch (obj) {
        case null      -> throw new NullPointerException();
        case String s  -> System.out.println("String: " + s);
        case Integer i -> System.out.println("Integer");
        default        -> System.out.println("default");
    }
}

// ----------------------------------------

// As of Java 21
Object obj = ...
switch (obj) {
    ...
    case null, default ->
        System.out.println("The rest (including null)");
}
```

[java_119] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Errors

Sumber: ht

Pattern matching can complete abruptly. For example, when matching a value against a record pattern, the record's accessor method can complete abruptly. In this case, pattern matching is defined to complete abruptly by throwing a
MatchException
. If such a pattern appears as a label in a
switch
then the
switch
will also complete abruptly by throwing a
MatchException
.
If a
case
pattern has a guard, and evaluating the guard completes abruptly, then the
switch
completes abruptly for the same reason.
If no label in a pattern
switch
matches the value of the selector expression then the

switch
completes abruptly by throwing a
MatchException
, since pattern switches must be exhaustive.
For example:
The invocation
exampleAnR(new R(42))
causes a
MatchException
to be thrown.
(A record accessor method which always throws an exception is highly irregular,
and an exhaustive pattern
switch
which throws a
MatchException
is highly
unusual.)
By contrast:
The invocation
example(new R(42))
causes an
ArithmeticException
to be
thrown.
To align with pattern
switch
semantics,
switch
expressions over
enum
classes now throw
MatchException
rather than
IncompatibleClassChangeError
when no switch label applies at run time. This is a minor incompatible change to
the language. (An exhaustive
switch
over an enum fails to match only if the
enum
class is changed after the
switch
has been compiled, which is highly
unusual.)

Contoh Kode / Perintah CLI:

```
// As of Java 21
record R(int i) {
    public int i() {    // bad (but legal) accessor method for i
        return i / 0;
    }
}

static void exampleAnR(R r) {
    switch(r) {
```

```
        case R(var i): System.out.println(i);
    }
}

// ----------------------------------------

// As of Java 21
static void example(Object obj) {
    switch (obj) {
        case R r when (r.i / 0 == 1): System.out.println("It's an R!");
        default: break;
    }
}
```

[java_120] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Future work

Sumber: ht

At the moment, pattern
switch
does not support the primitive types
boolean
,
long
,
float
, and
double
. Allowing these primitive types would
also mean allowing them in
instanceof
expressions, and aligning primitive
type patterns with reference type patterns, which would require considerable
additional work. This is left for a possible future JEP.
We expect that, in the future, general classes will be able to declare
deconstruction patterns to specify how they can be matched against. Such
deconstruction patterns can be used with a pattern
switch
to yield very
succinct code. For example, if we have a hierarchy of
Expr
with subtypes for
IntExpr
(containing a single
int
),
AddExpr
and
MulExpr
(containing two
Expr
s), and
NegExpr
(containing a single

Expr
), we can match against an
Expr
and act on the specific subtypes all in one step:
Without such pattern matching, expressing ad-hoc polymorphic calculations like
this requires using the cumbersome
visitor pattern
. Pattern
matching is generally more transparent and straightforward.
It may also be useful to add AND and OR patterns, to allow more expressivity
for
case
labels with patterns.

Contoh Kode / Perintah CLI:

```
// Some future Java
int eval(Expr n) {
     return switch (n) {
         case IntExpr(int i) -> i;
         case NegExpr(Expr n) -> -eval(n);
         case AddExpr(Expr left, Expr right) -> eval(left) + eval(right);
         case MulExpr(Expr left, Expr right) -> eval(left) * eval(right);
         default -> throw new IllegalStateException();
     };
}
```

[java_121] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Alternatives

Sumber: ht

Rather than support pattern
switch
we could instead define a
type
switch
that just supports switching on the type of the selector
expression. This feature is simpler to specify and implement but
considerably less expressive.
There are many other syntactic options for guarded pattern labels, such as
p where e
,
p if e
, or even
p &&& e
.
An alternative to guarded pattern labels is to support
guarded patterns
directly as a special pattern form, e.g.,
p && e
. Having experimented with
this in earlier previews, the resulting ambiguity with boolean expressions
led us to prefer guarded
case
labels over guarded patterns.

[java_122] Java - Pattern Matching - Switch Patterns

JEP 441: Pattern Matching for switch: Dependencies

Sumber: ht

This JEP builds on
Pattern Matching for
instanceof
(
JEP 394
),
delivered in JDK 16, and also the enhancements offered by
Switch
Expressions
(
JEP 361
). It has co-evolved with
Record Patterns
(
JEP 440
).
Installing
Contributing
Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds
Mailing lists
Wiki
*
IRC
Mastodon
Bluesky
Bylaws
*
Census
Legal
*
AI
Workshop
JEP Process
Source code
GitHub
Mercurial
Tools
Git
jtreg harness
Groups
(overview
,
archive
)
Adoption

Build
Client Libraries
Compatibility & Specification Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
Members
Networking
Porters
Quality
Security
Serviceability
Vulnerability
Web
Projects
(
overview
,
archive
)
Amber
Babylon
Brisbane
CRaC
Code Tools
Coin
Common VM Interface
Detroit
Developers' Guide
Duke
IcedTea
JDK 8 Updates
JDK 9
JDK
(...,
26
,
27
,
28
)
JDK Updates
JMC
Jigsaw
Lanai
Leyden
Lilliput
Loom

Memory Model Update
Multi-Language VM
Nashorn
New I/O
OpenJFX
Panama
Port: AArch32
Port: AArch64
Port: BSD
Port: Haiku
Port: MIPS
Port: Mobile
Port: PowerPC/AIX
Port: RISC-V
Port: s390x
SCTP
Shenandoah
Skara
Sumatra
Tsan
Valhalla
Wakefield
Zero
ZGC
(c) 2026 Oracle Corporation and/or its affiliates
Terms of Use
*

License:
GPLv2
*
Privacy
*
Trademarks

[java_123] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Overview & Metadata

Sumber: ht

<noscript></noscript>
TABLE { border-collapse: collapse; padding: 0px; margin: 1em 0 1em 2em; }
TR:first-child TH, TR:first-child TD { padding-top: 0; }
TH, TD { padding: 0px; padding-top: .5ex; vertical-align: baseline; text-align: left; }
TD + TD, TH + TH { padding-left: 1em; }
TD:first-child, TH:first-child, TD.jep { text-align: right; }
TABLE.head TD:first-child { font-style: italic; padding-left: 2em; white-space: nowrap; }
PRE { padding-left: 2em; margin: 1ex 0; font-size: inherit; }
TABLE PRE { padding-left: 0; margin: 0; }
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { padding-left: .5em; }
TABLE.jeps TD:first-child,
TABLE.jeps TD:first-child + TD,

```
TABLE.jeps TD:first-child + TD + TD { font-size: smaller; }
TABLE.jeps TD.cl { font-size: smaller; padding-right: 0; text-align: right; }
TABLE.jeps TD.cm { font-size: smaller; padding-left: .1em; padding-right: .1em; }
TABLE.jeps TD.cr { font-size: smaller; padding-left: 0; }
TABLE.jeps TD.z { padding-left: 0; padding-right: 0; }
TABLE.head TD { padding-top: 0; }
```

JEP 440: Record Patterns
Owner
Gavin Bierman
Type
Feature
Scope
SE
Status
Closed / Delivered
Release
21
Component
specification / language
Discussion
amber dash dev at openjdk dot org
Relates to
JEP 432: Record Patterns (Second Preview)
Reviewed by
Brian Goetz
Endorsed by
Brian Goetz
Created
2023/01/18 14:38
Updated
2023/08/28 16:51
Issue
8300541

[java_124] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Summary

Sumber: ht

Enhance the Java programming language with
record patterns
to deconstruct
record values. Record patterns and type patterns can be nested to enable a
powerful, declarative, and composable form of data navigation and processing.

[java_125] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: History

Sumber: ht

Record patterns were proposed as a preview feature by
JEP 405
and
delivered in
JDK 19

, and previewed
a second time by
JEP 432
and delivered in
JDK 20
. This feature has co-evolved
with
Pattern Matching for
switch
(
JEP 441
), with which it has
considerable interaction. This JEP proposes to finalize the feature with further refinements based upon continued experience and feedback.
Apart from some minor editorial changes, the main change since the second preview is to remove support for record patterns appearing in the header of an
enhanced
for
statement. This feature may be re-proposed in a future JEP.

[java_126] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Goals

Sumber: ht

Extend pattern matching to destructure instances of
record
classes,
enabling more sophisticated data queries.
Add nested patterns, enabling more composable data queries.

[java_127] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Motivation

Sumber: ht

In Java 16,
JEP 394
extended the
instanceof
operator to take a
type pattern
and perform
pattern matching
. This modest extension allows the
familiar instanceof-and-cast idiom to be simplified, making it both more concise and less error-prone:
In the new code,
obj
matches the type pattern
String s
if, at run time, the
value of
obj
is an instance of

String
. If the pattern matches then the
instanceof
expression is
true
and the pattern variable
s
is initialized to
the value of
obj
cast to
String
, which can then be used in the contained
block.
Type patterns remove many occurrences of casting at a stroke. However, they are only the first step towards a more declarative, data-focused style of programming. As Java supports new and more expressive ways of modeling data, pattern matching can streamline the use of such data by enabling developers to express the semantic intent of their models.

Contoh Kode / Perintah CLI:

```
// Prior to Java 16
if (obj instanceof String) {
    String s = (String)obj;
    ... use s ...
}

// As of Java 16
if (obj instanceof String s) {
    ... use s ...
}
```

[java_128] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Pattern matching and records

Sumber: ht

Records (
JEP 395
) are transparent carriers for data. Code that
receives an instance of a record class will typically extract the data, known as
the
components
, using the built-in component accessor methods. For example, we
can use a type pattern to test whether a value is an instance of the record
class
Point
and, if so, extract the
x
and
y
components from the value:
The pattern variable
p
is used here solely to invoke the accessor methods

x()
and
y()
, which return the values of the components
x
and
y
. (In
every record class there is a one-to-one correspondence between its accessor
methods and its components.) It would be better if the pattern could not only
test whether a value is an instance of
Point
but also extract the
x
and
y
components from the value directly, invoking the accessor methods on our behalf.
In other words:
Point(int x, int y)
is a
record pattern
. It lifts the declaration of local
variables for extracted components into the pattern itself, and initializes those
variables by invoking the accessor methods when a value is matched against the
pattern. In effect, a record pattern disaggregates an instance of a record into
its components.

Contoh Kode / Perintah CLI:

```
// As of Java 16
record Point(int x, int y) {}

static void printSum(Object obj) {
    if (obj instanceof Point p) {
        int x = p.x();
        int y = p.y();
        System.out.println(x+y);
    }
}

// ----------------------------------------

// As of Java 21
static void printSum(Object obj) {
    if (obj instanceof Point(int x, int y)) {
        System.out.println(x+y);
    }
}
```

[java_129] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Nested record patterns

Sumber: ht

The true power of pattern matching is that it scales elegantly to match more complicated object graphs. For example, consider the following declarations:
We have already seen that we can extract the components of an object with a record pattern. If we want to extract the color from the upper-left point, we could write:
But the
ColoredPoint
value
ul
is itself a record value, which we might want to decompose further. Record patterns therefore support
nesting
, which allows the record component to be further matched against, and decomposed by, a nested pattern. We can nest another pattern inside the record pattern and decompose both the outer and inner records at once:
Nested patterns allow us, further, to take apart an aggregate with code that is as clear and concise as the code that puts it together. If we were creating a rectangle, for example, we would likely nest the constructors in a single expression:
With nested patterns we can deconstruct such a rectangle with code that echoes the structure of the nested constructors:
Nested patterns can, of course, fail to match:
Here the record pattern
Pair(String s, String t)
contains two nested type patterns, namely
String s
and
String t
. A value matches the pattern
Pair(String s, String t)
if it is a
Pair
and, recursively, its component values match the type patterns
String s
and
String t
. In our example code above these recursive pattern matches fail since neither of the record component values are strings, and thus the
else
block is executed.
In summary, nested patterns elide the accidental complexity of navigating objects so that we can focus on the data expressed by those objects. They also give us the power to centralize error handling, since a value fails to match a nested pattern

P(Q)
if either, or both, of the subpatterns fail to match. We
need not check and handle each individual subpattern matching failure -- either
the entire pattern matches, or not.

Contoh Kode / Perintah CLI:

```
// As of Java 16
record Point(int x, int y) {}
enum Color { RED, GREEN, BLUE }
record ColoredPoint(Point p, Color c) {}
record Rectangle(ColoredPoint upperLeft, ColoredPoint lowerRight) {}

// ----------------------------------------

// As of Java 21
static void printUpperLeftColoredPoint(Rectangle r) {
    if (r instanceof Rectangle(ColoredPoint ul, ColoredPoint lr)) {
         System.out.println(ul.c());
    }
}

// ----------------------------------------

// As of Java 21
static void printColorOfUpperLeftPoint(Rectangle r) {
    if (r instanceof Rectangle(ColoredPoint(Point p, Color c),
                               ColoredPoint lr)) {
        System.out.println(c);
    }
}

// ----------------------------------------

// As of Java 16
Rectangle r = new Rectangle(new ColoredPoint(new Point(x1, y1), c1),
                            new ColoredPoint(new Point(x2, y2), c2));

// ----------------------------------------

// As of Java 21
static void printXCoordOfUpperLeftPointWithPatterns(Rectangle r) {
    if (r instanceof Rectangle(ColoredPoint(Point(var x, var y), var c),
                               var lr)) {
        System.out.println("Upper-left corner: " + x);
    }
}

// ----------------------------------------

// As of Java 21
record Pair(Object x, Object y) {}

Pair p = new Pair(42, 42);

if (p instanceof Pair(String s, String t)) {
    System.out.println(s + ", " + t);
} else {
    System.out.println("Not a pair of strings");
}
```

[java_130] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Description

Sumber: ht

We extend the Java programming language with nestable record patterns.
The grammar for patterns becomes:

Contoh Kode / Perintah CLI:

```
Pattern:
  TypePattern
  RecordPattern

TypePattern:
  LocalVariableDeclaration

RecordPattern:
  ReferenceType ( [ PatternList ] )

PatternList :
  Pattern { , Pattern }
```

[java_131] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Record patterns

Sumber: ht

A
record pattern
consists of a record class type and a (possibly empty)
pattern list which is used to match against the corresponding record component
values.
For example, given the declaration
a value
v
matches the record pattern
Point(int i, int j)
if it is an instance of
the record type
Point
; if so, the pattern variable
i
is initialized with the
result of invoking the accessor method corresponding to
i
on the value
v
, and
the pattern variable
j
is initialized to the result of invoking the accessor
method corresponding to
j
on the value
v
. (The names of the pattern
variables do not need to be the same as the names of the record components; i.e.,

the record pattern
Point(int x, int y)
acts identically except that the
pattern variables
x
and
y
are initialized.)
The
null
value does not match any record pattern.
A record pattern can use
var
to match against a record component without
stating the type of the component. In that case the compiler infers the type of
the pattern variable introduced by the
var
pattern. For example, the pattern
Point(var a, var b)
is shorthand for the pattern
Point(int a, int b)
.
The set of pattern variables declared by a record pattern includes all of the
pattern variables declared in the pattern list.
An expression is compatible with a record pattern if it could be cast to the
record type in the pattern without requiring an unchecked conversion.
If a record pattern names a generic record class but gives no type arguments
(i.e., the record pattern uses a raw type) then the type arguments are always
inferred. For example:
Inference of type arguments for record patterns is supported in all constructs
that support record patterns, namely
instanceof
expressions and
switch
statements and expressions.
Inference works with nested record patterns; for example:
Here the type argument for the nested pattern
Box(var s)
is inferred to be
String
, so the pattern itself is inferred to be
Box<String>(var s)
.
In fact it is possible to drop the type arguments in the outer record pattern as
well, leading to the concise code:
Here the compiler will infer that the entire
instanceof
pattern is
Box<Box<String>>(Box<String>(var s))
,
For compatibility, type patterns do not support the implicit inference of type
arguments; e.g., the type pattern

List l
is always treated as a raw type
pattern.

Contoh Kode / Perintah CLI:

```
record Point(int i, int j) {}

// ----------------------------------------

// As of Java 21
record MyPair<S,T>(S fst, T snd){};

static void recordInference(MyPair<String, Integer> pair){
    switch (pair) {
        case MyPair(var f, var s) ->
            ... // Inferred record pattern MyPair<String,Integer>(var f, var s)
        ...
    }
}

// ----------------------------------------

// As of Java 21
record Box<T>(T t) {}

static void test1(Box<Box<String>> bbs) {
    if (bbs instanceof Box<Box<String>>(Box(var s))) {
        System.out.println("String " + s);
    }
}

// ----------------------------------------

// As of Java 21
static void test2(Box<Box<String>> bbs) {
    if (bbs instanceof Box(Box(var s))) {
        System.out.println("String " + s);
    }
}
```

[java_132] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Record patterns and exhaustive switch

Sumber: ht

JEP 441
enhances both
switch
expressions
and
switch
statements to support pattern labels. Both
switch
expressions
and pattern
switch
statements must be
exhaustive:
The switch block must

have clauses that deal with all possible values of the selector expression. For pattern labels this is determined by analysis of the types of the patterns; for example, the case label
case Bar b
matches values of type
Bar
and all possible subtypes of
Bar
.
With pattern labels involving record patterns, the analysis is more complex since we must consider the types of the component patterns and make allowances for
sealed
hierarchies. For example, consider the declarations:
The following
switch
is not exhaustive, since there is no match for a pair containing two values both of type
A
:
These two switches are exhaustive, since the interface
I
is
sealed
and so the types
C
and
D
cover all possible instances:
In contrast, this
switch
is not exhaustive since there is no match for a pair containing two values both of type
D
:

Contoh Kode / Perintah CLI:

```
class A {}
class B extends A {}
sealed interface I permits C, D {}
final class C implements I {}
final class D implements I {}
record Pair<T>(T x, T y) {}

Pair<A> p1;
Pair<I> p2;

// ----------------------------------------

// As of Java 21
switch (p1) {                 // Error!
    case Pair<A>(A a, B b) -> ...
    case Pair<A>(B b, A a) -> ...
```

```
}

// ----------------------------------------

// As of Java 21
switch (p2) {
    case Pair<I>(I i, C c) -> ...
    case Pair<I>(I i, D d) -> ...
}

switch (p2) {
    case Pair<I>(C c, I i) -> ...
    case Pair<I>(D d, C c) -> ...
    case Pair<I>(D d1, D d2) -> ...
}

// ----------------------------------------

// As of Java 21
switch (p2) {                        // Error!
    case Pair<I>(C fst, D snd) -> ...
    case Pair<I>(D fst, C snd) -> ...
    case Pair<I>(I fst, C snd) -> ...
}
```

[java_133] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Future Work

Sumber: ht

There are many directions in which the record patterns described here could be extended:
Varargs patterns, for records of variable arity;
Unnamed patterns, which can appear in record-pattern pattern lists and which match any value but do not declare pattern variables; and
Patterns that can apply to values of arbitrary classes rather than only record classes.
We may consider some of these in future JEPs.

[java_134] Java - Pattern Matching - Record Patterns

JEP 440: Record Patterns: Dependencies

Sumber: ht

This JEP builds on
Pattern Matching for
instanceof
(
JEP 394
),
delivered in JDK 16. It has co-evolved with
Pattern Matching for
switch
(
JEP 441
).
Installing
Contributing

Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds
Mailing lists
Wiki
*
IRC
Mastodon
Bluesky
Bylaws
*
Census
Legal
*
AI
Workshop
JEP Process
Source code
GitHub
Mercurial
Tools
Git
jtreg harness
Groups
(overview
,
archive
)
Adoption
Build
Client Libraries
Compatibility & Specification Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
Members
Networking
Porters
Quality
Security
Serviceability
Vulnerability
Web
Projects
(
overview
,

archive
)
Amber
Babylon
Brisbane
CRaC
Code Tools
Coin
Common VM Interface
Detroit
Developers' Guide
Duke
IcedTea
JDK 8 Updates
JDK 9
JDK
(...,
26
,
27
,
28
)
JDK Updates
JMC
Jigsaw
Lanai
Leyden
Lilliput
Loom
Memory Model Update
Multi-Language VM
Nashorn
New I/O
OpenJFX
Panama
Port: AArch32
Port: AArch64
Port: BSD
Port: Haiku
Port: MIPS
Port: Mobile
Port: PowerPC/AIX
Port: RISC-V
Port: s390x
SCTP
Shenandoah
Skara
Sumatra
Tsan
Valhalla
Wakefield

Zero
ZGC
(c) 2026 Oracle Corporation and/or its affiliates
Terms of Use
*

License:
GPLv2
*
Privacy
*
Trademarks

[java_135] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Overview & Metadata

Sumber: ht

<noscript></noscript>
TABLE { border-collapse: collapse; padding: 0px; margin: 1em 0 1em 2em; }
TR:first-child TH, TR:first-child TD { padding-top: 0; }
TH, TD { padding: 0px; padding-top: .5ex; vertical-align: baseline; text-align: left; }
TD + TD, TH + TH { padding-left: 1em; }
TD:first-child, TH:first-child, TD.jep { text-align: right; }
TABLE.head TD:first-child { font-style: italic; padding-left: 2em; white-space: nowrap; }
PRE { padding-left: 2em; margin: 1ex 0; font-size: inherit; }
TABLE PRE { padding-left: 0; margin: 0; }
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { padding-left: .5em; }
TABLE.jeps TD:first-child,
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { font-size: smaller; }
TABLE.jeps TD.cl { font-size: smaller; padding-right: 0; text-align: right; }
TABLE.jeps TD.cm { font-size: smaller; padding-left: .1em; padding-right: .1em; }
TABLE.jeps TD.cr { font-size: smaller; padding-left: 0; }
TABLE.jeps TD.z { padding-left: 0; padding-right: 0; }
TABLE.head TD { padding-top: 0; }
JEP 456: Unnamed Variables & Patterns
Owner
Angelos Bimpoudis
Type
Feature
Scope
SE
Status
Closed / Delivered
Release
22
Component
specification / language
Discussion
amber dash dev at openjdk dot org
Effort

S
Duration
S
Relates to
JEP 443: Unnamed Patterns and Variables (Preview)
Reviewed by
Brian Goetz
Endorsed by
Brian Goetz
Created
2023/07/10 16:17
Updated
2026/07/08 17:28
Issue
8311828

[java_136] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Summary

Sumber: ht

Enhance the Java programming language with unnamed variables and unnamed patterns, which can be used when variable declarations or nested patterns are required but never used. Both are denoted by the underscore character,

_
.

[java_137] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: History

Sumber: ht

Unnamed variables and unnamed patterns first previewed in JDK 21 via
JEP
443
, which was titled
Unnamed Patterns and
Variables
. We here propose to finalize this feature without change.

[java_138] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Goals

Sumber: ht

Capture developer intent that a given binding or lambda parameter is unused, and enforce that property, so as to clarify programs and reduce opportunities for error.
Improve the maintainability of all code by identifying variables that must be declared (e.g., in
catch
clauses) but are not used.
Reduce ceremony when different
switch
cases have the same action, by
allowing multiple patterns to appear as alternatives in a single case.

Improve the readability of record patterns by eliding unnecessary nested type patterns.

[java_139] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Non-Goals

Sumber: ht

It is not a goal to allow unnamed fields or method parameters.
It is not a goal to alter the semantics of local variables in, e.g., definite assignment analysis
.

[java_140] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Motivation

Sumber: ht

Developers sometimes declare variables that they do not intend to use, whether as a matter of code style or because the language requires variable declarations in certain contexts. The intent of non-use is known at the time the code is written, but if it is not captured explicitly then later maintainers might accidentally use the variable, thereby violating the intent. If we could make it impossible to accidentally use such variables then code would be more informative, more readable, and less prone to error.

[java_141] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Unused variables

Sumber: ht

The need to declare a variable that is never used is especially common in code whose side-effect is more important than its result. For example, this code calculates
total
as the side effect of a loop, without using the loop variable
order
:
The prominence of the declaration of
order
is unfortunate, given that
order
is not used. The declaration can be shortened to
var order
, but there is no way to avoid giving this variable a name. The name itself can be shortened to, e.g.,
o
, but this syntactic trick does not communicate the intent that the variable never be used. In addition, static analysis tools typically complain about unused variables, even when the developer intends non-use and may not have a way to silence the warnings.
For another example where the side effect of an expression is more important than its result, the following code dequeues data but needs only two out of every three elements:
The third call to

remove()
has the desired side effect -- dequeuing an element -- regardless of whether its result is assigned to a variable, so the declaration of
z
could be elided. However, for maintainability, the author of this code may wish to consistently denote the result of
remove()
by declaring a
variable. They currently have two options, both unpleasant:
Do not declare the variable
z
, which leads to an asymmetry and possibly a static-analysis warning about ignoring a return value, or else
Declare a variable that is not used and possibly get a static-analysis warning about an unused variable.
Unused variables occur frequently in two other statements that focus on side effects:
The
try
-with-resources statement is always used for its side effect, namely the automatic closing of resources. In some cases a resource represents a context in which the code of the
try
block executes; the code does not use the context directly, so the name of the resource variable is irrelevant. For example, assuming a
ScopedContext
resource that is
AutoCloseable
, the following code acquires and automatically releases a context:
The name
acquiredContext
is just clutter, so it would be nice to elide it.
Exceptions are the ultimate side effect, and handling one often gives rise to an unused variable. For example, most developers have written
catch
blocks of this form, where the exception parameter
ex
is unused:
Even code without side effects must sometimes declare unused variables. For example:
This code generates a map which maps each key to the same placeholder value. Since the lambda parameter
v
is not used, its name is irrelevant.
In all these scenarios, where variables are unused and their names are irrelevant, it would be better if we could simply declare variables with no name. This would free maintainers from having to understand irrelevant names, and would avoid false positives on non-use from static analysis tools.

The kinds of variables that can reasonably be declared with no name are those which have no visibility outside a method: local variables, exception parameters, and lambda parameters, as shown above. These kinds of variables can be renamed or made unnamed without external impact. In contrast, fields -- even if they are
private
-- communicate the state of an object across methods, and unnamed state is neither helpful nor maintainable.

Contoh Kode / Perintah CLI:

```
static int count(Iterable<Order> orders) {
    int total = 0;
    for (Order order : orders)    // order is unused
        total++;
    return total;
}

// ----------------------------------------

Queue<Integer> q = ... // x1, y1, z1, x2, y2, z2 ..
while (q.size() >= 3) {
   int x = q.remove();
   int y = q.remove();
   int z = q.remove();            // z is unused
    ... new Point(x, y) ...
}

// ----------------------------------------

try (var acquiredContext = ScopedContext.acquire()) {
    ... acquiredContext not used ...
}

// ----------------------------------------

String s = ...;
try {
    int i = Integer.parseInt(s);
    ... i ...
} catch (NumberFormatException ex) {
    System.out.println("Bad number: " + s);
}

// ----------------------------------------

...stream.collect(Collectors.toMap(String::toUpperCase,
                                   v -> "NODATA"));
```

[java_142] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Unused pattern variables

Sumber: ht

Local variables can also be declared by type patterns -- such local variables are known as
pattern variables
-- and so type patterns can also declare variables that are unused. Consider the following code, which uses type patterns in the case
labels of a

switch
statement that switches over an instance of a
sealed
class
Ball
:
The cases of the
switch
examine the type of the
Ball
using type patterns,
but the pattern variables
red
,
blue
, and
green
are not used on the
right-hand sides of the
case
clauses. This code would be clearer if we could
elide these variable names.
Now suppose that we define a record class
Box
which can hold any type of
Ball
, but might also hold the
null
value:
The nested type patterns still declare pattern variables that are not
used. Since this
switch
is more involved than the previous one, eliding the
names of the unused variables in the nested type patterns would even further
improve readability.

Contoh Kode / Perintah CLI:

```
sealed abstract class Ball permits RedBall, BlueBall, GreenBall { }
final  class RedBall   extends Ball { }
final  class BlueBall  extends Ball { }
final  class GreenBall extends Ball { }

Ball ball = ...
switch (ball) {
    case RedBall   red   -> process(ball);
    case BlueBall  blue  -> process(ball);
    case GreenBall green -> stopProcessing();
}

// ----------------------------------------

record Box<T extends Ball>(T content) { }

Box<? extends Ball> box = ...
switch (box) {
    case Box(RedBall   red)     -> processBox(box);
```

```
    case Box(BlueBall  blue)    -> processBox(box);
    case Box(GreenBall green)   -> stopProcessing();
    case Box(var       itsNull) -> pickAnotherBox();
}
```

[java_143] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Unused nested patterns

Sumber: ht

We can nest records within records, leading to situations in which the shape of
a data structure is as important as the data items within it. For example:
In this code, one part of the program creates a
ColoredPoint
instance while
another part uses a pattern
instanceof
to test whether a variable is a
ColoredPoint
and, if so, extract its two component values.
Record patterns such as
ColoredPoint(Point p, Color c)
are pleasingly
descriptive, but it is common for programs to use only some of the component
values for further processing. For example, the code above uses only
p
in the
if
block, not
c
. It is laborious to write out type patterns for all the
components of a record class every time we do such pattern matching.
Furthermore, it is not visually clear that the entire
Color
component is
irrelevant; this makes the condition in the
if
statement harder to read, too.
This is especially evident when record patterns are nested to extract data
within components, as in:
We could shrink the
Color
component visually by replacing
Color
with
var
,
i.e.,
ColoredPoint(Point(int x, int y), var c)
, but it would be better to omit
unnecessary components altogether. This would both simplify the task of writing
record patterns and improve readability, by removing clutter from the code.

Contoh Kode / Perintah CLI:

```
record Point(int x, int y) { }
```

```
enum Color { RED, GREEN, BLUE }
record ColoredPoint(Point p, Color c) { }

... new ColoredPoint(new Point(3,4), Color.GREEN) ...

if (r instanceof ColoredPoint(Point p, Color c)) {
    ... p.x() ... p.y() ...
}

// ----------------------------------------

if (r instanceof ColoredPoint(Point(int x, int y), Color c)) {
    ... x ... y ...
}
```

[java_144] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Description

Sumber: ht

An
unnamed variable
is declared by using an underscore character,
_
(U+005F), to stand in for the name of the local variable in a local variable declaration statement, or an exception parameter in a
catch
clause, or a
lambda parameter in a lambda expression.
An
unnamed pattern variable
is declared by using an underscore character to stand in for the pattern variable in a type pattern.
The
unnamed pattern
is denoted entirely by an underscore character. It allows both the type and name of a record component to be elided in pattern matching.
A single underscore character is the lightest reasonable syntax for signifying the absence of a name. It is commonly used in other languages, such as Scala and Python, for this purpose. A single underscore was, originally, a valid identifier in Java 1.0, but we later reclaimed it for unnamed variables and patterns: We started issuing compile-time warnings when underscore was used as an identifier in Java 8 (2014), and we removed such identifiers from the language specification, thereby turning those warnings into errors, in Java 9 (2017,
JEP 213
).
The ability to use underscore in
identifiers
of length two or more is unchanged, since underscore remains a Java letter and a Java letter-or-digit. For example, identifiers such as
_age
and
MAX_AGE
and

__
(two underscores) continue to be legal.
The ability to use underscore as a
digit separator
is also unchanged. For example, numeric literals such as
123_456_789
and
0b1010_0101
continue to be legal.

[java_145] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Unnamed variables

Sumber: ht

The following constructs can declare an
unnamed variable
by using an
underscore (
_
) in place of an identifier:
A local variable declaration statement in a block (JLS Section 14.4.2),
The resource specification of a
try
-with-resources statement (JLS Section 14.20.3),
The header of a basic
for
loop (JLS Section 14.14.1),
The header of an enhanced
for
loop (JLS Section 14.14.2),
An exception parameter of a
catch
block (JLS Section 14.20), and
A formal parameter of a lambda expression (JLS Section 15.27.1).
Declaring an unnamed variable does not place a name in scope, so the variable
cannot be written or read after it is initialized. An initializer must be
provided for an unnamed variable declared in a local variable declaration
statement or in the resource specification of a
try
-with-resources statement.
An unnamed variable never shadows any other variable, since it has no name, so
multiple unnamed variables can be declared in the same block.
Here are the examples given above, rewritten to use unnamed variables.
An enhanced
for
loop with side effects:
The initialization of a simple
for
loop can also declare unnamed local
variables:
An assignment statement, where the result of the expression on the right-hand
side is not needed:

If the program needed to process only the
x1
,
x2
, etc., coordinates then
unnamed variables could be used in multiple assignment statements:
A
catch
block:
Unnamed variables can be used in multiple
catch
blocks:
In
try
-with-resources:
A lambda whose parameter is irrelevant:

Contoh Kode / Perintah CLI:

```
static int count(Iterable<Order> orders) {
    int total = 0;
    for (Order _ : orders)    // Unnamed variable
        total++;
    return total;
}

// ----------------------------------------

for (int i = 0, _ = sideEffect(); i < 10; i++) { ... i ... }

// ----------------------------------------

Queue<Integer> q = ... // x1, y1, z1, x2, y2, z2, ...
while (q.size() >= 3) {
   var x = q.remove();
   var y = q.remove();
   var _ = q.remove();        // Unnamed variable
   ... new Point(x, y) ...
}

// ----------------------------------------

while (q.size() >= 3) {
    var x = q.remove();
    var _ = q.remove();       // Unnamed variable
    var _ = q.remove();       // Unnamed variable
    ... new Point(x, 0) ...
}

// ----------------------------------------

String s = ...
try {
    int i = Integer.parseInt(s);
    ... i ...
} catch (NumberFormatException _) {        // Unnamed variable
    System.out.println("Bad number: " + s);
}

// ----------------------------------------
```

```
try { ... }
catch (Exception _) { ... }               // Unnamed variable
catch (Throwable _) { ... }               // Unnamed variable

// ----------------------------------------

try (var _ = ScopedContext.acquire()) {    // Unnamed variable
    ... no use of acquired resource ...
}

// ----------------------------------------

...stream.collect(Collectors.toMap(String::toUpperCase,
                                   _ -> "NODATA"))    // Unnamed variable
```

[java_146] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Unnamed pattern variables

Sumber: ht

A type pattern declares a pattern variable, which is
unnamed
when denoted by
an underscore (
_
). An unnamed pattern variable can appear in a top level type
pattern or be nested in a record pattern. For example, the
Ball
example can
now be written:
and the
Box
and
Ball
example:
A type pattern with an unnamed pattern variable is called an
unnamed type
pattern
. By allowing us to elide names, unnamed type patterns make run-time
data exploration based on type patterns visually clearer, both in
switch
blocks and with the
instanceof
operator.

Contoh Kode / Perintah CLI:

```
switch (ball) {
    case RedBall _   -> process(ball);          // Unnamed pattern variable
    case BlueBall _  -> process(ball);          // Unnamed pattern variable
    case GreenBall _ -> stopProcessing();       // Unnamed pattern variable
}

// ----------------------------------------

switch (box) {
    case Box(RedBall _)   -> processBox(box);   // Unnamed pattern variable
    case Box(BlueBall _)  -> processBox(box);   // Unnamed pattern variable
```

```
    case Box(GreenBall _) -> stopProcessing();  // Unnamed pattern variable
    case Box(var _)       -> pickAnotherBox();  // Unnamed pattern variable
}
```

[java_147] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Multiple patterns in case labels

Sumber: ht

Currently,
case
labels are restricted to contain at most one pattern. With the
introduction of unnamed pattern variables and unnamed patterns, it is more
likely that we will have within a single
switch
block several
case
clauses
with different patterns but the same right-hand side. For example, in the
Box
and
Ball
example the first two clauses have the same right-hand side but
different patterns:
We could simplify matters by allowing the first two patterns to appear in the
same
case
label:
We therefore revise the grammar for switch labels (JLS Section 14.11.1) to
and define the semantics of a
case
label with multiple patterns as matching a
value if the value matches any of the patterns.
If a
case
label has multiple patterns then it is a compile-time error for any
of the patterns to declare any pattern variables.
A
case
label with multiple
case
patterns can have a
guard
. The guard governs the
case
as a whole, rather than the individual patterns. For example, assuming
that there is an
int
variable
x
, the first
case
of the previous example

could be further constrained:
Guards are properties of
case
labels, not individual patterns within a
case
label, so writing more than one guard is prohibited:

Contoh Kode / Perintah CLI:

```
switch (box) {
    case Box(RedBall _)   -> processBox(box);
    case Box(BlueBall _)  -> processBox(box);
    case Box(GreenBall _) -> stopProcessing();
    case Box(var _)       -> pickAnotherBox();
}

// ----------------------------------------

switch (box) {
    case Box(RedBall _), Box(BlueBall _) -> processBox(box);
    case Box(GreenBall _)                -> stopProcessing();
    case Box(var _)                      -> pickAnotherBox();
}

// ----------------------------------------

SwitchLabel:
    case CaseConstant {, CaseConstant}
    case null [, default]
    case CasePattern {, CasePattern } [Guard]
    default

// ----------------------------------------

case Box(RedBall _), Box(BlueBall _) when x == 42 -> processBox(b);

// ----------------------------------------

case Box(RedBall _) when x == 0, Box(BlueBall _) when x == 42 -> processBox(b);
    // compile-time error
```

[java_148] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: The unnamed pattern

Sumber: ht

The unnamed pattern is an unconditional pattern that matches anything but declares and initializes nothing. It is equivalent to the unnamed type pattern
var _
. The unnamed pattern can be nested in a record pattern but cannot be used as a top-level pattern in, e.g., an
instanceof
expression or a
case
label.
Consequently, the earlier example can omit the type pattern for the
Color
component entirely:
Likewise, we can extract the
Color

component value while eliding the record
pattern for the
Point
component:
In deeply nested positions, using the unnamed pattern improves the readability
of code that does complex data extraction. For example:
This code extracts the
x
coordinate of the nested
Point
while making it
clear that the
y
and
Color
component values are not extracted.
Revisiting the
Box
and
Ball
example, we can further simplify its final case
label by using the unnamed pattern instead of
var _
:

Contoh Kode / Perintah CLI:

```
if (r instanceof ColoredPoint(Point(int x, int y), _)) { ... x ... y ... }

// ----------------------------------------

if (r instanceof ColoredPoint(_, Color c)) { ... c ... }

// ----------------------------------------

if (r instanceof ColoredPoint(Point(int x, _), _)) { ... x ... }

// ----------------------------------------

switch (box) {
    case Box(RedBall _), Box(BlueBall _) -> processBox(box);
    case Box(GreenBall _)                -> stopProcessing();
    case Box(_)                          -> pickAnotherBox();
}
```

[java_149] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Risks and Assumptions

Sumber: ht

We assume that little if any actively-maintained code uses underscore as a
variable name. Developers migrating from Java 7 to Java 22
without having seen the warnings issued in Java 8 or the errors issued
since Java 9 could be surprised. They face the risk of dealing with
compile-time errors when reading or writing variables named

_
and when

declaring any other kind of element (class, field, etc.) with the name

_
.
We expect developers of static analysis tools to understand the new role of underscore for unnamed variables and avoid flagging the non-use of such variables in modern code.

[java_150] Java - Pattern Matching - Unnamed Patterns

JEP 456: Unnamed Variables & Patterns: Alternatives

Sumber: ht

It is possible to define an analogous concept of
unnamed method parameters
.
However, this has some subtle interactions with the specification (e.g., what does it mean to override a method with unnamed parameters?) and tooling (e.g., how do you write JavaDoc for unnamed parameters?). This may be the subject of a future JEP.
JEP 302
(
Lambda Leftovers
) examined the
issue of unused lambda parameters and identified the role of underscore to denote them, but also covered many other issues which were handled better in other ways. This JEP addresses the use of unused lambda parameters explored in JEP 302 but does not address the other issues explored there.
Installing
Contributing
Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds
Mailing lists
Wiki
*
IRC
Mastodon
Bluesky
Bylaws
*
Census
Legal
*
AI
Workshop
JEP Process
Source code
GitHub
Mercurial
Tools
Git
jtreg harness

Groups
(overview
,
archive
)
Adoption
Build
Client Libraries
Compatibility & Specification Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
Members
Networking
Porters
Quality
Security
Serviceability
Vulnerability
Web
Projects
(
overview
,
archive
)
Amber
Babylon
Brisbane
CRaC
Code Tools
Coin
Common VM Interface
Detroit
Developers' Guide
Duke
IcedTea
JDK 8 Updates
JDK 9
JDK
(...,
26
,
27
,
28
)
JDK Updates

JMC
Jigsaw
Lanai
Leyden
Lilliput
Loom
Memory Model Update
Multi-Language VM
Nashorn
New I/O
OpenJFX
Panama
Port: AArch32
Port: AArch64
Port: BSD
Port: Haiku
Port: MIPS
Port: Mobile
Port: PowerPC/AIX
Port: RISC-V
Port: s390x
SCTP
Shenandoah
Skara
Sumatra
Tsan
Valhalla
Wakefield
Zero
ZGC
(c) 2026 Oracle Corporation and/or its affiliates
Terms of Use
*

License:
GPLv2
*
Privacy
*
Trademarks

[java_151] Java - Pattern Matching - Exhaustiveness

Preparing for Change: Safe Switching over Sealed APIs: Overview

Sumber: ht

Preparing for Change: Safe Switching over Sealed APIs
Angelos Bimpoudis, Alex Buckley and Brian Goetz {.author}
2026-08-11 {.date}

[java_152] Java - Pattern Matching - Exhaustiveness

Preparing for Change: Safe Switching over Sealed APIs: Summary

Sumber: ht

An exhaustive `switch` on a `sealed` type covers the permitted subtypes the
compiler knows about; it does not cover future additions, which can cause
`MatchException` at run time. API authors should document the expected
evolution of `sealed` types so that client code can choose how to handle future
additions. It is almost never necessary to use `default` when switching over a
`sealed` type -- and often not the best option.

[java_153] Java - Pattern Matching - Exhaustiveness

Preparing for Change: Safe Switching over Sealed APIs: `switch` expressions: Exhaustive by design

Sumber: ht

A `switch` expression must be _exhaustive_, meaning that the `switch` must
produce a result for every possible non-null value of the selector.
For primitive types other than `boolean`, `default` is the only label that
covers all the unlisted values and makes the `switch` exhaustive:
In contrast, when the selector is a `sealed` type, the `switch` can be
made exhaustive by listing the permitted subtypes of the `sealed`
type, `case` by `case`. No `default` is needed. For example:
However, an exhaustive `switch` is not guaranteed to remain so, because it is
possible to evolve `sealed` types over time to acquire new permitted subtypes.
This means that switches over a `sealed` type that were exhaustive when they were written, may no
longer be exhaustive, because they no longer cover all possible values. The
language provides a variety of static and dynamic tools for confidently
navigating these transitions.
For example, suppose `Fruit` is recompiled to add a `Pear` subtype:
If the `switch` above is also recompiled, the compiler realizes that `Pear` values
are not covered by a `case`. Because the `switch` is no longer exhaustive, it no
longer compiles. While this might at first seem "bad", this is actually
very good! When the `switch` was written, the developer intent was "I have
covered all the fruits." The compiler was able to validate the developer
intent, ensuring that all fruits are indeed covered. And when the world of
fruits changes in a way that invalidates the original developer intent, the
developer is notified of this change so that the `switch` can be adjusted to meet
the new reality.
Adding `case Pear p` brings the code back into consistency with its intent: covering all the fruits.
In the meantime, it is possible (if the `switch` is not recompiled) that a
`Pear` value matches no `case` at run time, causing the `switch` to throw
`MatchException`. Again, while this may at first seem bad, this is also good!
If the program continues to encounter only fruits it is familiar with,
everything continues to work. But if a fruit appears that it is unprepared to
deal with, we get feedback at run time (in the form of an exception) again telling
us that our program's assumptions -- which were valid when it was written, but
which have been undermined by separately-compiled changes -- are no longer valid.

Contoh Kode / Perintah CLI:

```
In contrast, when the selector is a `sealed` type, the `switch` can be
made exhaustive by listing the permitted subtypes of the `sealed`
type, `case` by `case`. No `default` is needed. For example:
```

```
// ----------------------------------------

However, an exhaustive `switch` is not guaranteed to remain so, because it is
possible to evolve `sealed` types over time to acquire new permitted subtypes.
This means that switches over a `sealed` type that were exhaustive when they were written, may no
longer be exhaustive, because they no longer cover all possible values. The
language provides a variety of static and dynamic tools for confidently
navigating these transitions.

For example, suppose `Fruit` is recompiled to add a `Pear` subtype:
```

[java_154] Java - Pattern Matching - Exhaustiveness

Preparing for Change: Safe Switching over Sealed APIs: By default, avoid `default`

Sumber: ht

It might be tempting to try to "future proof" switches by including a catch-all
`default` label. However, in most cases, this is the wrong move, because it
sweeps errors, both present and future, under the rug.
If the developer intent is that the switch is exhaustive, adding a `default`
label hides this intent, and thus deprives you of having the compiler (and your
reviewers) double-checking your work and letting you know if you've made a bad
assumption. A `default`-free switch captures this intent so that it can
be verified later, when the world might have changed. In the case of our
`Fruit` example above, had we included a `default` label, we would have been
deprived of the notification that our intent has been invalidated and of the
opportunity to update our program in a timely manner.
In addition, a `default` may interfere with the run-time detection of unexpected values,
which may allow a program to continue but cause it to produce an erroneous result.
Finally, a `default` is not a true catch-all: it does not handle a `null` selector.
To avoid a `NullPointerException` from a `null` selector, you must either write
an explicit `case null` or, preferably, use `Objects.requireNonNull`
to reject `null` before the `switch`.
In almost all switches, it is better to write `case` labels that cover a `sealed`
type than to use a `default` as a catch-all; we will outline the few situations
where this is appropriate later in this document.

[java_155] Java - Pattern Matching - Exhaustiveness

Preparing for Change: Safe Switching over Sealed APIs: Sealed types can evolve, and often will

Sumber: ht

A `sealed` type is an API that describes a hierarchy of permitted subtypes _at a
single point in time_. As with any API in Java, it can change over time. It is
important that adding new permitted subtypes later be a permissible and compatible evolution.
Adding a new type to an API is usually harmless, but
adding a new permitted subtype to a `sealed` type could cause pre-existing
`switch` expressions to become non-exhaustive. While at first this seems
disturbing -- such switches may throw `MatchException` at run time and will
encounter compilation errors when recompiled -- this is how we balance safety
with resilience. It is unreasonable to expect that when a new `Fruit` is
discovered, that all fruit-related code can change instantly. But also, we
don't want to give up the considerable benefits that sealing offers in capturing
design intent and enhancing compile-time and run-time type safety.
There are two primary reasons why one might declare a type `sealed`:

- To signal to clients that this type models a closed-end domain, such as when `Fruit` is defined to be an `Apple` or an `Orange` but nothing else.
- To control all the _implementations_ of the type, but not model a specific closed-end domain.

It is helpful when authors of `sealed` types explicitly document the reason for sealing, and how they expect the type to evolve in the future. A `sealed` type that models a `Boolean` domain is unlikely to acquire subtypes that correspond to anything other than `True` and `False`, because the domain is extremely stable; on the other hand, a `sealed` type that models the different kinds of AST nodes describing a program is highly likely to change over time, either because the language being modeled has acquired new features, or because of changes in how programs are modeled.

When a `sealed` type is encapsulated within a single maintenance domain, it is reasonable to expect that changes to the `sealed` type will immediately be reflected at all the uses; when a `sealed` type is published across maintenance domains, we should expect there will be some time lag between the update of the `sealed` type and properly updating its clients. We should program with idioms that do not unnecessarily "sweep changes under the rug" to avoid silently ignoring feedback that might portend actionable changes.

[java_156] Java - Pattern Matching - Exhaustiveness

Preparing for Change: Safe Switching over Sealed APIs: Guidelines for switches over sealed types

Sumber: ht

Switches over `sealed` types should heed the following guidelines:

1. A `default` label is almost never the right answer, for reasons discussed already.
3. Where practical, the best approach is to handle each permitted subtype with its own `case`.
5. If it is impractical to handle each permitted subtype with its own `case`, we should use the narrowest possible type pattern to render the switch exhaustive. We call this a match-all `case`.

In the `switch` above with a `Fruit` selector type, the match-all `case` would be `case Fruit other`. It is not recommended to use a match-all `case` with a broader type pattern than the selector type, e.g., `case Object other`.

There are several reasons why it may not be practical to handle each permitted subtype with its own `case`:

- Some of the permitted subtypes are inaccessible to the client. This can arise when the author of the `sealed` type used sealing primarily to achieve control over implementations, not to model a closed-end domain of well-known alternatives.

_(The idiom of an inaccessible subtype is most likely to occur in widely distributed APIs such as the JDK, e.g., the Attribute interface of the Class-File API.
The javadoc for `Attribute` omits inaccessible subtypes, triggering the "(not exhaustive)" note.)_

- The `sealed` type has many permitted subtypes, and the client wants to handle only a small subset of these. In this situation, listing all the permitted subtypes would likely be low-value ceremony, while picking off a small number of subtypes and ignoring the rest is analogous to an unbalanced `if` statement.

_(For example, given `sealed interface A permits A1, A2, A3, A4, ... A100 {}`,

a match-all `case A other` is reasonable after handling `case A1 a1` and `case A2 a2`.)_
Further, even if all subtypes are handled by their own `case`, it may sometimes be
necessary to use a match-all `case` anyway, to provide custom error handling in the
event of unexpected change. However, we should be aware that this is giving up
much of the compile-time type safety that `sealed` types afford.
There is one unusual situation where a match-all `case` is needed to make the `switch` exhaustive:
when the `sealed` type is a concrete class and therefore can be instantiated.
There are some situations where a narrower type pattern than the selector type may
render the switch exhaustive. This happens when a `sealed` type has multiple branches:
While it is possible to "totalize" a `Food` selector with a match-all `case Food other`,
it is preferable to be more restrained and totalize each branch explicitly:
When a `sealed` type has many permitted subtypes, it is again preferable to totalize on a per-branch basis.
For a `switch` with selector type `AB`, it may be practical to cover the three `B` options but not the 100 `A` options.
It is best to render the `switch` exhaustive by totalizing with `case A other` rather than `case AB other`.
This preserves the ability for exhaustiveness checking of the `B` options,
whereas using the broader type `AB` would give up all exhaustiveness checking.

Contoh Kode / Perintah CLI:

```
There are some situations where a narrower type pattern than the selector type may
render the switch exhaustive. This happens when a `sealed` type has multiple branches:

// ----------------------------------------

While it is possible to "totalize" a `Food` selector with a match-all `case Food other`,
it is preferable to be more restrained and totalize each branch explicitly:

// ----------------------------------------

When a `sealed` type has many permitted subtypes, it is again preferable to totalize on a per-branch
basis.
```

[java_157] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Why Virtual Threads?

Sumber: ht

When Java 1.0 was released in 1995, its API had about a hundred classes, among them `java.lang.Thread`. Java was the first mainstream programming language that directly supported concurrent programming.
Since Java 1.2, each Java thread runs on a *platform thread* supplied by the underlying operating system. (Up to Java 1.1, on some platforms, all Java threads were executed by a single platform thread.)
Platform threads have nontrivial costs. They require a few thousand CPU instructions to start, and they consume a few megabytes of memory. Server applications can serve so many concurrent requests that it becomes infeasible to have each of them execute on a separate platform thread. In a typical server application, these requests spend much of their time *blocking*, waiting for a result from a database or another service.
The classic remedy for increasing throughput is a non-blocking API. Instead of waiting for a result, the programmer indicates which method should be called when the result has become available, and perhaps another method that is called in case of failure. This gets unpleasant quickly, as the callbacks nest ever more deeply.
JEP 425 introduced *virtual threads* in Java 19. Many virtual threads run on a platform thread. Whenever a virtual thread blocks, it is *unmounted*, and the platform thread runs another virtual thread. (The name "virtual thread" is supposed to be reminiscent of virtual memory that is mapped to actual RAM.) Virtual threads became a preview feature in Java 20 (JEP 436) and are final in Java 21.
With virtual threads, blocking is cheap. When a result is not immediately available, you simply block in a virtual thread. You use familiar programming structures -- branches, loops, try blocks -- instead of a pipeline of callbacks.
Virtual threads are useful when the number of concurrent tasks is large, and the tasks mostly block on network I/O. They offer no benefit for CPU-intensive tasks. For such tasks, consider parallel streams or recursive fork-join tasks.

[java_158] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Creating Virtual Threads

Sumber: ht

The factory method `Executors.newVirtualThreadPerTaskExecutor()` yields an `ExecutorService` that runs each task in a separate virtual thread. For example:
By the way, the code uses `LockSupport.parkNanos` instead of `Thread.sleep` so that we don't have to catch the pesky `InterruptedException`.
Perhaps you are using a lower-level API that asks for a thread factory. To obtain a factory for virtual threads, use the new `Thread.Builder` class:
Now, calling `factory.newThread(myRunnable)` creates a new (unstarted) virtual thread. The `name` method configures the builder to set thread names `request-1`, `request-2`, and so on.
You can also use a builder to create a single virtual thread:
Alternatively, if you want to start the thread right away:
Finally, for a quick demo, there is a convenience method:
Note that only the first approach, with an executor service, works with result-bearing tasks (callables).

Contoh Kode / Perintah CLI:

```
import java.util.concurrent.*;

public class VirtualThreadDemo {
   public static void main(String[] args) {
     final int NTASKS = 100;
     ExecutorService service = Executors.newVirtualThreadPerTaskExecutor();
      for (int i = 0; i < NTASKS; i++) {
         service.submit(() -> {
            long id = Thread.currentThread().threadId();
            LockSupport.parkNanos(1_000_000_000);
            System.out.println(id);
         });
      }
      service.close();
   }
}

// ----------------------------------------

Thread.Builder builder = Thread.ofVirtual().name("request-", 1);
ThreadFactory factory = builder.factory();

// ----------------------------------------

Thread t = builder.unstarted(myRunnable);

// ----------------------------------------

Thread t = builder.started(myRunnable);

// ----------------------------------------

Thread t = Thread.startVirtualThread(myRunnable);
```

[java_159] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Thread API Changes

Sumber: ht

After a series of experiments with different APIs, the designers of Java virtual threads decided to simply reuse the familiar `Thread` API. A virtual thread is an instance of `Thread`. Cancellation works the same way as for platform threads, by calling `interrupt`. As always, the thread code must check the "interrupted" flag or call a method that does. (Most blocking methods do.)
There are a few differences. In particular, all virtual threads:
* Are in a single thread group
* Have priority `NORM_PRIORITY`
* Are daemon threads
There is no API for constructing a virtual thread with another thread group. Trying to call `setPriority` or `setDaemon` on a virtual thread has no effect.
The static `Thread::getAllStackTraces` method returns a map of stack traces of all *platform* threads. Virtual threads are not included.
A new `Thread::isVirtual` instance method tells whether a thread is virtual.
Note that there is no way to find the platform thread on which a virtual thread executes.
Java 19 has a couple of changes to the `Thread` API that have nothing to do with virtual threads:
* There are now instance methods `join(Duration)` and `sleep(Duration)`.
* The non-final `getId` method is deprecated since someone might override it to return something other than the thread ID. Call the final `threadId` method instead.
As of Java 20, the `stop`, `suspend`, and `resume` methods throw an `UnsupportedOperationException` for both platform and virtual threads. These methods have been deprecated since Java 1.2 and deprecated for removal since Java 18.

[java_160] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Capturing Task Results

Sumber: ht

You often want to combine the results of multiple concurrent tasks:
Before virtual threads, you might have felt bad about the blocking `get` calls. But now blocking is cheap. Here is a sample program with a more concrete example:
If you have a list of tasks with the same result type, you can use the `invokeAll` method and then call `get` on each `Future`:
Again, a more concrete sample program:

Contoh Kode / Perintah CLI:

```
Future<T1> f1 = service.submit(callable1);
Future<T2> f2 = service.submit(callable2);
result = combine(f1.get(), f2.get());

// ----------------------------------------

import java.util.concurrent.*;
import java.net.*;
import java.net.http.*;

public class VirtualThreadDemo {
   public static void main(String[] args) throws InterruptedException, ExecutionException {
      ExecutorService service = Executors.newVirtualThreadPerTaskExecutor();
      Future<String> f1 = service.submit(() -> get("https://horstmann.com/random/adjective"));
      Future<String> f2 = service.submit(() -> get("https://horstmann.com/random/noun"));
      String result = f1.get() + " " + f2.get();
      System.out.println(result);
      service.close();
   }
```

```
   private static HttpClient client = HttpClient.newHttpClient();

   public static String get(String url) {
      try {
         var request = HttpRequest.newBuilder().uri(new URI(url)).GET().build();
         return client.send(request, HttpResponse.BodyHandlers.ofString()).body();
      } catch (Exception ex) {
         var rex = new RuntimeException();
         rex.initCause(ex);
         throw rex;
      }
   }
}

// ----------------------------------------

List<Callable<T>> callables = ...;
List<T> results = new ArrayList<>();
for (Future<T> f : service.invokeAll(callables))
  results.add(f.get());

// ----------------------------------------

import java.util.*;
import java.util.concurrent.*;
import java.net.*;
import java.net.http.*;

public class VirtualThreadDemo {
   public static void main(String[] args) throws InterruptedException, ExecutionException {
      ExecutorService service = Executors.newVirtualThreadPerTaskExecutor();
      List<Callable<String>> callables = new ArrayList<>();
      final int ADJECTIVES = 4;
      for (int i = 1; i <= ADJECTIVES; i++)
         callables.add(() -> get("https://horstmann.com/random/adjective"));
      callables.add(() -> get("https://horstmann.com/random/noun"));
      List<String> results = new ArrayList<>();
      for (Future<String> f : service.invokeAll(callables))
         results.add(f.get());
      System.out.println(String.join(" ", results));
      service.close();
   }

   private static HttpClient client = HttpClient.newHttpClient();

   public static String get(String url) {
      try {
         var request = HttpRequest.newBuilder().uri(new URI(url)).GET().build();
         return client.send(request, HttpResponse.BodyHandlers.ofString()).body();
      } catch (Exception ex) {
         var rex = new RuntimeException();
         rex.initCause(ex);
         throw rex;
      }
   }
}
```

[java_161] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Rate Limiting

Sumber: ht

Virtual threads improve application throughput since you can have many more concurrent tasks than with platform threads. That can put pressure on the services that the tasks invoke. For example, a web service may not tolerate huge numbers of concurrent requests.
With platform threads, an easy (if crude) tuning factor is the size of the thread pool for those tasks. But you should not pool virtual threads. Scheduling tasks on virtual threads that are then scheduled on platform threads is clearly inefficient. And what is the upside? To limit the number virtual threads to the smallish number of concurrent requests that your service tolerates? Then why are you using virtual threads in the first place?
With virtual threads, you should use alternative mechanisms for controlling access to limited resources. Instead of an overall limit on concurrent tasks, protect each resource in an appropriate way. For database connections, the connection pool may already do the right thing. When accessing a web service, you know your service, and can provide appropriate rate limiting.
As an example, on my personal web site, I provide demo services for producing random items. If a large number of requests comes at an instant from the same IP address, the hosting company blacklists the IP address.
The following sample program shows rate limiting with a simple semaphore that allows a small number of concurrent requests. When the maximum is exceeded, the `acquire` method blocks, but that is ok. With virtual threads, blocking is cheap.

Contoh Kode / Perintah CLI:

```
import java.util.*;
import java.util.concurrent.*;
import java.net.*;
import java.net.http.*;

public class RateLimitDemo {
   public static void main(String[] args) throws InterruptedException, ExecutionException {
      ExecutorService service = Executors.newVirtualThreadPerTaskExecutor();
      List<Future<String>> futures = new ArrayList<>();
      final int TASKS = 250;
      for (int i = 1; i <= TASKS; i++)
         futures.add(service.submit(() -> get("https://horstmann.com/random/word")));
      for (Future<String> f : futures)
         System.out.print(f.get() + " ");
      System.out.println();
      service.close();
   }

   private static HttpClient client = HttpClient.newHttpClient();

   private static final Semaphore SEMAPHORE = new Semaphore(20);

   public static String get(String url) {
      try {
         var request = HttpRequest.newBuilder().uri(new URI(url)).GET().build();
         SEMAPHORE.acquire();
         try {
            Thread.sleep(100);
            return client.send(request, HttpResponse.BodyHandlers.ofString()).body();
         } finally {
            SEMAPHORE.release();
         }
      } catch (Exception ex) {
         ex.printStackTrace();
         var rex = new RuntimeException();
         rex.initCause(ex);
         throw rex;
```

```
      }
   }
}
```

[java_162] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Pinning

Sumber: ht

The virtual thread scheduler mounts virtual threads onto carrier threads. By default, there are as many carrier threads as there are CPU cores. You can tune that count with the `jdk.virtualThreadScheduler.parallelism` VM option.
When a virtual thread executes a blocking operation, it is supposed to be unmounted from its its carrier thread, which can then execute a different virtual thread. However, there are situations where this unmounting is not possible. In some situations, the virtual thread scheduler will compensate by starting another carrier thread. For example, in JDK 21, this happens for many file I/O operations, and when calling `Object.wait`. You can control the maximum number of carrier threads with the `jdk.virtualThreadScheduler.maxPoolSize` VM option.
A thread is called *pinned* in either of the two following situations:
1. When executing a `synchronized` method or block
2. When calling a native method or foreign function
Being pinned is not bad in itself. But when a pinned thread blocks, it cannot be unmounted. The carrier thread is blocked, and, in Java 21, no additional carrier thread is started. That leaves fewer carrier threads for running virtual threads.
Pinning is harmless if `synchronized` is used to avoid a race condition in an in-memory operation. However, if there are blocking calls, it would be best to replace `synchronized` with a `ReentrantLock`. This is of course only an option if you have control over the source code.
To find out whether pinned threads are blocked, start the JVM with one of the options
You get a stack trace that shows when a pinned thread blocks:
Note that you get only one warning per pinning location!
Alternatively, record with Java Flight Recorder, view with your favorite mission control viewer, and look for `VirtualThreadPinned` and `VirtualThreadSubmitFailed` events.
The JVM will eventually be implemented so that `synchronized` methods or blocks no longer lead to pinning. Then you only need to worry about pinning for native code.
The following sample program shows pinning in action. We launch a number of virtual threads that sleep in a synchronized method, blocking their carrier threads. A number of virtual threads are added that do no work at all. But they can't be scheduled because the carrier thread pool has been completely exhausted. Note that the problem goes away when you
* use a `ReentrantLock`
* don't use virtual threads

Contoh Kode / Perintah CLI:

```
-Djdk.tracePinnedThreads=short
-Djdk.tracePinnedThreads=full

// ----------------------------------------

...
org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:49) <== monitors:1
...

// ----------------------------------------

import java.util.concurrent.*;
import java.util.concurrent.locks.*;

public class PinningDemo {
   public static void main(String[] args) throws InterruptedException, ExecutionException {
      ExecutorService service =
```

```
         Executors.newVirtualThreadPerTaskExecutor();
         // Executors.newCachedThreadPool();

      final int TASKS = 20;
      long start = System.nanoTime();
      for (int i = 1; i <= TASKS; i++) {
         service.submit(() -> block());
         // service.submit(() -> rblock());
      }
      for (int i = 1; i <= TASKS; i++) {
         service.submit(() -> noblock());
      }
      service.close();
      long end = System.nanoTime();
      System.out.printf("%.2f%n", (end - start) * 1E-9);
   }

   public static synchronized void block() {
      System.out.println("Entering block " + Thread.currentThread());
      LockSupport.parkNanos(1_000_000_000);
      System.out.println("Exiting block " + Thread.currentThread());
   }
   private static Lock lock = new ReentrantLock();
   public static void rblock() {
      lock.lock();
      try {
         System.out.println("Entering rblock " + Thread.currentThread());
         LockSupport.parkNanos(1_000_000_000);
         System.out.println("Exiting rblock " + Thread.currentThread());
      } finally {
         lock.unlock();
      }
   }
   public static void noblock() {
      System.out.println("Entering noblock " + Thread.currentThread());
      LockSupport.parkNanos(1_000_000_000);
      System.out.println("Exiting noblock " + Thread.currentThread());
   }
}
```

[java_163] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Thread Locals

Sumber: ht

A *thread-local variable* is an object whose `get` and `set` methods access a value that depends on the current thread. Why would you want such a thing instead of using a global or local variable? The classic application is a service that is not threadsafe, such as `SimpleDateFormat`, or that would suffer from contention, such as a random number generator. Per-thread instances can perform better than a global instance that is protected by a lock.

Another common use for thread locals is to provide "implicit" context, such as a database connection, that is properly configured for each task. Instead of passing the context from one method to another, the task code simply reads the thread-local variable whenever it needs to access the database.

Thread locals can be a problem when migrating to virtual threads. There will likely be far more virtual threads than threads in a thread pool, and now you have many more thread-local instances. In such a situation, you should rethink your sharing strategy.

To locate uses of thread locals in your app, run with the VM flag `jdk.traceVirtualThreadLocals`. You will get a stack trace when a virtual thread mutates a thread-local variable.

[java_164] Java - Concurrency - Virtual Threads Tutorial

Virtual Threads: Conclusion

Sumber: ht

* Use virtual threads to increase throughput when you have many tasks that mostly block on network I/O
* The primary benefit is the familiar "synchronous" programming style, without callbacks
* Don't pool virtual threads; use other mechanisms for rate limiting
* Check for pinning and mitigate if necessary
* Minimize thread-local variables in virtual threads

[java_165] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Overview & Metadata

Sumber: ht

<noscript></noscript>
TABLE { border-collapse: collapse; padding: 0px; margin: 1em 0 1em 2em; }
TR:first-child TH, TR:first-child TD { padding-top: 0; }
TH, TD { padding: 0px; padding-top: .5ex; vertical-align: baseline; text-align: left; }
TD + TD, TH + TH { padding-left: 1em; }
TD:first-child, TH:first-child, TD.jep { text-align: right; }
TABLE.head TD:first-child { font-style: italic; padding-left: 2em; white-space: nowrap; }
PRE { padding-left: 2em; margin: 1ex 0; font-size: inherit; }
TABLE PRE { padding-left: 0; margin: 0; }
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { padding-left: .5em; }
TABLE.jeps TD:first-child,
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { font-size: smaller; }
TABLE.jeps TD.cl { font-size: smaller; padding-right: 0; text-align: right; }
TABLE.jeps TD.cm { font-size: smaller; padding-left: .1em; padding-right: .1em; }
TABLE.jeps TD.cr { font-size: smaller; padding-left: 0; }
TABLE.jeps TD.z { padding-left: 0; padding-right: 0; }
TABLE.head TD { padding-top: 0; }
JEP 444: Virtual Threads
Author
Ron Pressler & Alan Bateman
Owner
Alan Bateman
Type
Feature
Scope
SE
Status
Closed / Delivered
Release
21
Component
core-libs
Discussion
loom dash dev at openjdk dot org
Relates to
JEP 436: Virtual Threads (Second Preview)
JEP 491: Synchronize Virtual Threads without Pinning

Reviewed by
Alex Buckley
Endorsed by
Brian Goetz
Created
2023/03/06 18:00
Updated
2025/10/30 21:01
Issue
8303683

[java_166] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Summary

Sumber: ht

Introduce
virtual threads
to the Java Platform. Virtual threads are lightweight threads that dramatically reduce the effort of writing, maintaining, and observing high-throughput concurrent applications.

[java_167] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: History

Sumber: ht

Virtual threads were proposed as a preview feature by
JEP 425
and delivered in
JDK 19
. To allow time for feedback and to get more experience, they were proposed as a preview feature again by
JEP 436
and delivered in
JDK 20
. This JEP proposes to finalize virtual threads in JDK 21 with the following changes from JDK 20, informed by developer feedback:
Virtual threads now always support
thread-local variables
. It is no longer possible, as it was in the preview releases, to create virtual threads that cannot have thread-local variables. Guaranteed support for thread-local variables ensures that many more existing libraries can be used unchanged with virtual threads, and helps with the migration of task-oriented code to use virtual threads.
Virtual threads created directly with the
Thread.Builder
API (as opposed to those created through
Executors.newVirtualThreadPerTaskExecutor()
) are now also, by default, monitored throughout their lifetime and observable via the new thread dump described in the
Observing virtual threads
section.

[java_168] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Goals

Sumber: ht

Enable server applications written in the simple thread-per-request style to scale with near-optimal hardware utilization.
Enable existing code that uses the
java.lang.Thread
API to adopt virtual threads with minimal change.
Enable easy troubleshooting, debugging, and profiling of virtual threads with existing JDK tools.

[java_169] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Non-Goals

Sumber: ht

It is not a goal to remove the traditional implementation of threads, or to silently migrate existing applications to use virtual threads.
It is not a goal to change the basic concurrency model of Java.
It is not a goal to offer a new data parallelism construct in either the Java language or the Java libraries. The
Stream API
remains the preferred way to process large data sets in parallel.

[java_170] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Motivation

Sumber: ht

Java developers have relied on threads as the building block of concurrent server applications for nearly three decades. Every statement in every method is executed inside a thread and, since Java is multithreaded, multiple threads of execution happen at once. The thread is Java's
unit of concurrency:
a piece of sequential code that runs concurrently with -- and largely independently of -- other such units. Each thread provides a stack to store local variables and coordinate method calls, as well as context when things go wrong: Exceptions are thrown and caught by methods in the same thread, so developers can use a thread's stack trace to find out what happened. Threads are also a central concept for tools: Debuggers step through the statements in a thread's methods, and profilers visualize the behavior of multiple threads to help understand their performance.

[java_171] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: The thread-per-request style

Sumber: ht

Server applications generally handle concurrent user requests that are independent of each other, so it makes sense for an application to handle a request by dedicating a thread to that request for its entire duration. This
thread-per-request style
is easy to understand, easy to program, and easy to debug and profile because it uses the platform's unit of concurrency to represent the application's unit of concurrency.
The scalability of server applications is governed by
Little's Law
, which relates latency, concurrency, and throughput: For a given request-processing duration (i.e., latency), the number of requests an application handles at the same time (i.e., concurrency) must grow in proportion to the rate of arrival (i.e., throughput). For example, suppose an application with an average latency of 50ms achieves a throughput of 200 requests per second by processing 10 requests concurrently. In order for that application to scale to a throughput of 2000 requests per second, it will need to process 100 requests concurrently. If each request is handled in a thread for the request's duration then, for the application to keep up, the number of threads must grow as throughput grows.
Unfortunately, the number of available threads is limited because the JDK implements threads as wrappers around operating system (OS) threads. OS threads are costly, so we cannot have too many of them, which makes the

implementation ill-suited to the thread-per-request style. If each request consumes a thread, and thus an OS thread, for its duration, then the number of threads often becomes the limiting factor long before other resources, such as CPU or network connections, are exhausted. The JDK's current implementation of threads caps the application's throughput to a level well below what the hardware can support. This happens even when threads are pooled, since pooling helps avoid the high cost of starting a new thread but does not increase the total number of threads.

[java_172] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Improving scalability with the asynchronous style

Sumber: ht

Some developers wishing to utilize hardware to its fullest have given up the thread-per-request style in favor of a thread-sharing style. Instead of handling a request on one thread from start to finish, request-handling code returns its thread to a pool when it waits for another I/O operation to complete so that the thread can service other requests. This fine-grained sharing of threads -- in which code holds on to a thread only while it performs calculations, not while it waits for I/O -- allows a high number of concurrent operations without consuming a high number of threads. While it removes the limitation on throughput imposed by the scarcity of OS threads, it comes at a high price: It requires what is known as an
asynchronous
programming style, employing a separate set of I/O methods that do not wait for I/O operations to complete but rather, later on, signal their completion to a callback. Without a dedicated thread, developers must break down their request-handling logic into small stages, typically written as lambda expressions, and then compose them into a sequential pipeline with an API (see
CompletableFuture
, for example, or so-called "reactive" frameworks). They thus forsake the language's basic sequential composition operators, such as loops and
try/catch
blocks.
In the asynchronous style, each stage of a request might execute on a different thread, and every thread runs stages belonging to different requests in an interleaved fashion. This has deep implications for understanding program behavior: Stack traces provide no usable context, debuggers cannot step through request-handling logic, and profilers cannot associate an operation's cost with its caller. Composing lambda expressions is manageable when using Java's
stream API
to process data in a short pipeline but problematic when all of the request-handling code in an application must be written in this way. This programming style is at odds with the Java Platform because the application's unit of concurrency -- the asynchronous pipeline -- is no longer the platform's unit of concurrency.

[java_173] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Preserving the thread-per-request style with virtual threads

Sumber: ht

To enable applications to scale while remaining harmonious with the platform, we should strive to preserve the thread-per-request style. We can do this by implementing threads more efficiently, so they can be more plentiful. Operating systems cannot implement OS threads more efficiently because different languages and runtimes use the thread stack in different ways. It is possible, however, for a Java runtime to implement Java threads in a way that severs their one-to-one correspondence to OS threads. Just as operating systems give the illusion of plentiful memory by mapping a large virtual address space to a limited amount of physical RAM, a Java runtime can give the illusion of plentiful threads by mapping a large number of
virtual
threads to a small number of OS threads.
A
virtual thread
is an instance of
java.lang.Thread

that is not tied to a particular OS thread. A
platform thread
, by contrast, is an instance of
java.lang.Thread
implemented in the traditional way, as a thin wrapper around an OS thread.
Application code in the thread-per-request style can run in a virtual thread for the entire duration of a request, but the virtual thread consumes an OS thread only while it performs calculations on the CPU. The result is the same scalability as the asynchronous style, except it is achieved transparently: When code running in a virtual thread calls a blocking I/O operation in the
java.*
API, the runtime performs a non-blocking OS call and automatically suspends the virtual thread until it can be resumed later. To Java developers, virtual threads are simply threads that are cheap to create and almost infinitely plentiful. Hardware utilization is close to optimal, allowing a high level of concurrency and, as a result, high throughput, while the application remains harmonious with the multithreaded design of the Java Platform and its tooling.

[java_174] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Implications of virtual threads

Sumber: ht

Virtual threads are cheap and plentiful, and thus should never be pooled: A new virtual thread should be created for every application task. Most virtual threads will thus be short-lived and have shallow call stacks, performing as little as a single HTTP client call or a single JDBC query. Platform threads, by contrast, are heavyweight and expensive, and thus often must be pooled. They tend to be long-lived, have deep call stacks, and be shared among many tasks.
In summary, virtual threads preserve the reliable thread-per-request style that is harmonious with the design of the Java Platform while utilizing the available hardware optimally. Using virtual threads does not require learning new concepts, though it may require unlearning habits developed to cope with today's high cost of threads. Virtual threads will not only help application developers -- they will also help framework designers provide easy-to-use APIs that are compatible with the platform's design without compromising on scalability.

[java_175] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Description

Sumber: ht

Today, every instance of
java.lang.Thread
in the JDK is a
platform thread
. A platform thread runs Java code on an underlying OS thread and captures the OS thread for the code's entire lifetime. The number of platform threads is limited to the number of OS threads.
A
virtual thread
is an instance of
java.lang.Thread
that runs Java code on an underlying OS thread but does not capture the OS thread for the code's entire lifetime. This means that many virtual threads can run their Java code on the same OS thread, effectively sharing it. While a platform thread monopolizes a precious OS thread, a virtual thread does not. The number of virtual threads can be much larger than the number of OS threads.
Virtual threads are a lightweight implementation of threads that is provided by the JDK rather than the OS. They are a form of
user-mode threads
, which have been successful in other multithreaded languages (e.g., goroutines in Go and processes in Erlang). User-mode threads even featured as so-called
"green threads"

in early versions of Java, when OS threads were not yet mature and widespread. However, Java's green threads all shared one OS thread (M:1 scheduling) and were eventually outperformed by platform threads, implemented as wrappers for OS threads (1:1 scheduling). Virtual threads employ M:N scheduling, where a large number (M) of virtual threads is scheduled to run on a smaller number (N) of OS threads.

[java_176] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Using virtual threads vs. platform threads

Sumber: ht

Developers can choose whether to use virtual threads or platform threads. Here is an example program that creates a large number of virtual threads. The program first obtains an
ExecutorService
that will create a new virtual thread for each submitted task. It then submits 10,000 tasks and waits for all of them to complete:
The task in this example is simple code -- sleep for one second -- and modern hardware can easily support 10,000 virtual threads running such code concurrently. Behind the scenes, the JDK runs the code on a small number of OS threads, perhaps as few as one.
Things would be very different if this program used an
ExecutorService
that creates a new platform thread for each task, such as
Executors.newCachedThreadPool()
. The
ExecutorService
would attempt to create 10,000 platform threads, and thus 10,000 OS threads, and the program might crash, depending on the machine and operating system.
Things would be not much better if the program, instead, used an
ExecutorService
that obtains platform threads from a pool, such as
Executors.newFixedThreadPool(200)
. The
ExecutorService
would create 200 platform threads to be shared by all 10,000 tasks, so many of the tasks would run sequentially rather than concurrently and the program would take a long time to complete. For this program, a pool with 200 platform threads can only achieve a throughput of 200 tasks-per-second, whereas virtual threads achieve a throughput of about 10,000 tasks-per-second (after sufficient warmup). Moreover, if the
10_000
in the example program is changed to
1_000_000
, then the program would submit 1,000,000 tasks, create 1,000,000 virtual threads that run concurrently, and (after sufficient warmup) achieve a throughput of about 1,000,000 tasks-per-second.
If the tasks in this program performed a calculation for one second (e.g., sorting a huge array), rather than merely sleeping, then increasing the number of threads beyond the number of processor cores would not help, whether they are virtual threads or platform threads. Virtual threads are not faster threads -- they do not run code any faster than platform threads. They exist to provide scale (higher throughput), not speed (lower latency). There can be many more of them than platform threads, so they enable the higher concurrency needed for higher throughput according to Little's Law.
To put it another way, virtual threads can significantly improve application throughput when
The number of concurrent tasks is high (more than a few thousand), and
The workload is not CPU-bound, since having many more threads than processor cores cannot improve throughput in that case.
Virtual threads help to improve the throughput of typical server applications precisely because such applications consist of a great number of concurrent tasks that spend much of their time waiting.

A virtual thread can run any code that a platform thread can run. In particular, virtual threads support
thread-local variables
and thread interruption, just like platform threads. This means that existing Java code that processes requests will easily run in a virtual thread. Many server frameworks will choose to do this automatically, starting a new virtual thread for every incoming request and running the application's business logic in it.
Here is an example of a server application that aggregates the results of two other services. A hypothetical server framework (not shown) creates a new virtual thread for each request and runs the application's
handle
code in that virtual thread. The application code, in turn, creates two new virtual threads to fetch resources concurrently via the same
ExecutorService
as the first example:
A server application like this, with straightforward blocking code, scales well because it can employ a large number of virtual threads.
Executor.newVirtualThreadPerTaskExecutor()
is not the only way to create virtual threads. The new
java.lang.Thread.Builder
API,
discussed below
, can create and start virtual threads. In addition,
structured concurrency
offers a more powerful API to create and manage virtual threads, particularly in code similar to this server example, whereby the relationships among threads are made known to the platform and its tools.

Contoh Kode / Perintah CLI:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i -> {
        executor.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1));
            return i;
        });
    });
}  // executor.close() is called implicitly, and waits

// ----------------------------------------

void handle(Request request, Response response) {
    var url1 = ...
    var url2 = ...

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        var future1 = executor.submit(() -> fetchURL(url1));
        var future2 = executor.submit(() -> fetchURL(url2));
        response.send(future1.get() + future2.get());
    } catch (ExecutionException | InterruptedException e) {
        response.fail(e);
    }
}

String fetchURL(URL url) throws IOException {
    try (var in = url.openStream()) {
        return new String(in.readAllBytes(), StandardCharsets.UTF_8);
    }
}
```

[java_177] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Do not pool virtual threads

Sumber: ht

Developers will typically migrate application code to the virtual-thread-per-task
ExecutorService
from a traditional thread-pool based
ExecutorService
. A thread pool, like any resource pool, is intended to share expensive resources, but virtual threads are not expensive so there is never a need to pool them.
Developers sometimes use thread pools to limit concurrent access to limited resources. For example, if a service cannot handle more than 20 concurrent requests then making all requests to the service via tasks submitted to a thread pool of size 20 will ensure that. This idiom has become ubiquitous because the high cost of platform threads has made thread pools ubiquitous, but do not be tempted to pool virtual threads in order to limit concurrency. Instead use constructs specifically designed for that purpose, such as semaphores.
In conjunction with thread pools, developers sometimes use thread-local variables to share expensive resources among multiple tasks that share the same thread. For example, if a database connection is expensive to create then you can open it once and store it in a thread-local variable for later use by other tasks in the same thread. If you migrate code from using a thread pool to using a virtual thread per task, be wary of usages of this idiom since creating an expensive resource for every virtual thread may degrade performance significantly. Change such code to use alternative caching strategies so that expensive resources can be shared efficiently among a very large number of virtual threads.

[java_178] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Observing virtual threads

Sumber: ht

Writing clear code is not the full story. A clear presentation of the state of a running program is also essential for troubleshooting, maintenance, and optimization, and the JDK has long offered mechanisms to debug, profile, and monitor threads. Such tools should do the same for virtual threads -- perhaps with some accommodation to their large quantity -- since they are, after all, instances of
java.lang.Thread
.
Java debuggers can step through virtual threads, show call stacks, and inspect variables in stack frames. JDK Flight Recorder (JFR), which is the JDK's low-overhead profiling and monitoring mechanism, can associate events from application code (such as object allocation and I/O operations) with the correct virtual thread. These tools cannot do these things for applications written in the asynchronous style. In that style tasks are not related to threads, so debuggers cannot display or manipulate the state of a task, and profilers cannot tell how much time a task spends waiting for I/O.
The thread dump is another popular tool for troubleshooting applications written in the thread-per-request style. Unfortunately the JDK's traditional thread dump, obtained with
jstack
or
jcmd
, presents a flat list of threads. This is suitable for dozens or hundreds of platform threads, but is unsuitable for thousands or millions of virtual threads. Accordingly, we will not extend traditional thread dumps to include virtual threads; we will, rather, introduce a new kind of thread dump in
jcmd
to present virtual threads alongside platform threads, all grouped in a meaningful way. Richer relationships among threads can be shown when programs use
structured concurrency
.
Because visualizing and analyzing a great many threads can benefit from tooling,
jcmd

can emit the new thread dump in JSON format in addition to plain text:
The new thread dump format does not include object addresses, locks, JNI statistics, heap statistics, and other information that appears in traditional thread dumps. Moreover, because it might need to list a great many threads, generating a new thread dump does not pause the application.
If the system property
jdk.trackAllThreads
is set to
false
, i.e. with the
-Djdk.trackAllThreads=false
command-line option, virtual threads created directly with the
Thread.Builder
API will not always be tracked by the runtime and may not appear in the new thread dump. In that case, the new thread dump would list virtual threads that are blocked in network I/O operations, and virtual threads that are created by the new-thread-per-task
ExecutorService
shown above.
Here is an example of such a thread dump, taken from an application similar to the
second example above
, rendered in a JSON viewer (click to enlarge):
Since virtual threads are implemented in the JDK and are not tied to any particular OS thread, they are invisible to the OS, which is unaware of their existence. OS-level monitoring will observe that a JDK process uses fewer OS threads than there are virtual threads.

Contoh Kode / Perintah CLI:

```
$ jcmd <pid> Thread.dump_to_file -format=json <file>
```

[java_179] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Scheduling virtual threads

Sumber: ht

To do useful work a thread needs to be scheduled, that is, assigned for execution on a processor core. For platform threads, which are implemented as OS threads, the JDK relies on the scheduler in the OS. For virtual threads, by contrast, the JDK has its own scheduler. Rather than assigning virtual threads to processors directly, the JDK's scheduler assigns virtual threads to platform threads (this is the M:N scheduling of virtual threads mentioned earlier). The platform threads are then scheduled by the OS as usual.
The JDK's virtual thread scheduler is a work-stealing
ForkJoinPool
that operates in FIFO mode. The
parallelism
of the scheduler is the number of platform threads available for the purpose of scheduling virtual threads. By default it is equal to the number of
available processors
, but it can be tuned with the system property
jdk.virtualThreadScheduler.parallelism
. This
ForkJoinPool
is distinct from the
common pool
which is used, for example, in the implementation of parallel streams, and which operates in LIFO mode.
The platform thread to which the scheduler assigns a virtual thread is called the virtual thread's
carrier
. A virtual thread can be scheduled on different carriers over the course of its lifetime; in other words, the scheduler

does not maintain
affinity
between a virtual thread and any particular platform thread. From the perspective of Java code, a running virtual thread is logically independent of its current carrier:
The identity of the carrier is unavailable to the virtual thread. The value returned by
Thread.currentThread()
is always the virtual thread itself.
The stack traces of the carrier and the virtual thread are separate. An exception thrown in the virtual thread will not include the carrier's stack frames. Thread dumps will not show the carrier's stack frames in the virtual thread's stack, and vice-versa.
Thread-local variables of the carrier are unavailable to the virtual thread, and vice-versa.
In addition, from the perspective of Java code, the fact that a virtual thread and its carrier temporarily share an OS thread is invisible. From the perspective of native code, by contrast, both the virtual thread and its carrier run on the same native thread. Native code that is called multiple times on the same virtual thread may thus observe a different OS thread identifier at each invocation.
The scheduler does not currently implement
time sharing
for virtual threads. Time sharing is the forceful preemption of a thread that has consumed an allotted quantity of CPU time. While time sharing can be effective at reducing the latency of some tasks when there are a relatively small number of platform threads and CPU utilization is at 100%, it is not clear that time sharing would be as effective with a million virtual threads.

[java_180] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Executing virtual threads

Sumber: ht

To take advantage of virtual threads, it is not necessary to rewrite your program. Virtual threads do not require or expect application code to explicitly hand control back to the scheduler; in other words, virtual threads are not
cooperative
. User code must not make assumptions about how or when virtual threads are assigned to platform threads any more than it makes assumptions about how or when platform threads are assigned to processor cores.
To run code in a virtual thread, the JDK's virtual thread scheduler assigns the virtual thread for execution on a platform thread by
mounting
the virtual thread on a platform thread. This makes the platform thread become the carrier of the virtual thread. Later, after running some code, the virtual thread can
unmount
from its carrier. At that point the platform thread is free so the scheduler can mount a different virtual thread on it, thereby making it a carrier again.
Typically, a virtual thread will unmount when it blocks on I/O or some other blocking operation in the JDK, such as
BlockingQueue.take()
. When the blocking operation is ready to complete (e.g., bytes have been received on a socket), it submits the virtual thread back to the scheduler, which will mount the virtual thread on a carrier to resume execution.
The mounting and unmounting of virtual threads happens frequently and transparently, and without blocking any OS threads. For example, the server application shown earlier included the following line of code, which contains calls to blocking operations:
These operations will cause the virtual thread to mount and unmount multiple times, typically once for each call to
get()
and possibly multiple times in the course of performing I/O in
send(...)
.
The vast majority of blocking operations in the JDK will unmount the virtual thread, freeing its carrier and the underlying

OS thread to take on new work. However, some blocking operations in the JDK do not unmount the virtual thread, and thus block both its carrier and the underlying OS thread. This is because of limitations at either the OS level (e.g., many filesystem operations) or the JDK level (e.g.,
Object.wait()
). The implementations of these blocking operations compensate for the capture of the OS thread by temporarily expanding the parallelism of the scheduler. Consequently, the number of platform threads in the scheduler's
ForkJoinPool
may temporarily exceed the number of available processors. The maximum number of platform threads available to the scheduler can be tuned with the system property
jdk.virtualThreadScheduler.maxPoolSize
.
There are two scenarios in which a virtual thread cannot be unmounted during blocking operations because it is
pinned
to its carrier:
When it executes code inside a
synchronized
block or method, or
When it executes a
native
method or a
foreign function
.
Pinning does not make an application incorrect, but it might hinder its scalability. If a virtual thread performs a blocking operation such as I/O or
BlockingQueue.take()
while it is pinned, then its carrier and the underlying OS thread are blocked for the duration of the operation. Frequent pinning for long durations can harm the scalability of an application by capturing carriers.
The scheduler does not compensate for pinning by expanding its parallelism. Instead, avoid frequent and long-lived pinning by revising
synchronized
blocks or methods that run frequently and guard potentially long I/O operations to use
java.util.concurrent.locks.ReentrantLock
instead. There is no need to replace
synchronized
blocks and methods that are used infrequently (e.g., only performed at startup) or that guard in-memory operations. As always, strive to keep locking policies simple and clear.
New diagnostics assist in migrating code to virtual threads and in assessing whether you should replace a particular use of
synchronized
with a
java.util.concurrent
lock:
A JDK Flight Recorder (JFR) event is emitted when a thread blocks while pinned (see
JDK Flight Recorder
).
The system property
jdk.tracePinnedThreads
triggers a stack trace when a thread blocks while pinned. Running with
-Djdk.tracePinnedThreads=full
prints a complete stack trace when a thread blocks while pinned, highlighting native frames and frames holding monitors. Running with
-Djdk.tracePinnedThreads=short

limits the output to just the problematic frames.
In a future release we may be able to remove the first limitation above, namely pinning inside
synchronized
. The second limitation is required for proper interaction with native code.

Contoh Kode / Perintah CLI:

```
response.send(future1.get() + future2.get());
```

[java_181] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Memory use and interaction with garbage collection

Sumber: ht

The stacks of virtual threads are stored in Java's garbage-collected heap as
stack chunk
objects. The stacks grow and shrink as the application runs, both to be memory-efficient and to accommodate stacks of depth up to the JVM's configured platform thread stack size. This efficiency is what enables a large number of virtual threads and thus the continued viability of the thread-per-request style in server applications.
In the
second example above
, recall that a hypothetical framework processes each request by creating a new virtual thread and calling the
handle
method. Even if it calls
handle
at the end of a deep call stack (after authentication, transactions, etc.),
handle
itself spawns multiple virtual threads that only perform short-lived tasks. Therefore, for each virtual thread with a deep call stack, there will be multiple virtual threads with shallow call stacks consuming little memory.
The amount of heap space and garbage collector activity that virtual threads require is difficult, in general, to compare to that of asynchronous code. A million virtual threads require at least a million objects, but so do a million tasks sharing a pool of platform threads. In addition, application code that processes requests typically maintains data across I/O operations. Thread-per-request code can keep that data in local variables, which are stored on virtual thread stacks in the heap, while asynchronous code must keep that same data in heap objects that are passed from one stage of the pipeline to the next. On the one hand, the stack frame layout needed by virtual threads is more wasteful than that of a compact object; on the other hand, virtual threads can mutate and reuse their stacks in many situations (depending on low-level GC interactions) while asynchronous pipelines always need to allocate new objects, and so virtual threads might require fewer allocations. Overall, the heap consumption and garbage collector activity of thread-per-request versus asynchronous code should be roughly similar. Over time, we expect to make the internal representation of virtual thread stacks significantly more compact.
Unlike platform thread stacks, virtual thread stacks are not GC roots. Thus the references they contain are not traversed in a stop-the-world pause by garbage collectors, such as G1, that perform concurrent heap scanning.
A current limitation of virtual threads is that the G1 GC does not support
humongous
stack chunk objects. If a virtual thread's stack reaches half the region size, which could be as small as 512KB, then a
StackOverflowError
might be thrown.

[java_182] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Detailed changes

Sumber: ht

The remaining subsections describe, in detail, the changes we propose across the Java Platform and its implementation:
java.lang.Thread
Thread-local variables
java.util.concurrent
Networking
java.io
Java Native Interface (JNI)
Debugging (JVM TI, JDWP, and JDI)
JDK Flight Recorder (JFR)
Java Management Extensions (JMX)

[java_183] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: java.lang.Thread

Sumber: ht

We update the
java.lang.Thread
API as follows:
Thread.Builder
,
Thread.ofVirtual()
, and
Thread.ofPlatform()
are new APIs to create virtual and platform threads. For example,
creates a new unstarted virtual thread named
"duke
".
Thread.startVirtualThread(Runnable)
is a convenient way to create and then start a virtual thread.
A
Thread.Builder
can create either a thread or a
ThreadFactory
, which can then create multiple threads with identical properties.
Thread.isVirtual()
tests whether a thread is a virtual thread.
Thread.getAllStackTraces()
now returns a map of all platform threads rather than all threads.
The
java.lang.Thread
API is otherwise unchanged by this JEP. The constructors defined by the
Thread
class create platform threads, as before. There are no new public constructors.
(Three methods in
Thread
which throw
UnsupportedOperationException
for virtual threads --

stop()
,
suspend()
, and
resume()
-- were
changed in JDK 20
to throw
UnsupportedOperationException
for platform threads too.)
The main API differences between virtual and platforms threads are:
The public
Thread
constructors cannot create virtual threads.
Virtual threads are always daemon threads. The
Thread.setDaemon(boolean)
method cannot change a virtual thread to be a non-daemon thread.
Virtual threads have a fixed priority of
Thread.NORM_PRIORITY
. The
Thread.setPriority(int)
method has no effect on virtual threads. This limitation may be revisited in a future release.
Virtual threads are not active members of thread groups. When invoked on a virtual thread,
Thread.getThreadGroup()
returns a placeholder thread group with the name
"VirtualThreads"
. The
Thread.Builder
API does not define a method to set the thread group of a virtual thread.
Virtual threads have no permissions when running with a
SecurityManager
set.

Contoh Kode / Perintah CLI:

```
Thread thread = Thread.ofVirtual().name("duke").unstarted(runnable);
```

[java_184] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Thread-local variables

Sumber: ht

Virtual threads support thread-local variables (
ThreadLocal
) and inheritable thread-local variables (
InheritableThreadLocal
), just like platform threads, so they can run existing code that uses thread locals. However, because virtual threads can be very numerous, use thread locals only after careful consideration. In particular, do not use thread locals to pool costly resources among multiple tasks sharing the same thread in a thread pool.
Virtual threads should never be pooled
since each is intended to run only a single task over its lifetime. We have removed many uses of thread locals from the JDK's
java.base
module in preparation for virtual threads in order to reduce memory footprint when running with millions of threads.
The system property

jdk.traceVirtualThreadLocals
can be used to trigger a stack trace when a virtual thread sets the value of any thread-local variable. This diagnostic output may assist with removing thread locals when migrating code to use virtual threads. Set the system property to
true
to trigger stack traces; the default value is
false
.
Scoped values (
JEP 429
) may prove to be a better alternative to thread locals for some use cases.

[java_185] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: java.util.concurrent

Sumber: ht

The primitive API to support locking,
java.util.concurrent.LockSupport
, now supports virtual threads: Parking a virtual thread releases the underlying platform thread to do other work, and unparking a virtual thread schedules it to continue. This change to
LockSupport
enables all APIs that use it (
Lock
s,
Semaphore
s, blocking queues, etc.) to park gracefully when invoked in virtual threads.
Additionally,
Executors.newThreadPerTaskExecutor(ThreadFactory)
and
Executors.newVirtualThreadPerTaskExecutor()
create an
ExecutorService
that creates a new thread for each task. These methods enable migration and interoperability with existing code that uses thread pools and
ExecutorService
.

[java_186] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Networking

Sumber: ht

The implementations of the networking APIs in the
java.net
and
java.nio.channels
packages now work with virtual threads: An operation on a virtual thread that blocks, e.g., to establish a network connection or read from a socket, releases the underlying platform thread to do other work.
To allow for interruption and cancellation, the blocking I/O methods defined by
java.net.Socket
,
ServerSocket
, and
DatagramSocket
are now specified to be

interruptible
when invoked in a virtual thread: Interrupting a virtual thread blocked on a socket will unpark the thread and close the socket. Blocking I/O operations on these types of sockets when obtained from an
InterruptibleChannel
have always been interruptible, so this change aligns the behavior of these APIs when created with their constructors with their behavior when obtained from a channel.

[java_187] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: java.io

Sumber: ht

The
java.io
package provides APIs for streams of bytes and characters. The implementations of these APIs are heavily synchronized and require changes to avoid pinning when they are used in virtual threads.
As background, the byte-oriented input/output streams are not specified to be thread-safe and do not specify the expected behavior when
close()
is invoked while a thread is blocked in a read or write method. In most scenarios it does not make sense to use a particular input or output stream from multiple concurrent threads. The character-oriented reader/writers are also not specified to be thread-safe, but they do expose a lock object for sub-classes. Aside from pinning, the synchronization in these classes is problematic and inconsistent; e.g., the stream decoders and encoders used by
InputStreamReader
and
OutputStreamWriter
synchronize on the stream object rather than the lock object.
To prevent pinning, the implementations now work as follows:
BufferedInputStream
,
BufferedOutputStream
,
BufferedReader
,
BufferedWriter
,
PrintStream
, and
PrintWriter
now use an explicit lock rather than a monitor when used directly. These classes synchronize as before when they are sub-classed.
The stream decoders and encoders used by
InputStreamReader
and
OutputStreamWriter
now use the same lock as the enclosing
InputStreamReader
or
OutputStreamWriter
.
Going further and eliminating all this often-needless locking is beyond the scope of this JEP.
Additionally, the initial sizes of the buffers used by
BufferedOutputStream

,
BufferedWriter
, and the stream encoder for
OutputStreamWriter
are now smaller so as to reduce memory usage when there are many streams or writers in the heap -- as might arise if there are a million virtual threads, each with a buffered stream on a socket connection.

[java_188] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Java Native Interface (JNI)

Sumber: ht

JNI defines one new function,
IsVirtualThread
, to test if an object is a virtual thread.
The JNI specification is otherwise unchanged.

[java_189] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Debugging

Sumber: ht

The debugging architecture consists of three interfaces: the JVM Tool Interface (JVM TI), the Java Debug Wire Protocol (JDWP), and the Java Debug Interface (JDI). All three interfaces now support virtual threads.
The updates to
JVM TI
are:
Most functions that are called with a
jthread
(i.e., a JNI reference to a
Thread
object) can be called with a reference to a virtual thread. A small number of functions, namely
AgentStartFunction
,
PopFrame
,
ForceEarlyReturn*
,
StopThread
, and
GetThreadCpuTime
, are not supported or are optionally supported on virtual threads. The
SetLocal*
functions are limited to setting local variables in the top-most frames of virtual threads that are suspended at a breakpoint or single step event.
The
GetAllThreads
and
GetAllStackTraces
functions are now specified to return all platform threads rather than all threads.
All events, with the exception of those posted during early VM startup or during heap iteration, can have event callbacks invoked in the context of a virtual thread.
The suspend/resume implementation allows virtual threads to be suspended and resumed by debuggers, and it allows platform threads to be suspended when a virtual thread is mounted.
A new capability,

can_support_virtual_threads
, gives agents finer control over thread start and end events for virtual threads.
New functions support the bulk suspension and resumption of virtual threads; these require the
can_support_virtual_threads
capability.
Existing JVM TI agents will mostly work as before, but may encounter errors if they invoke functions that are not supported on virtual threads. These will arise when an agent that is unaware of virtual threads is used with an application that uses virtual threads. The change to
GetAllThreads
to return an array containing only the platform threads may be an issue for some agents. Existing agents that enable the
ThreadStart
and
ThreadEnd
events may encounter performance issues since they lack the ability to limit these events to platform threads.
The updates to
JDWP
are:
A new command allows debuggers to test if a thread is a virtual thread.
A new modifier on the
EventRequest
command allows debuggers to restrict thread start and end events to platform threads.
The updates to
JDI
are:
A new method in
com.sun.jdi.ThreadReference
tests if a thread is a virtual thread.
New methods in
com.sun.jdi.request.ThreadStartRequest
and
com.sun.jdi.request.ThreadDeathRequest
limit the events generated for the request to platform threads.
As noted above, virtual threads are not considered to be active threads in a thread group. Consequently the thread lists returned by the JVM TI function
GetThreadGroupChildren
, the JDWP command
ThreadGroupReference/Children
, and the JDI method
com.sun.jdi.ThreadGroupReference.threads()
include only platform threads.

[java_190] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: JDK Flight Recorder (JFR)

Sumber: ht

JFR supports virtual threads with several new events:
jdk.VirtualThreadStart
and
jdk.VirtualThreadEnd
indicate virtual thread start and end. These events are disabled by default.
jdk.VirtualThreadPinned
indicates that a virtual thread was parked while pinned, i.e., without releasing its platform thread (see

above
). This event is enabled by default, with a threshold of 20ms.
jdk.VirtualThreadSubmitFailed
indicates that starting or unparking a virtual thread failed, probably due to a resource issue. This event is enabled by default.

[java_191] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Java Management Extensions (JMX)

Sumber: ht

java.lang.management.ThreadMXBean
only supports the monitoring and management of platform threads. The
findDeadlockedThreads()
method finds cycles of platform threads that are in deadlock; it does not find cycles of virtual threads that are in deadlock.
A new method in
com.sun.management.HotSpotDiagnosticsMXBean
generates the new-style thread dump described
above
. This method can also be invoked indirectly via the platform
MBeanServer
from a local or remote JMX tool.

[java_192] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Alternatives

Sumber: ht

Continue to rely on asynchronous APIs. Asynchronous APIs are difficult to integrate with synchronous APIs, create a split world of two representations of the same I/O operations, and provide no unified concept of a sequence of operations that can be used by the platform as context for troubleshooting, monitoring, debugging, and profiling.
Add
syntactic stackless coroutines
(i.e.,
async/await
) to the Java language. These are easier to implement than user-mode threads and would provide a unifying construct representing the context of a sequence of operations.
That construct would be new, however, and separate from threads, similar to them in many respects yet different in some nuanced ways. It would split the world between APIs designed for threads and APIs designed for coroutines, and would require the new thread-like construct to be introduced into all layers of the platform and its tooling. This would take longer for the ecosystem to adopt, and would not be as elegant and harmonious with the platform as user-mode threads.
Most languages that have adopted syntactic coroutines have done so due to an inability to implement user-mode threads (e.g., Kotlin), legacy semantic guarantees (e.g., the inherently single-threaded JavaScript), or language-specific technical constraints (e.g., C++). These limitations do not apply to Java.
Introduce a new public class to represent user-mode threads, unrelated to
java.lang.Thread
. This would be an opportunity to jettison the unwanted baggage that the
Thread
class has accumulated over 25 years. We explored and prototyped several variants of this approach, but in every case grappled with the issue of how to run existing code.
The main problem is that
Thread.currentThread()
is used, directly or indirectly, pervasively in existing code (e.g., in determining lock ownership, or for thread-local

variables). This method must return an object that represents the current thread of execution. If we introduced a new class to represent user-mode threads then
currentThread()
would have to return some sort of wrapper object that looks like a
Thread
but delegates to the user-mode thread object.
It would be confusing to have two objects represent the current thread of execution, so we eventually concluded that preserving the old
Thread
API is not a significant hurdle. With the exception of a few methods such as
currentThread()
, developers rarely use the
Thread
API directly; they mostly interact use higher-level APIs such as
ExecutorService
. Over time we will jettison unwanted baggage from the
Thread
class, and associated classes such as
ThreadGroup
, by deprecating and removing obsolete methods.

[java_193] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Testing

Sumber: ht

Existing tests will ensure that the changes we propose here do not cause unexpected regressions in the multitude of configurations and execution modes in which they are run.
We will extend the
jtreg
test harness to allow existing tests to be run in the context of a virtual thread. This will avoid needing two versions of many tests.
New tests will exercise all new and revised APIs, and all areas changed to support virtual threads.
New stress tests will target areas that are critical to reliability and performance.
New microbenchmarks will target performance-critical areas.
We will use a number of existing servers, including
Helidon
and
Jetty
, for larger-scale testing.

[java_194] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Risks and Assumptions

Sumber: ht

The primary risks of this proposal are ones of compatibility due to changes in existing APIs and their implementations:
The revisions to the internal (and undocumented) locking protocol used in the
java.io.BufferedInputStream
,
BufferedOutputStream
,
BufferedReader
,
BufferedWriter

,
PrintStream
, and
PrintWriter
classes may impact code that assumes that I/O methods synchronize on the stream upon which they are invoked. These changes do not impact code that extends these classes and assumes locking by the superclass, nor do they impact code that extends
java.io.Reader
or
java.io.Writer
and uses the lock object exposed by those APIs.
A few source and binary incompatible changes may impact code that extends
java.lang.Thread
:
Thread
defines several new methods. If code in an existing source file extends
Thread
and a method in the subclass conflicts with any of the new
Thread
methods then the file will not compile without change.
Thread.Builder
is a new nested interface. If code in an existing source file extends
Thread
, imports a class named
Builder
, and code in the subclass references
Builder
as a simple name, then the file will not compile without change.
Thread.isVirtual()
is a new final method. If there is existing compiled code that extends
Thread
and the subclass declares a method with the same name and return type then an
IncompatibleClassChangeError
will be thrown at run-time if the subclass is loaded.
A few behavioral differences between platform threads and virtual threads may be observed when mixing existing code with newer code that takes advantage of virtual threads or the new APIs:
The
Thread.setPriority(int)
method has no effect on virtual threads, which always have a priority of
Thread.NORM_PRIORITY
.
The
Thread.setDaemon(boolean)
method has no effect on virtual threads, which are always daemon threads.
Thread.getAllStackTraces()
now returns a map of all platform threads rather than a map of all threads.
The blocking I/O methods defined by
java.net.Socket
,
ServerSocket
, and
DatagramSocket

are now interruptible when invoked in the context a virtual thread. Existing code could break when a thread blocked on a socket operation is interrupted, which will wake the thread and close the socket.
Virtual threads are not active members of a
ThreadGroup
. Invoking
Thread.getThreadGroup()
on a virtual thread returns a dummy
"VirtualThreads"
group that is empty.
Virtual threads have no permissions when running with a security manager set. See
JEP 411 (Deprecate the Security Manager for Removal)
for information about running with a security manager on Java 17 and later.
In JVM TI, the
GetAllThreads
and
GetAllStackTraces
functions do not return virtual threads. Existing agents that enable the
ThreadStart
and
ThreadEnd
events may encounter performance issues since they lack the ability to limit the events to platform threads.
The
java.lang.management.ThreadMXBean
API supports the monitoring and management of platform threads, but not virtual threads.
The
-XX:+PreserveFramePointer
flag has a drastic negative impact on virtual thread performance.

[java_195] Java - Concurrency - Virtual Threads Spec

JEP 444: Virtual Threads: Dependencies

Sumber: ht

JEP 416 (Reimplement Core Reflection with Method Handles)
in JDK 18 removed the VM-native reflection implementation. This allows virtual threads to park gracefully when methods are invoked reflectively.
JEP 353 (Reimplement the Legacy Socket API)
in JDK 13, and
JEP 373 (Reimplement the Legacy DatagramSocket API)
in JDK 15, replaced the implementations of
java.net.Socket
,
ServerSocket
, and
DatagramSocket
with new implementations designed for use with virtual threads.
JEP 418 (Internet-Address Resolution SPI)
in JDK 18 defined a service-provider interface for host name and address lookup. This will allow third-party libraries to implement alternative
java.net.InetAddress
resolvers that do not pin threads during host lookup.
Installing
Contributing

Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds
Mailing lists
Wiki
*
IRC
Mastodon
Bluesky
Bylaws
*
Census
Legal
*
AI
Workshop
JEP Process
Source code
GitHub
Mercurial
Tools
Git
jtreg harness
Groups
(overview
,
archive
)
Adoption
Build
Client Libraries
Compatibility & Specification Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
Members
Networking
Porters
Quality
Security
Serviceability
Vulnerability
Web
Projects
(
overview
,

archive
)
Amber
Babylon
Brisbane
CRaC
Code Tools
Coin
Common VM Interface
Detroit
Developers' Guide
Duke
IcedTea
JDK 8 Updates
JDK 9
JDK
(...,
26
,
27
,
28
)
JDK Updates
JMC
Jigsaw
Lanai
Leyden
Lilliput
Loom
Memory Model Update
Multi-Language VM
Nashorn
New I/O
OpenJFX
Panama
Port: AArch32
Port: AArch64
Port: BSD
Port: Haiku
Port: MIPS
Port: Mobile
Port: PowerPC/AIX
Port: RISC-V
Port: s390x
SCTP
Shenandoah
Skara
Sumatra
Tsan
Valhalla
Wakefield

Zero
ZGC
(c) 2026 Oracle Corporation and/or its affiliates
Terms of Use
*

License:
GPLv2
*
Privacy
*
Trademarks

[java_196] Java - Functional Programming - Loops to Streams

Converting Simple Loops: Imperative vs. Functional Styles

Sumber: ht

The older versions of Java supported the Object-Oriented paradigm mixed with the imperative style of programming. Starting Java 8, you can also mix the functional style of programming in your code. If your code base was started during the Java 7 or earlier times or later by programmers mostly familiar with older versions of Java, it will be filled with the imperative style code.
Imperative style is where we tell what to do and also how to do it. Functional style is declarative in nature, where we tell what to do and delegate the how or the details to the underlying libraries. Imperative style code may be easier to write since most of us are very familiar with it. However, the code becomes verbose, complex, and hard to read. Functional style may be hard at first, mainly because most programmers are less familiar with it. In general, it's easier to read, understand, and change. With practice, it becomes easier to write as well.
In this tutorial series we will take a look at a number of common imperative style code and find a mapping or an equivalent functional style code that we can use instead. As you work with your code based, when you're ready to fix a bug or make an enhancement, you may find it useful to refactor some of the imperative style code to the functional style. You can use this tutorial as a guide to find the imperative to functional style mappings for some common situations.
In this tutorial we'll focus on simple loops.

[java_197] Java - Functional Programming - Loops to Streams

Converting Simple Loops: Simple for Loops

Sumber: ht

Let's start with the traditional for loop where we perform an action for values of an index over a range.
In the above code, the essence is the range, from `0` to one less than `5`. The ceremony, the noise, from the __how__, is the syntax plus the increment operation on the index variable `i`. We can keep the essence and remove the ceremony by turning the code to the functional style.
If you like to use the functional style to write this `for` loop, you can do quite easily and the clue is in the sentence before the code: __index over a range__. Since we're iterating over a range, the `range` method of `IntStream` is the direct equivalent for this.
You can further make this concise by using a methor reference for the `println` method.
The functional style code is more concise, easier to read, and the intention is clearer in this version than in the imperative version.
What if your `for` loop runs to include the ending value, like in the following code, you may wonder.
The `IntStream` interface has you covered, it has a `rangeClosed` method exactly for this purpose.
The `rangeClosed` method is useful to iterate from the starting value all the way to include the ending value.
Whether you use the `range` method or the `rangeClosed` method, you get a stream of `int` values over which you can use the internal iterator to perform actions. Later in this series we will look at operations beyond `forEach`.
In the previous code examples, the internal iterator removed the burden of iterating from your shoulders. The stream

takes care of stepping over the range of values, one at a time. You only have to focus on what to do for each element, as they're provided to you, in the `forEach` method. In our examples, we merely printed the provided value. You can do just about any operation you like, like saving the information to a database, sending it off to a remote service, etc.

Unlike the external iterator provided by the `for` loop, the code that uses the internal iterator is more concise, has less noise, avoids the need to explicitly mutate the index variable, is easier to read, easier to modify, and more pleasant to work with.

Proceed to look for opportunities in your own code base where you see the traditional `for` loop and modify it to using the `IntStream`'s `range` or `rangeClosed` method. Make sure you verify that the code works as expected after the change, preferably by running automated tests that you may already have.

Contoh Kode / Perintah CLI:

```
for(int i = 0; i < 5; i++) {
  System.out.println(i);
}

// ----------------------------------------

import java.util.stream.IntStream;

...
IntStream.range(0, 5)
  .forEach(i -> System.out.println(i));

// ----------------------------------------

import java.util.stream.IntStream;

...
IntStream.range(0, 5)
  .forEach(System.out::println);

// ----------------------------------------

for(int i = 0; i <= 5; i++) {
  System.out.println(i);
}

// ----------------------------------------

import java.util.stream.IntStream;

...
IntStream.rangeClosed(0, 5)
  .forEach(System.out::println);
```

[java_198] Java - Functional Programming - Loops to Streams

Converting Simple Loops: Mappings

Sumber: ht

Anywhere you see a simple `for` loop, you can use either the `range` or the `rangeClosed` method of `IntStream`. Use the `range` method if you want to iterate until but not including the ending value. Use the `rangeClosed` to include the ending value as well in your iteration.

[java_199] Java - Functional Programming - Filtering Streams

Converting foreach with if: Iterating with foreach

Sumber: ht

In the previous articles in this tutorial series we looked at converting loops written in the imperative style to the functional style. In this article we'll see how to convert an imperative style iteration using `foreach` to the functional style. In addition, we'll also see how to pick select elements using `if` transforms to the functional style.
Java 5 introduced the very popular `foreach` syntax. For example, to iterate over a collection of `String`s representing names, we'd write something like `for(String name: names)`. Under the hood, the `foreach` is converted, at the bytecode level, to use an `Iterator`—while the iterator tells us there is another element, fetch the next element for processing. In other words, the `foreach` is a nice concise syntax sugar for iteration with a `while` loop over the elements provided by an `Iterator`. We can convert a `foreach` into the functional style quite easily. Let's see how.

[java_200] Java - Functional Programming - Filtering Streams

Converting foreach with if: From Imperative to Functional Style

Sumber: ht

Here's an example of iteration, using the `foreach`, over a collection of names:
Each step through the iteration, the `name` variable is bound to a new value, as the iteration advances from one element to the next in the given collection. Converting the imperative style `foreach` to the functional style is a straight up use of the `forEach` internal iterator method. It is called an internal iterator because the advancing to the next element is handled internally and automatically instead of externally or explicitly.
Let's refactor the loop to use functional style.
That was pretty straightforward, the `for` loop turned into a call to the `forEach()` method on the collection. Each step through the iteration, the lambda provided to the `forEach()` as an argument is invoked with the next element in the collection.
A slightly variation of this iteration, using `stream()`, is shown next.
The `forEach()` method is available both on a `Collection<T>` and on a `Stream<T>`. Functions like `filter()`, which we'll use soon, are available only on a `Stream<T>` and not on the `Collection`. This is by design in order to provide efficiency when multiple intermediate operations may precede the terminal operation like `forEach()`, `findFirst()`, etc.

Contoh Kode / Perintah CLI:

```
List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

for(String name: names) {
  System.out.println(name);
}

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

names.forEach(name -> System.out.println(name));

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

names.stream()
  .forEach(name -> System.out.println(name));
```

[java_201] Java - Functional Programming - Filtering Streams

Converting foreach with if: Picking select elements with if

Sumber: ht

Suppose, in the middle of the iteration, we want to pick some values from the collection based on some condition. For example, what if we want to print only names of length 4? In the imperative style we could do the following:
For the functional style, the `filter` method of `Stream` becomes a direct replacement of the imperative style `if`. The `filter` method will allow an element in the collection to pass through to the next stage in the functional pipeline if the predicate, passed in as a lambda, to the `filter()` method evaluates to `true`; otherwise, the value is discarded from further processing.
Let's convert the previous code to functional style:
The `filter()` method acts like a gate, it opens to let some elements pass through and closes to reject or discard some elements, as the iteration moves forward.
We saw in the previous articles the functional style equivalent for the traditional `for` loops. In this article we saw how the imperative style `foreach` of Java 5 transforms into an elegant syntax in the functional style. Furthermore, the `if` condition within a loop of the imperative style translates to a call to the `filter()` method of the `Stream` API.

Contoh Kode / Perintah CLI:

```
List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

for(String name: names) {
  if(name.length() == 4) {
    System.out.println(name);
  }
}

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

names.stream()
  .filter(name -> name.length() == 4)
  .forEach(name -> System.out.println(name));
```

[java_202] Java - Functional Programming - Filtering Streams

Converting foreach with if: Mappings

Sumber: ht

Anywhere you see a `foreach` loop, use the `forEach()` method directly on the collection. If the body of the `foreach` has a `if` statement to selectively pick some value, then use the `stream()` API with the call to the `filter()` method.

[java_203] Java - Functional Programming - Map & Transformations

Converting Iteration with transformation: Transforming while Iterating

Sumber: ht

In the previous articles in this tutorial series we looked at converting loops with `if` or conditional statements in the imperative style to the functional style. In this article we'll see how to convert an imperative style iteration that transforms data to the functional style. In addition, we'll also refactor code that mixes transforming data with code that picks select elements before the transformation.
Anytime we are transforming data in an imperative style loop, we can use the `map()` function in the functional style. Let's see how.

[java_204] Java - Functional Programming - Map & Transformations

Converting Iteration with transformation: From Imperative to Functional Style

Sumber: ht

Here's an example of an iteration, using the `foreach`, that transforms to uppercase a collection of names:
Each step through the iteration, the `name` variable is bound to a new value. As the iteration advances from one element to the next in the given collection, each name is transformed to uppercase using the `toUpperCase()` function and the resulting value is printed. We have already seen, in the previous article, how to convert the imperative style `foreach` to the functional style—using the `stream()` internal iteration. If we merely apply what we have seen before, the resulting functional style code would be rather unwieldy, with the lambda passed to `forEach` performing both transformation and printing, like so:
Even though the above functional style code will produce the same results as the imperative style code, the lambda passed to the `forEach()` function is not cohesive, hard to read, and hard to change.
Before refactoring the imperative style code above to the function style, we should first refactor the imperative style to yet another imperative style implementation to make each line more cohesive, like so:
From the previous articles in this series, we know that the `for` can turn into a `stream()` and the printing of the value can be done from within the `forEach()`. That leaves us with the transformation, the call to the `toUpperCase()` function. Such transformations can be done using the `map()` operation on the `stream()`.
The `map()` operation transforms the data to a different value based on the function invoked from within the lambda expression that's passed to `map()`. In this example, each name is transformed to its uppercase value. The transformed value is then printed out from within the lambda expression passed to `forEach()`.
We can make the code a bit more concise by using method references instead of lambda expressions, like so:
Use the `map()` function to transform data while iterating over a collection of data.

Contoh Kode / Perintah CLI:

```
List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

for(String name: names) {
  System.out.println(name.toUpperCase());
}

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

names.forEach(name -> System.out.println(name.toUpperCase())); //Don't do this

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

for(String name: names) {
  String nameInUpperCase = name.toUpperCase();
  System.out.println(nameInUpperCase);
}

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

names.stream()
  .map(name -> name.toUpperCase())
  .forEach(nameInUpperCase -> System.out.println(nameInUpperCase));

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

names.stream()
```

```
  .map(String::toUpperCase)
  .forEach(System.out::println);
```

[java_205] Java - Functional Programming - Map & Transformations

Converting Iteration with transformation: Picking Elements to Transform

Sumber: ht

Suppose, in the middle of the iteration, we want to pick some values from the collection based on some condition and apply a transformation only on those elements. For example, what if we want to transform and print only names of length 4? In the imperative style we could do the following:
We already know that the imperative style `if` can be refactored to the `filter()` function in the functional style. We can perform the transformation, using `map()`, after the `filter()` operation, like so:
The `filter()` function discards data that's not desired and passes on only the values we like to use. The `map()` function transforms the values it sees after the filter.

Contoh Kode / Perintah CLI:

```
List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

for(String name: names) {
  if(name.length() == 4) {
    System.out.println(name.toUpperCase());
  }
}

// ----------------------------------------

List<String> names = List.of("Jack", "Paula", "Kate", "Peter");

names.stream()
  .filter(name -> name.length() == 4)
  .map(String::toUpperCase)
  .forEach(System.out::println);
```

[java_206] Java - Functional Programming - Map & Transformations

Converting Iteration with transformation: Mappings

Sumber: ht

Anywhere you see transformation of data within a for loop, use the `map()` function to perform the transformation in the functional style. In addition, if the body of the loop has a `if` statement to selectively some value for transformation, then use the `stream()` API with the call to the `filter()` method before using the `map()` method.

[java_207] Java - Functional Programming - Stream Pipelines

Converting Data Sources to Streams: Thinking in Streams

Sumber: ht

In the previous articles in this tutorial series we looked at converting loops written in the imperative style to the functional style. In this article we'll look at viewing the source of data, through the functional eyes, as a stream of data and convert the iteration to use the Streams API.
We saw how we can use `filter()` and `map()` functions to select and transform data, respectively. We can perform these operations in the middle of the functional pipeline. In the examples in the previous articles we used functions like `range()` and `rangeClosed()` to create a stream of values in a range of numbers. That worked nicely when we want to iterate over a known range of values, but, often we may want to work with data that comes from external resources, like from a file, for example. If we are able to work with the external resource as a stream, then we can readily apply the operations of the functional pipeline. In this article we'll take a look at an example that illustrates that idea.

[java_208] Java - Functional Programming - Stream Pipelines

Converting Data Sources to Streams: From Imperative to Functional Style

Sumber: ht

Suppose we want to iterate over a file and count the number of lines with one or more occurrences of a word. Here's an all too familiear imperative style code to accomplish that task:
In order to make it easy to work with this example, we look for the number of lines with the word "public" in the same source file as the code resides. You may change the value of the `filePath` to refer to a different file and/or the value of the `wordOfInterest` to something else if you like.
There are two major parts in this example. We use the `BufferedReader` returned by the `newBufferedReader()` method to access the contents of the file we're interested in looking into. Then, in the `while` loop we check each line to see if it contains the desired word and, if so, increment the `count` to indicate we found another line with the word. Let's examine the two parts, with the second one first.
Looking closely at the loop, from our discussions in the previous articles, we can recognize that the presence of `if` is a sign that we may use the `filter()` operation if we can write the code as a functional pipeline. Once we filter out or select the lines with the desired word, we can count the number of lines, using the `count()` method of stream. You're most likely curious and bursting to ask, "but, where's the Stream?" To answer that question, let's take a look at the first part of the code.
The data, that is the lines of text, is coming from the file whose path is provided in the variable `filePath`. We're reading the data using the `BufferedReader`'s `readLine()` method and the imperative style to iterate over each line of text. In order to use the functional pipeline, with the operations like `filter()` we need a `Stream` of data. Hence the question, "is it possible to get a Stream of data for the contents in a file?"
The answer, thankfully, is a resounding yes. The developers behind the JDK and the Java language did not merely introduce the capability to do functional programming and say "good luck." They took the pains to enhance the JDK to add functions so we, as programmers, can make good use of the functional capabilities of Java for our routine tasks.
An easy way to turn the contents of a file into a stream of data is using the `lines()` method of the `Files` class that is part of the `java.nio.file` package. Let's refactor the previous imperative style code to the functional style, with the help of the `lines()` method that gives us the `Stream` over the contents of a file, like so:
Not only does the `lines()` method provide a stream of data over the contents of a file, it remove so much of the cruft around reading the lines. Instead of the external iterator where we fetched one line at a time, the stream makes it possible to use the internal iterator where we can focus on what to do for each line of text as it emerges in the stream's pipeline.

Contoh Kode / Perintah CLI:

```
//Sample.java
import java.nio.file.*;

public class Sample {
  public static void main(String[] args) {
    try {
      final var filePath = "./Sample.java";
      final var wordOfInterest = "public";

      try (var reader = Files.newBufferedReader(Path.of(filePath))) {
        String line = "";
        long count = 0;

        while((line = reader.readLine()) != null) {
          if(line.contains(wordOfInterest)) {
            count++;
          }
        }

        System.out.println(String.format("Found %d lines with the word %s", count, wordOfInterest));
      }
    } catch(Exception ex) {
      System.out.println("ERROR: " + ex.getMessage());
```

```
    }
  }
}

// ----------------------------------------

//Sample.java
import java.nio.file.*;

public class Sample {
  public static void main(String[] args) {
    try {
      final var filePath = "./Sample.java";
      final var wordOfInterest = "public";

      try(var stream = Files.lines(Path.of(filePath))) {
        long count = stream.filter(line -> line.contains(wordOfInterest))
          .count();

        System.out.println(String.format("Found %d lines with the word %s", count, wordOfInterest));
      }
    } catch(Exception ex) {
      System.out.println("ERROR: " + ex.getMessage());
    }
  }
}
```

[java_209] Java - Functional Programming - Stream Pipelines

Converting Data Sources to Streams: Mappings

Sumber: ht

Whenever you're working with a collection of data from an external resource, ask if there is a way to get a stream of data over the contents of that resource. The chances are that you may find a function for that within the JDK or a third-party library. Once we get a stream, we can use the highly effective functional operators like `filter()`, `map()`, etc. to fluently iterate over the collection of data that's part of the resource.

[java_210] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Overview & Metadata

Sumber: ht

<noscript></noscript>
TABLE { border-collapse: collapse; padding: 0px; margin: 1em 0 1em 2em; }
TR:first-child TH, TR:first-child TD { padding-top: 0; }
TH, TD { padding: 0px; padding-top: .5ex; vertical-align: baseline; text-align: left; }
TD + TD, TH + TH { padding-left: 1em; }
TD:first-child, TH:first-child, TD.jep { text-align: right; }
TABLE.head TD:first-child { font-style: italic; padding-left: 2em; white-space: nowrap; }
PRE { padding-left: 2em; margin: 1ex 0; font-size: inherit; }
TABLE PRE { padding-left: 0; margin: 0; }
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { padding-left: .5em; }
TABLE.jeps TD:first-child,
TABLE.jeps TD:first-child + TD,
TABLE.jeps TD:first-child + TD + TD { font-size: smaller; }
TABLE.jeps TD.cl { font-size: smaller; padding-right: 0; text-align: right; }
TABLE.jeps TD.cm { font-size: smaller; padding-left: .1em; padding-right: .1em; }

TABLE.jeps TD.cr { font-size: smaller; padding-left: 0; }
TABLE.jeps TD.z { padding-left: 0; padding-right: 0; }
TABLE.head TD { padding-top: 0; }
JEP 431: Sequenced Collections
Owner
Stuart Marks
Type
Feature
Scope
SE
Status
Closed / Delivered
Release
21
Component
core-libs / java.util:collections
Discussion
core dash libs dash dev at openjdk dot org
Reviewed by
Brian Goetz
Endorsed by
Brian Goetz
Created
2022/01/27 22:13
Updated
2023/10/23 17:55
Issue
8280836

[java_211] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Summary

Sumber: ht

Introduce new interfaces to represent collections with a defined encounter order. Each such collection has a well-defined first element, second element, and so forth, up to the last element. It also provides uniform APIs for accessing its first and last elements, and for processing its elements in reverse order.
"Life can only be understood backwards; but it must be lived forwards."
-- Kierkegaard

[java_212] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Motivation

Sumber: ht

Java's
collections framework
lacks a collection type that represents
a sequence of elements with a defined encounter order. It also lacks a uniform set of operations that apply across such collections. These gaps have been a repeated source of problems and complaints.
For example,
List

and
Deque
both define an encounter order but
their common supertype is
Collection
, which does not. Similarly,
Set
does not define an encounter order, and subtypes such as
HashSet
do not define one, but subtypes such as
SortedSet
and
LinkedHashSet
do. Support for encounter order is thus spread across
the type hierarchy, making it difficult to express certain useful
concepts in APIs. Neither
Collection
nor
List
can describe a
parameter or return value that has an encounter order.
Collection
is
too general, relegating such constraints to the prose specification,
possibly leading to hard-to-debug errors.
List
is too specific,
excluding
SortedSet
and
LinkedHashSet
.
A related problem is that view collections are often forced to
downgrade to weaker semantics. Wrapping a
LinkedHashSet
with
Collections::unmodifiableSet
yields a
Set
, discarding the
information about encounter order.
Without interfaces to define them, operations related to encounter
order are either inconsistent or missing. While many implementations
support getting the first or last element, each collection defines its
own way, and some are not obvious or are missing entirely:
First element
Last element
List
list.get(0)
list.get(list.size() - 1)
Deque
deque.getFirst()

deque.getLast()
SortedSet
sortedSet.first()
sortedSet.last()
LinkedHashSet
linkedHashSet.iterator().next()
// missing
Some of these are unnecessarily cumbersome, such as getting the last element of a
List
. Some are not even possible without heroics: The only way to get the last element of a
LinkedHashSet
is to iterate the entire set.
Similarly, iterating the elements of a collection from first to last is straightforward and consistent, but iterating in reverse order is neither. All of these collections can be iterated forward with an
Iterator
, the enhanced
for
loop, a
stream()
, or
toArray()
.
Iterating in reverse is different in every case.
NavigableSet
provides the
descendingSet()
view for reverse iteration:
Deque
does so with a reverse
Iterator
:
List
does so but with
ListIterator
:
LinkedHashSet
, finally, provides no support for reverse iteration.
The only practical way to process the elements of a
LinkedHashSet
in reverse order is to copy its elements into another collection.
Similarly, processing a collection's elements using streams is a powerful and effective alternative to processing elements using loops, but obtaining a stream in reverse order can be difficult. Of the various collections that define encounter order, the only one that supports this conveniently is
NavigableSet
:

The others require either copying the elements to another collection or creating a stream from a customized Spliterator that reverses iteration.
This is an unfortunate state of affairs. The concept of a collection with defined encounter order exists in multiple places in the collections framework, but there is no single type that represents it. As a result, some operations on such collections are inconsistent or missing, and processing elements in reverse order ranges from inconvenient to impossible. We should fill these gaps.

Contoh Kode / Perintah CLI:

```
for (var e : navSet.descendingSet())
    process(e);

// ----------------------------------------

for (var it = deque.descendingIterator(); it.hasNext();) {
    var e = it.next();
    process(e);
}

// ----------------------------------------

for (var it = list.listIterator(list.size()); it.hasPrevious();) {
    var e = it.previous();
    process(e);
}

// ----------------------------------------

navSet.descendingSet().stream()
```

[java_213] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Description

Sumber: ht

We define new interfaces for sequenced collections, sequenced sets, and sequenced maps, and then retrofit them into the existing collections type hierarchy. All of the new methods declared in these interfaces have default implementations.

[java_214] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Sequenced collections

Sumber: ht

A sequenced collection is a Collection whose elements have a defined encounter order. (The word "sequenced" as used here is the past participle of the verb to sequence, meaning "to arrange

elements in a particular order.") A sequenced collection has first and last elements, and the elements between them have successors and predecessors. A sequenced collection supports common operations at either end, and it supports processing the elements from first to last and from last to first (i.e., forward and reverse).

The new
reversed()
method provides a reverse-ordered view of the original collection. Any modifications to the original collection are visible in the view. If permitted, modifications to the view write through to the original collection.

The reverse-ordered view enables all the different sequenced types to process elements in both directions, using all the usual iteration mechanisms: Enhanced
for
loops, explicit
iterator()
loops,
forEach()
,
stream()
,
parallelStream()
, and
toArray()
.

For example, obtaining a reverse-ordered stream from a
LinkedHashSet
was previously quite difficult; now it is simply
(The
reversed()
method is essentially a renamed
NavigableSet::descendingSet
, promoted to
SequencedCollection
.)

The following methods of
SequencedCollection
are promoted from
Deque
. They support adding, getting, and removing elements at both ends:

void addFirst(E)
void addLast(E)
E getFirst()
E getLast()
E removeFirst()
E removeLast()

The
add*(E)
and
remove*()

methods are optional, primarily to
support the case of unmodifiable collections. The
get*()
and
remove*()
methods throw
NoSuchElementException
if the collection
is empty.
There are no definitions of
equals()
and
hashCode()
in
SequencedCollection
because its sub-interfaces have conflicting
definitions.

Contoh Kode / Perintah CLI:

```
interface SequencedCollection<E> extends Collection<E> {
    // new method
    SequencedCollection<E> reversed();
    // methods promoted from Deque
    void addFirst(E);
    void addLast(E);
    E getFirst();
    E getLast();
    E removeFirst();
    E removeLast();
}

// ----------------------------------------

linkedHashSet.reversed().stream()
```

[java_215] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Sequenced sets

Sumber: ht

A
sequenced set
is a
Set
that is a
SequencedCollection
that
contains no duplicate elements.
Collections such as
SortedSet
, which position elements by relative
comparison, cannot support explicit-positioning operations such as the
addFirst(E)
and
addLast(E)
methods declared in the

SequencedCollection
superinterface. Thus, these methods can throw
UnsupportedOperationException
.
The
addFirst(E)
and
addLast(E)
methods of
SequencedSet
have
special-case semantics for collections such as
LinkedHashSet
: If the
element is already present in the set then it is moved to the
appropriate position. This remedies a long-standing deficiency in
LinkedHashSet
, namely the inability to reposition elements.

Contoh Kode / Perintah CLI:

```
interface SequencedSet<E> extends Set<E>, SequencedCollection<E> {
    SequencedSet<E> reversed();    // covariant override
}
```

[java_216] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Sequenced maps

Sumber: ht

A
sequenced map
is a
Map
whose entries have a defined encounter
order.
The new
put*(K, V)
methods have special-case semantics, similar to
the corresponding
add*(E)
methods of
SequencedSet
: For maps such
as
LinkedHashMap
, they have the additional effect of repositioning
the entry if it is already present in the map. For maps such as
SortedMap
, these methods throw
UnsupportedOperationException
.
The following methods of
SequencedMap
are promoted from

NavigableMap
. They support getting and removing entries at both
ends:
Entry<K, V> firstEntry()
Entry<K, V> lastEntry()
Entry<K, V> pollFirstEntry()
Entry<K, V> pollLastEntry()

Contoh Kode / Perintah CLI:

```
interface SequencedMap<K,V> extends Map<K,V> {
    // new methods
    SequencedMap<K,V> reversed();
    SequencedSet<K> sequencedKeySet();
    SequencedCollection<V> sequencedValues();
    SequencedSet<Entry<K,V>> sequencedEntrySet();
    V putFirst(K, V);
    V putLast(K, V);
    // methods promoted from NavigableMap
    Entry<K, V> firstEntry();
    Entry<K, V> lastEntry();
    Entry<K, V> pollFirstEntry();
    Entry<K, V> pollLastEntry();
}
```

[java_217] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Retrofitting

Sumber: ht

The three new interfaces defined above fit neatly into the existing
collections type hierarchy (click to enlarge):
In detail, we make the following adjustments to retrofit existing
classes and interfaces:
List
now has
SequencedCollection
as its immediate superinterface,
Deque
now has
SequencedCollection
as its immediate superinterface,
LinkedHashSet
additionally implements
SequencedSet
,
SortedSet
now has
SequencedSet
as its immediate superinterface,
LinkedHashMap
additionally implements
SequencedMap
, and
SortedMap
now has
SequencedMap

as its immediate superinterface.
We define covariant overrides for the
reversed()
method in the
appropriate places. For example,
List::reversed
is overridden to
return a value of type
List
rather than a value of type
SequencedCollection
.
We also add new methods to the
Collections
utility class to create
unmodifiable wrappers for the three new types:
Collections.unmodifiableSequencedCollection(sequencedCollection)
Collections.unmodifiableSequencedSet(sequencedSet)
Collections.unmodifiableSequencedMap(sequencedMap)

[java_218] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Types

Sumber: ht

An alternative to adding new types would be to repurpose the
List
interface as a general sequenced collection type. Indeed
List
is
sequenced, but it also supports element access by integer index. Many
sequenced data structures do not naturally support indexing and would
thus be required to support it iteratively. This would result in indexed
access having O(n) performance instead of the expected O(1), perpetuating
the mistake of
LinkedList
.
Deque
seems promising as a general sequence type, since it already
supports the right set of operations. However, it is cluttered with other
operations, including a family of null-returning operations (offer, peek,
and poll), stack operations (push and pop), and operations inherited from
Queue
. These operations are sensible for a queue but less so for other
collections. If
Deque
were repurposed as a general sequence type then
List
would also be a
Queue
and would support stack operations,
resulting in a cluttered and confusing API.

[java_219] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Naming

Sumber: ht

The term
sequence
, which we have chosen here, implies elements that
are arranged in order. It is commonly used across various platforms to
represent collections with semantics similar to those described above.
The term
ordered
is not quite specific enough. We require iteration
in both directions, and operations at both ends. An ordered collection
such as a
Queue
is a notable outlier: It is ordered, but it is also
decidedly asymmetric.
The term
reversible
, used in an earlier version of this proposal,
does not immediately evoke the concept of having two ends. Perhaps a
bigger issue is that the
Map
variant would be named
ReversibleMap
,
which misleadingly implies that it supports lookup by key and by value
(sometimes called a
BiMap
or
BidiMap
).

[java_220] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Add, put, and UnsupportedOperationException

Sumber: ht

As described above, explicit-positioning APIs such as
SortedSet::addFirst
and
SortedMap::putLast
throw
UnsupportedOperationException
because the sequence of their elements
is determined by relative comparison. The asymmetry of having some
collections not implement all of the
SequencedCollection
operations
may seem unpleasant. It is nonetheless valuable because it brings
SortedSet
and
SortedMap
into the sequenced collection family,

allowing them to be used more broadly than otherwise. This asymmetry is, also, consistent with prior design decisions in the collections framework. For example, the Map::keySet method returns a Set, even though the implementation returned does not support addition. Alternatively, the addition operations could be kept separate by rearranging the interfaces along structural lines. That would result in new interface types with very thin semantics (e.g., AddableCollection) that are not useful in practice and that clutter up the type hierarchy.

[java_221] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: History

Sumber: ht

This proposal is an incremental evolution of our 2021 ReversibleCollections proposal. The major changes from that proposal are renaming, the addition of the SequencedMap interface, and the addition of unmodifiable wrapper methods. The ReversibleCollection proposal was in turn based on Tagir Valeev's 2020 OrderedMap/OrderedSet proposal. Several fundamental concepts from that proposal are still present, although there are many differences in detail. Over the years we have received many requests and proposals in the vein of combining a List with a Set or Map. The recurring themes are a List that contains unique elements, or a Set or Map that maintains ordering. These requests include

4152834
,
4245809
,
4264420
,
4268146
,
6447049
, and
8037382
.
Some of these requests were partially addressed with the introduction of
LinkedHashSet
and
LinkedHashMap
in Java 1.4. While those classes do satisfy some use cases, their introduction left gaps in the abstractions and operations provided by the collections framework, as described above.

[java_222] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Testing

Sumber: ht

We will add a comprehensive set of tests to the JDK's regression test suite.

[java_223] Java - Collections - Sequenced Collections

JEP 431: Sequenced Collections: Risks and Assumptions

Sumber: ht

Introducing new methods high in the inheritance hierarchy runs the risk of clashes over obvious method names such as
reversed()
and
getFirst()
.
Of particular concern are the covariant overrides of the
reversed()
method on
List
and
Deque
. These are source and binary incompatible with existing collections that implement both
List
and
Deque
. There are two examples of such collections in the JDK:
LinkedList
and an internal class

sun.awt.util.IdentityLinkedList
.
The
LinkedList
class was handled by introducing a new
reversed()
covariant override on
LinkedList
itself. The internal
IdentityLinkedList
class was removed as it was no longer necessary.
An earlier version of the proposal introduced covariant overrides for the
keySet()
,
values()
, and
entrySet()
methods of the
SequencedMap
interface. After some
analysis it was determined that this approach introduced too great a risk of incompatibilities;
essentially, it invalidates any existing subclasses. An alternative approach was selected,
which was to introduce new methods
sequencedKeySet()
,
sequencedValues()
, and
sequencedEntrySet()
into
SequencedMap
instead of adjusting the existing methods to
be covariant overrides. In retrospect, it may have been for the same reason that a similar
approach was taken in Java 6 with the introduction of the
navigableKeySet()
method instead of modifying the existing
keySet()
method to be a covariant override.
See the report attached to the CSR,
JDK-8266572
,
for a full analysis of the incompatibility risk.
Installing
Contributing
Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds
Mailing lists
Wiki
*
IRC
Mastodon

Bluesky
Bylaws
*
Census
Legal
*
AI
Workshop
JEP Process
Source code
GitHub
Mercurial
Tools
Git
jtreg harness
Groups
(overview
,
archive
)
Adoption
Build
Client Libraries
Compatibility & Specification Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
Members
Networking
Porters
Quality
Security
Serviceability
Vulnerability
Web
Projects
(
overview
,
archive
)
Amber
Babylon
Brisbane
CRaC
Code Tools
Coin
Common VM Interface

Detroit
Developers' Guide
Duke
IcedTea
JDK 8 Updates
JDK 9
JDK
(...,
26
,
27
,
28
)
JDK Updates
JMC
Jigsaw
Lanai
Leyden
Lilliput
Loom
Memory Model Update
Multi-Language VM
Nashorn
New I/O
OpenJFX
Panama
Port: AArch32
Port: AArch64
Port: BSD
Port: Haiku
Port: MIPS
Port: Mobile
Port: PowerPC/AIX
Port: RISC-V
Port: s390x
SCTP
Shenandoah
Skara
Sumatra
Tsan
Valhalla
Wakefield
Zero
ZGC
(c) 2026 Oracle Corporation and/or its affiliates
Terms of Use
*

License:
GPLv2
*

Privacy
*
Trademarks

[java_224] Java - Standard Library - Modern I/O

Common I/O Tasks in Modern Java: Introduction

Sumber: ht

This article focuses on tasks that application programmers are likely to encounter, particularly in web applications, such as:
* Reading and writing text files
* Reading text, images, JSON from the web
* Visiting files in a directory
* Reading a ZIP file
* Creating a temporary file or directory
The Java API supports many other tasks, which are explained in detail in the Java I/O API tutorial.
This article focuses on API improvements since Java 8. In particular:
* UTF-8 is the default for I/O since Java 18 (since UTF-8 by Default)
* The `java.nio.file.Files` class, which first appeared in Java 7, added useful methods in Java 8, 11, and 12
* `java.io.InputStream` gained useful methods in Java 9, 11, and 12
* The `java.io.File` and `java.io.BufferedReader` classes are now thoroughly obsolete, even though they appear frequently in web searches and AI chats.

[java_225] Java - Standard Library - Modern I/O

Common I/O Tasks in Modern Java: Reading Text Files

Sumber: ht

You can read a text file into a string like this:
Here, `path` is an instance of `java.nio.Path`, obtained like this:
Before Java 18, you were strongly encouraged to specify the character encoding with any file operations that read or write strings. Nowadays, by far the most common character encoding is UTF-8, but for backwards compatibility, Java used the "platform encoding", which can be a legacy encoding on Windows. To ensure portability, text I/O operations needed parameters `StandardCharsets.UTF_8`. This is no longer necessary.
If you want the file as a sequence of lines, call
If the file is large, process the lines lazily as a `Stream<String>`:
Also use `Files.lines`) if you can naturally process lines with stream operations (such as `map`), `filter`)). Note that the stream returned by `Files.lines`) needs to be closed. To ensure that this happens, use a _try-with-resources_ statement, as in the preceding code snippet.
There is no longer a good reason to use the `readLine`) method of `java.io.BufferedReader`.
To split your input into something else than lines, use a `java.util.Scanner`. For example, here is how you can read words, separated by non-letters:
The `Scanner` class also has methods for reading numbers, but it is generally simpler to read the input as one string per line, or a single string, and then parse it.
Be careful when parsing numbers from text files, since their format may be locale-dependent. For example, the input `100.000` is 100.0 in the US locale but 100000.0 in the German locale. Use `java.text.NumberFormat` for locale-specific parsing. Alternatively, you may be able to use `Integer.parseInt`)/`Double.parseDouble`).

Contoh Kode / Perintah CLI:

```
String content = Files.readString(path);

// ----------------------------------------

var path = Path.of("/usr/share/dict/words");
```

```
// ----------------------------------------

List<String> lines = Files.readAllLines(path);

// ----------------------------------------

try (Stream<String> lines = Files.lines(path)) {
    . . .
}

// ----------------------------------------

Stream<String> tokens = new Scanner(path).useDelimiter("\\PL+").tokens();
```

[java_226] Java - Standard Library - Modern I/O

Common I/O Tasks in Modern Java: Writing Text Files

Sumber: ht

You can write a string to a text file with a single call:
If you have a list of lines rather than a single string, use:
For more general output, use a `PrintWriter` if you want to use the `printf`) method:
Note that `printf`) is locale-specific. When writing numbers, be sure to write them in the appropriate format. Instead of using `printf`), consider `java.text.NumberFormat` or `Integer.toString`)/`Double.toString`).
Weirdly enough, as of Java 21, there is no `PrintWriter` constructor with a `Path` parameter.
If you don't use `printf`), you can use the `BufferedWriter` class and write strings with the `write`) method.
Remember to close the `writer` when you are done.

Contoh Kode / Perintah CLI:

```
String content = . . .;
Files.writeString(path, content);

// ----------------------------------------

List<String> lines = . . .;
Files.write(path, lines);

// ----------------------------------------

var writer = new PrintWriter(path.toFile());
writer.printf(locale, "Hello, %s, next year you'll be %d years old!%n", name, age + 1);

// ----------------------------------------

var writer = Files.newBufferedWriter(path);
writer.write(line); // Does not write a line separator
writer.newLine();
```

[java_227] Java - Standard Library - Modern I/O

Common I/O Tasks in Modern Java: Reading From an Input Stream

Sumber: ht

Perhaps the most common reason to use a stream is to read something from a web site.
If you need to set request headers or read response headers, use the `HttpClient`:
That is overkill if all you want is the data. Instead, use:
Then read the data into a byte array and optionally turn them into a string:
Or transfer the data to an output stream:
Note that no loop is required if you simply want to read all bytes of an input stream.
But do you really need an input stream? Many APIs give you the option to read from a file or URL.

Your favorite JSON library is likely to have methods for reading from a file or URL. For example, with Jackson jr:
Here is how to read the dog image from the preceding call:
This is better than passing an input stream to the `read`) method, because the library can use additional information from the URL to determine the image type.

Contoh Kode / Perintah CLI:

```
HttpClient client = HttpClient.newBuilder().build();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://horstmann.com/index.html"))
    .GET()
    .build();
HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
String result = response.body();

// ----------------------------------------

InputStream in = new URI("https://horstmann.com/index.html").toURL().openStream();

// ----------------------------------------

byte[] bytes = in.readAllBytes();
String result = new String(bytes);

// ----------------------------------------

try(OutputStream out = Files.newOutputStream(path)) {
    in.transferTo(out);
}

// ----------------------------------------

URL url = new URI("https://dog.ceo/api/breeds/image/random").toURL();
Map<String, Object> result = JSON.std.mapFrom(url);

// ----------------------------------------

URL url = new URI(result.get("message").toString()).toURL();
BufferedImage img = javax.imageio.ImageIO.read(url);
```

[java_228] Java - Standard Library - Modern I/O

Common I/O Tasks in Modern Java: The Files API

Sumber: ht

The `java.nio.file.Files` class provides a comprehensive set of file operations, such as creating, copying, moving, and deleting files and directories. The File System Basics tutorial provides a thorough description. In this section, I highlight a few common tasks.
Traversing Entries in Directories and Subdirectories
For most situations you can use one of two methods. The `Files.list`) method visits all entries (files, subdirectories, symbolic links) of a directory.
Use a _try-with-resources_ statement to ensure that the stream object, which keeps track of the iteration, will be closed.
If you also want to visit the entries of descendant directories, instead use the method `Files.walk`)
Then simply use stream methods to home in on the entries that you are interested in, and to collect the results:
Here are the other methods for traversing directory entries:
* An overloaded version of `Files.walk`) lets you limit the depth of the traversed tree.
* Two `Files.walkFileTree`) methods provide more control over the iteration process, by notifying a `FileVisitor` when a directory is visited for the first and last time. This can be occasionally useful, in particularly for emptying and deleting a tree of directories. See the tutorial Walking the File Tree for details. Unless you need this control, use the simpler `Files.walk`) method.

* The `Files.find`) method is just like `Files.walk`), but you provide a filter that inspects each path and its `BasicFileAttributes`. This is slightly more efficient than reading the attributes separately for each file.
* Two `Files.newDirectoryStream(Path)`) methods yield `DirectoryStream` instances, which can be used in enhanced `for` loops. There is no advantage over using `Files.list`).
* The legacy `File.list`) or `File.listFiles`) methods return file names or `File` objects. These are now obsolete.

Working with ZIP Files

Ever since Java 1.1, the `ZipInputStream` and `ZipOutputStream` classes provide an API for processing ZIP files. But the API is a bit clunky. Java 8 introduced a much nicer *ZIP file system*:

The _try-with-resources_ statement ensures that the `close`) method is called after the ZIP file operations. That method updates the ZIP file to reflect any changes in the file system.

You can then use the methods of the `Files` class. Here we get a list of all files in the ZIP file:

To read the file contents, just use `Files.readString`) or `Files.readAllBytes`):

You can remove files with `Files.delete`). To add or replace files, simply use `Files.writeString`) or `Files.write`).

Creating Temporary Files and Directories

Fairly often, I need to collect user input, produce files, and run an external process. Then I use temporary files, which are gone after the next reboot, or a temporary directory that I erase after the process has completed.

I use the two methods `Files.createTempFile`) and `Files.createTempDirectory`) for that.

This creates a temporary file or directory in a suitable location (`/tmp` in Linux) with the given prefix and, for a file, suffix.

Contoh Kode / Perintah CLI:

```
try (Stream<Path> entries = Files.list(pathToDirectory)) {
    . . .
}

// ----------------------------------------

Stream<Path> entries = Files.walk(pathToDirectory);

// ----------------------------------------

try (Stream<Path> entries = Files.walk(pathToDirectory)) {
    List<Path> htmlFiles = entries.filter(p -> p.toString().endsWith("html")).toList();
    . . .
}

// ----------------------------------------

try (FileSystem fs = FileSystems.newFileSystem(pathToZipFile)) {
    . . .
}

// ----------------------------------------

try (Stream<Path> entries = Files.walk(fs.getPath("/"))) {
    List<Path> filesInZip = entries.filter(Files::isRegularFile).toList();
}

// ----------------------------------------

String contents = Files.readString(fs.getPath("/LICENSE"));

// ----------------------------------------

Path filePath = Files.createTempFile("myapp", ".txt");
Path dirPath = Files.createTempDirectory("myapp");
```

[java_229] Java - Standard Library - Modern I/O

Common I/O Tasks in Modern Java: Conclusion

Sumber: ht

Web searches and AI chats can suggest needlessly complex code for common I/O operations. There are often better alternatives:

1. You don't need a loop to read or write strings or byte arrays.
2. You may not even need a stream, reader or writer.
3. Become familiar with the `Files` methods for creating, copying, moving, and deleting files and directories.
4. Use `Files.list`) or `Files.walk`) to traverse directory entries.
5. Use a ZIP file system for processing ZIP files.
6. Stay away from the legacy `File` class.